

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN TIN



ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn pháp luật sử dụng
công nghệ trí tuệ nhân tạo

Chuyên ngành: Toán tin

Giảng viên hướng dẫn:	TS. Trần Anh Tú
Sinh viên thực hiện:	Vũ Văn Nghĩa
Mã số sinh viên:	20206205
Lớp:	Toán Tin 02 - K65

Hà Nội, 2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN TIN



ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn pháp luật sử dụng
công nghệ trí tuệ nhân tạo

Chuyên ngành: Toán tin

Giảng viên hướng dẫn:	TS. Trần Anh Tú
Sinh viên thực hiện:	Vũ Văn Nghĩa
Mã số sinh viên:	20206205
Lớp:	Toán Tin 02 - K65

Hà Nội, 2026

Lời nói đầu

Qua thời gian học tập và rèn luyện tại *Đại học Bách khoa Hà Nội*, đến nay em đã hoàn thành đồ án tốt nghiệp. Trong thời gian học tập để có được kết quả hiện tại em xin chân thành cảm ơn tới các thầy cô giảng viên trong *Khoa Toán Tin* đã giảng dạy, quan tâm và tạo điều kiện thuận lợi để em học tập rèn luyện trong suốt thời gian qua, giúp em trang bị những kiến thức, kỹ năng cần thiết để em có thể hoàn thành tốt nhất báo cáo này với đề tài: *Xây dựng ứng dụng hỗ trợ tư vấn pháp luật sử dụng công nghệ trí tuệ nhân tạo*.

Đồng thời, em xin gửi lời cảm ơn chân thành và sâu sắc đến *Thầy TS. Trần Anh Tú*, người thầy đã tận tình hỗ trợ, giúp đỡ và hướng dẫn em suốt thời gian thực hiện đồ án. Những kiến thức và kinh nghiệm mà em đã tiếp thu được trong quá trình này sẽ đóng góp quan trọng vào sự phát triển và thành công của em trong tương lai. Em xin gửi lời chúc sức khỏe tốt nhất đến thầy, hy vọng thầy luôn dồi dào sức khỏe, đam mê và nhiệt huyết trong công việc giảng dạy.

Em cũng xin gửi lời cảm ơn tới bố mẹ em đã dành nhiều thời gian chăm lo, động viên và giúp đỡ em để em đạt được thành công trong học tập.

Mặc dù đã cố gắng nỗ lực học hỏi để hoàn thành đồ án, vậy nhưng thời gian thực hiện đồ án có hạn, kiến thức của em còn hạn chế. Do vậy, không tránh khỏi những thiếu sót, em rất mong nhận được những ý kiến đóng góp quý báu của thầy cô để em có thể cải thiện một cách tốt nhất.

Em xin chân thành cảm ơn!

Tóm tắt nội dung đề án

Tóm tắt nội dung của đồ án tốt nghiệp trong khoảng tối đa 300 chữ. Phần tóm tắt cần nêu được các ý: vấn đề cần thực hiện; phương pháp thực hiện; công cụ sử dụng (phần mềm, phần cứng...); kết quả của đồ án có phù hợp với các vấn đề đã đặt ra hay không; tính thực tế của đồ án, định hướng phát triển mở rộng của đồ án (nếu có); các kiến thức và kỹ năng mà sinh viên đã đạt được. Các chương trong báo cáo được trình bày theo thứ tự sau:

- Giới thiệu về công ty
Báo cáo này bắt đầu với phần giới thiệu về công ty, bao gồm: lịch sử hình thành, sứ mệnh, slogan, linh vật biểu tượng, giá trị cốt lõi, các sản phẩm nổi bật, ...
- Dự án bài đánh giá kỹ năng Backend ban đầu
Chương này đề cập đến các kiến thức và kỹ năng có liên quan và áp dụng cho dự án thực tế. Đặc biệt là ứng dụng mật mã TOTP xác thực đa yếu tố cho hệ thống.
- Dự án Kong API Gateway
Chương này đề cập đến các kiến thức về API Gateway và kiến thức về Kong API Gateway. Từ đó, sử dụng Kong xây dựng API Gateway cho hệ thống microservices.
- Dự án thực tế PXDR
Chương cuối cùng này đề cập đến dự án thực tế mà em được tham gia thực tập. Một số công việc nổi bật em đã học tập, tìm hiểu và tham gia phát triển trong dự án.

Hà Nội, Ngày 15 tháng 6 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Nghĩa

Vũ Văn Nghĩa

Mục	lục
Danh sách viết tắt	6
Danh sách hình vẽ	7
1 Tổng quan về hệ thống	11
1.1 Sự cần thiết	11
1.2 Khảo sát công nghệ xây dựng cốt lõi	11
1.2.1 Mô hình ngôn ngữ lớn LLM (Large Language Model)	11
1.2.2 Tạo tăng cường truy xuất RAG (Retrieval Augmented Generation)	13
1.2.3 Kiến trúc vi dịch vụ MSA (Microservice Architecture)	16
1.2.4 Thiết kế hướng miền DDD (Domain Driven Design)	17
1.3 Mục tiêu	19
2 Phân tích thiết kế hệ thống	20
2.1 Bài toán nghiệp vụ	20
2.1.1 Miền phụ cốt lõi (Core Subdomain)	20
2.1.2 Miền phụ hỗ trợ (Supporting Subdomain)	20
2.1.3 Miền phụ chung (Generic Subdomain)	21
2.2 Phân tích thiết kế hệ thống	22
2.2.1 Sơ đồ Use Case	22
2.2.2 Sơ đồ tuần tự (Sequence Diagram)	23
2.3 Kiến trúc hệ thống	23
2.3.1 Quản lý các thông tin bí mật (doppler)	23
2.3.2 CI/CD (Github Action)	24
2.3.3 Cơ sở hạ tầng dưới dạng mã (terraform)	25
2.3.4 API Gateway trong Microservice (Kong API Gateway)	28
2.3.5 Container và GitOps (Docker, Kubernetes, ArgoCD)	30
2.3.6 Giao tiếp giữa các services	31
2.4 Đường ống dữ liệu	31
2.4.1 Đường ống dữ liệu Văn bản pháp luật	33
2.4.2 Đường ống dữ liệu Pháp Điển	35
3 Triển khai hệ thống	39
3.1 Triển khai các dịch vụ	39
3.2 Tổng quan các Microservice của hệ thống	39
3.2.1 Tracing và Quan sát hệ thống (Observability)	41
3.3 Dịch vụ xác thực (auth service - dùng Supabase)	42
3.3.1 I. Xác thực và Quản lý tài khoản	42
3.3.2 II. Xác thực 2 yếu tố (MFA)	43
3.3.3 III. Quản lý Hồ sơ người dùng (Supabase Database - PostgREST API)	44
3.3.4 IV. Quản lý Ảnh đại diện (Supabase Storage)	44
3.3.5 Cấu hình trong Kong API Gateway	48
3.4 Dịch vụ cài đặt (setting service - dùng Neon Data API)	48
3.4.1 Cấu hình trong Kong API Gateway	50
3.5 Dịch vụ thanh toán (payment service)	50
3.5.1 I. Quản lý Gói dịch vụ (Plans)	51
3.5.2 II. Quản lý Thuê bao (Subscriptions) và Giao dịch	51
3.5.3 III. Tích hợp Cổng thanh toán	52
3.5.4 Sự kiện Thanh toán thành công (Subscription Purchased Event)	55
3.5.5 Cơ chế Instant Payment Notification (IPN)	56
3.5.6 Thiết kế hệ thống theo Domain-Driven Design (DDD)	57
3.6 Dịch vụ văn bản (document service)	60
3.6.1 Document Worker	61
3.6.2 Cơ chế tối ưu & Độ bền bỉ (Optimization & Resilience)	61
3.7 Dịch vụ nhân vật (persona service)	64
3.7.1 Các chức năng chính	64
3.8 Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service)	66
3.8.1 Dịch vụ trò chuyện (conversation service)	67

3.8.2	Dịch vụ chatbot (chatbot service)	71
3.8.3	Hệ thống trao đổi thông điệp qua RabbitMQ (RabbitMQ Messaging)	72
3.8.4	Chi tiết chức năng đàm thoại giọng nói (Voice Agent)	77
3.8.5	Cơ chế Lập lịch Dọn dẹp dữ liệu (Data Cleanup Scheduling)	80
Kết luận		82
Tài liệu tham khảo		83

Danh sách viết tắt

STT	Từ viết tắt	Từ viết đầy đủ	Mô tả
1	2FA	Two-Factor Authentication	Xác thực hai yếu tố
2	AI	Artificial Intelligence	Trí tuệ nhân tạo
3	API	Application Programming Interface	Giao diện lập trình ứng dụng
4	CD	Continuous Delivery / Deployment	Chuyển giao/Triển khai liên tục
5	CI	Continuous Integration	Tích hợp liên tục
6	CLI	Command Line Interface	Giao diện dòng lệnh
7	CSDL	Cơ sở dữ liệu	Cơ sở dữ liệu
8	DDD	Domain Driven Design	Thiết kế hướng miền
9	DLQ	Dead Letter Queue	Hàng đợi thư lỗi
10	GPU	Graphics Processing Unit	Bộ phận xử lý đồ họa
11	HTML	Hyper Text Markup Language	Ngôn ngữ Đánh dấu Siêu văn bản
12	IPN	Instant Payment Notification	Thông báo thanh toán tức thì
13	IaC	Infrastructure as Code	Cơ sở hạ tầng dưới dạng mã
14	JSON	JavaScript Object Notation	Một kiểu định dạng dữ liệu key - value
15	JWT	JSON Web Token	Tiêu chuẩn truyền tải thông tin JSON
16	K8s	Kubernetes	Kubernetes
17	LLM	Large Language Model	Mô hình ngôn ngữ lớn
18	MFA	Multi-Factor Authentication	Xác thực đa yếu tố
19	MSA	Microservice Architecture	Kiến trúc vi dịch vụ
20	NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
21	PSC	Payment Service Consumer	Đơn vị sử dụng dịch vụ thanh toán
22	PSP	Payment Service Provider	Đơn vị cung cấp dịch vụ thanh toán
23	RAG	Retrieval Augmented Generation	Tạo tăng cường truy xuất
24	RLS	Row-Level Security	Bảo mật cấp hàng
25	SMTP	Simple Mail Transfer Protocol	Giao thức truyền thư đơn giản
26	SQL	Structured Query Language	Ngôn ngữ truy vấn cấu trúc
27	SSE	Server-sent events	Sự kiện máy chủ gửi
28	STT	Speech to Text	Chuyển giọng nói thành văn bản
29	TOTP	Time-based One-Time Password	Mật khẩu dùng một lần dựa trên thời gian
30	TTS	Text to Speech	Chuyển văn bản thành giọng nói
31	UI	User Interface	Giao diện người dùng
32	URL	Uniform Resource Locator	Định danh tài nguyên Internet
33	VIP	Very Important Person	Người rất quan trọng
34	gRPC	gRPC Remote Procedure Call	Framework Gọi thủ tục từ xa do Google phát triển

Danh	sách	hình	vẽ
Hình 1.1	AI hỗ trợ tư vấn pháp luật		11
Hình 1.2	Mô phỏng quá trình ánh xạ các vector từ gần nghĩa.		12
Hình 1.3	Cách hoạt động của LLM		12
Hình 1.4	Một số nhược điểm của LLM		13
Hình 1.5	Sơ đồ RAG cơ bản (Extracting and Indexing)		13
Hình 1.6	Sơ đồ RAG cơ bản (Retrieval and Generation)		14
Hình 1.7	Các khái niệm quan trọng của LangGraph		15
Hình 1.8	Sơ đồ LangGraph của dự án		15
Hình 1.9	Giới thiệu Microservice		17
Hình 1.10	Kiến trúc tổng quan các dịch vụ của dự án		17
Hình 1.11	Các loại miền phụ trong Domain Driven Design		18
Hình 1.12	Tổng quan kiến trúc lục giác		19
Hình 2.1	Sơ đồ tổng quan SubDomain		20
Hình 2.2	Sơ đồ phân rã chức năng		21
Hình 2.3	Sơ đồ Use Case		22
Hình 2.4	Sơ đồ quản lý secret bằng doppler		24
Hình 2.5	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		24
Hình 2.6	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		24
Hình 2.7	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		25
Hình 2.8	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		25
Hình 2.9	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		25
Hình 2.10	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		26
Hình 2.11	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		26
Hình 2.12	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		26
Hình 2.13	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		27
Hình 2.14	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		27
Hình 2.15	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		28
Hình 2.16	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		28
Hình 2.17	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		28
Hình 2.18	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		28
Hình 2.19	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		29
Hình 2.20	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		29
Hình 2.21	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		30
Hình 2.22	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		30
Hình 2.23	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		30
Hình 2.24	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		30
Hình 2.25	Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler		31
Hình 2.26	Giao diện nguồn dữ liệu Văn bản pháp luật		31
Hình 2.27	Giao diện nguồn dữ liệu Pháp Điển		32
Hình 2.28	Sơ đồ thu thập dữ liệu chung		32
Hình 2.29	Sơ đồ thu thập dữ liệu chung		32
Hình 2.30	Giao diện nguồn dữ liệu Văn bản pháp luật		33
Hình 2.31	Data Load Tool tự động thích ứng nguồn dữ liệu có sự thay đổi		33
Hình 2.32	Sơ đồ thu thập dữ liệu chung		33
Hình 2.33	Sơ đồ thu thập dữ liệu chung		33
Hình 2.34	Sơ đồ thu thập dữ liệu chung		34
Hình 2.35	Sơ đồ thu thập dữ liệu chung		35
Hình 2.36	Sơ đồ thu thập dữ liệu chung		35
Hình 2.37	Sơ đồ thu thập dữ liệu chung		36
Hình 2.38	Sơ đồ thu thập dữ liệu chung		36
Hình 2.39	Sơ đồ thu thập dữ liệu chung		36
Hình 2.40	Sơ đồ thu thập dữ liệu chung		36
Hình 2.41	Sơ đồ thu thập dữ liệu chung		37
Hình 2.42	Sơ đồ thu thập dữ liệu chung		37
Hình 2.43	swagger		38
Hình 2.44	swagger		38
Hình 2.45	swagger		38

Hình 2.46	swagger	38
Hình 3.1	Sơ đồ tổng quan các Microservice	39
Hình 3.2	Bảng điều khiển cơ sở dữ liệu Neon	39
Hình 3.3	Dự án trên Vercel	40
Hình 3.4	Thống kê độ bao phủ kiểm thử trên Codecov	40
Hình 3.5	Giao diện phân tích mã nguồn trên SonarQube	41
Hình 3.6	Giao diện theo dõi vết distributed tracing trên Honeycomb	42
Hình 3.7	Thông tin JWT Payload từ Supabase	44
Hình 3.8	Sơ đồ luồng xử lý xác thực MFA	45
Hình 3.9	Tổng quan cấu hình Terraform quản lý hạ tầng Supabase	46
Hình 3.10	Mẫu email HTML được cấu hình	46
Hình 3.11	Sơ đồ cấu hình Terraform cho Supabase	47
Hình 3.12	Cấu hình các Storage Bucket trên Supabase	47
Hình 3.13	Cấu hình bảng cơ sở dữ liệu trên Supabase	48
Hình 3.14	Quy trình kiểm tra bảo mật trong gói thư viện dùng chung	48
Hình 3.15	Kích hoạt Data API trên Neon Console	49
Hình 3.16	Thông tin URL REST API trên Neon Console	49
Hình 3.17	Tích hợp cấu hình xác thực Supabase trong Neon Console	50
Hình 3.18	Bật RLS cho bảng settings trên Neon Console	50
Hình 3.19	Sơ đồ tích hợp thanh toán và sự kiện Kafka	51
Hình 3.20	Tài liệu Swagger API Dịch vụ thanh toán	52
Hình 3.21	Trạng thái deploy dịch vụ thanh toán trên Heroku	53
Hình 3.22	Kiểm tra Pull Request trên GitHub	53
Hình 3.23	Quy trình chạy Test trong GitHub Actions	54
Hình 3.24	Quản lý database migrations trên GitHub Actions	54
Hình 3.25	Cấu hình các bảng cơ sở dữ liệu của dịch vụ thanh toán	55
Hình 3.26	Sơ đồ nguyên lý hoạt động của IPN	56
Hình 3.27	Mô phỏng luồng cập nhật trạng thái qua IPN	57
Hình 3.28	Giao diện thanh toán VNPAY	57
Hình 3.29	Giao diện thanh toán MoMo	58
Hình 3.30	Giao diện thanh toán ZaloPay	58
Hình 3.31	Giao diện thanh toán SePay	59
Hình 3.32	Email thông báo thanh toán thành công	59
Hình 3.33	Dữ liệu thanh toán lưu trữ trong Redis	60
Hình 3.34	Sơ đồ tổng quan Dịch vụ văn bản	60
Hình 3.35	Tài liệu Swagger API Dịch vụ văn bản	60
Hình 3.36	Dữ liệu vector lưu trữ trên Milvus	62
Hình 3.37	Tệp tài liệu gốc lưu trữ trên Cloudflare R2	62
Hình 3.38	Bản tóm tắt lưu trữ trên Cloudflare R2	62
Hình 3.39	Quản lý migration cơ sở dữ liệu của dịch vụ văn bản	63
Hình 3.40	Cấu hình các bảng trong cơ sở dữ liệu document	64
Hình 3.41	Tài liệu Swagger API Dịch vụ nhân vật	64
Hình 3.42	Tài nguyên nhân vật lưu trữ trên Cloudflare R2	65
Hình 3.43	Quản lý migration cơ sở dữ liệu của dịch vụ nhân vật	66
Hình 3.44	Cấu trúc các bảng trong cơ sở dữ liệu của Dịch vụ nhân vật	66
Hình 3.45	Sơ đồ tổng quan Dịch vụ trò chuyện và Dịch vụ chatbot	67
Hình 3.46	Tài liệu Swagger API Dịch vụ trò chuyện	67
Hình 3.47	Sơ đồ trình tự xóa cuộc trò chuyện và kiểm duyệt chia sẻ	68
Hình 3.48	Kiểm tra Pull Request trên GitHub	69
Hình 3.49	Kiểm tra code linting dịch vụ trò chuyện	69
Hình 3.50	Quản lý database migrations dịch vụ trò chuyện	70
Hình 3.51	Cấu hình các bảng trong cơ sở dữ liệu conversation	70
Hình 3.52	Tài liệu Swagger API Dịch vụ chatbot	71
Hình 3.53	Quản lý database migrations dịch vụ chatbot	71
Hình 3.54	Cấu hình các bảng trong cơ sở dữ liệu chatbot	72
Hình 3.55	Sơ đồ xử lý lịch thứ tự tạo và thu hồi share	74
Hình 3.56	Sơ đồ luồng xử lý DLQ và thông báo Telegram	75
Hình 3.57	Sơ đồ xử lý đồng thời tạo share và xóa chat	76
Hình 3.58	Sơ đồ tích hợp LiveKit đàm thoại giọng nói	77
Hình 3.59	Bộ lọc làm sạch văn bản clean_transcript	78
Hình 3.60	Lưu trữ các tệp tin âm thanh chat trên Cloudflare R2	79

Hình 3.61	Developer gửi tin nhắn văn bản thay giọng nói để debug	79
Hình 3.62	Giao diện lấy thông tin nhân vật từ API Gateway	80
Hình 3.63	Terminal log của việc gọi công cụ lấy thông tin nhân vật	80
Hình 3.64	Logo của Đại học Bách Khoa Hà Nội	85

1 Tổng quan về hệ thống

Trong thời đại công nghệ hiện nay, việc ứng dụng công nghệ vào cuộc sống trở thành một phần không thể thiếu. Đặc biệt trong thời đại công nghệ trí tuệ nhân tạo (AI) đang bùng nổ. Mục tiêu của ứng dụng này là phát triển một hệ thống hỏi đáp về pháp luật Việt Nam. Việc ứng dụng công nghệ AI đánh dấu sự chuyển dịch lớn của ngành pháp lý từ sử dụng các công cụ, phương thức tìm kiếm thủ công sang hệ thống xử lý ngôn ngữ tự nhiên (NLP). Việc ứng dụng công nghệ xử lý ngôn ngữ tự nhiên giúp máy tính hiểu và xử lý được ngôn ngữ của con người.

1.1 Sự cần thiết

Hệ thống tài liệu văn bản pháp luật rất phức tạp. Sự phức tạp không chỉ nằm ở khối lượng dữ liệu, mà các văn bản có thể thay thế, bổ sung, hết hiệu lực, ... Trong một số trường hợp một văn bản vừa hết hiệu lực hoặc bị thay thế thì con người có thể chưa kịp cập nhật thông tin. Đặc biệt là đối với những người bận rộn. Từ đó, có thể dẫn đến những sai sót, thậm chí là thiệt hại về tài chính và rủi ro pháp lý.



Hình 1.1: AI hỗ trợ tư vấn pháp luật

Đối với ứng dụng AI thì sẽ có các điểm mạnh như sau:

- Tiết kiệm chi phí
- Hoạt động làm việc 24/7
- Phản hồi nhanh chóng
- Cập nhật thông tin thường xuyên
- Trích dẫn thông tin chi tiết

Vì AI phụ thuộc vào dữ liệu đầu vào, do việc cập nhật dữ liệu không được ngay lập tức, nên không đảm bảo tính chính xác của các văn bản pháp luật dẫn đến hạn chế và thách thức về độ chính xác của ứng dụng. Vì vậy, trong môi trường đòi hỏi sự chính xác tuyệt đối như pháp luật thì công cụ AI chỉ đóng vai trò trợ lý hỗ trợ, nguồn tham khảo và không thể thay thế con người. Do đó, để sử dụng AI hiệu quả và tránh những rủi ro sai lệch thông tin, người dùng phải kiểm chứng thông tin với chuyên gia và các CSDL chính thống trong các vấn đề mang tính quyết định quan trọng.

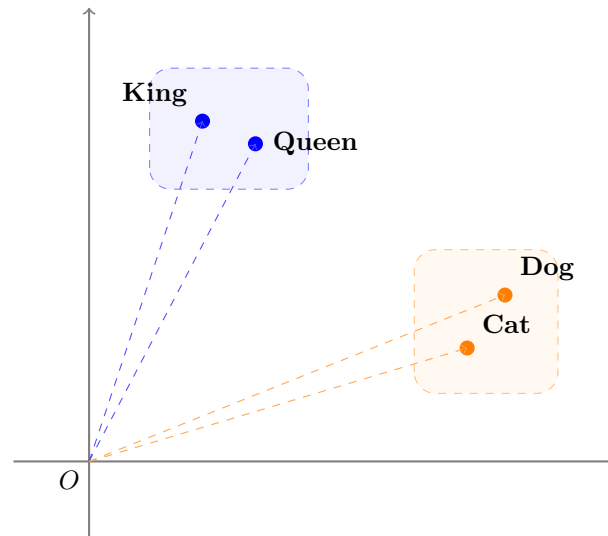
1.2 Khảo sát công nghệ xây dựng cốt lõi

1.2.1 Mô hình ngôn ngữ lớn LLM (Large Language Model)

Năm 2017, các kỹ sư của Google đưa ra bài báo "Attention Is All You Need" khai sinh kiến trúc Transformer với cơ chế Self-Attention (Tự chú ý), mở đường cho hầu hết kiến trúc LLM (Large Language Model) hiện nay.

Cách hoạt động của LLM là dự đoán xác suất của từ tiếp theo dựa theo các từ đã xuất hiện trước đó. LLM không đọc chữ như con người. Chia câu thành các phần nhỏ gọi là token. Biến mỗi token

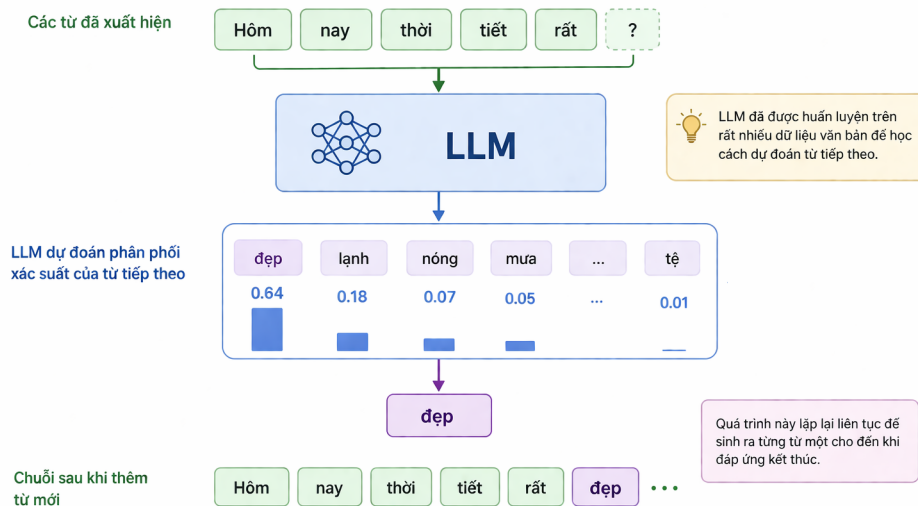
thành vector trong không gian nhiều chiều. Những từ có nghĩa gần nhau vua (king) và hoàng hậu (queen), hoặc mèo (cat) và chó (dog) sẽ có các tọa độ rất gần nhau trong không gian.



Hình 1.2: Mô phỏng quá trình ánh xạ các vector từ gần nghĩa.

Khi dự đoán từ tiếp theo, mô hình không chỉ nhìn vào từ ngay trước nó, mà nó nhìn vào toàn bộ các từ đã xuất hiện và dùng cơ chế gọi là Self-Attention để xem từ nào quan trọng nhất.

Ví dụ: Trong câu "Con mèo rượt đuổi con chuột vì nó chạy rất nhanh", cơ chế Attention sẽ giúp mô hình hiểu chữ "nó" ở đây có xác suất cao là đang chỉ "con chuột" chứ không phải "con mèo". Sau khi tính toán ngữ cảnh, mô hình sẽ quét qua toàn bộ kho từ vựng và gán cho mỗi từ một con số xác suất. Mô hình sẽ chọn từ tiếp theo dựa trên bảng xác suất này. Quá trình này lặp đi lặp lại liên tục cho đến khi nó tạo ra một ký tự đặc biệt ra lệnh dừng lại.



Hình 1.3: Cách hoạt động của LLM

Tuy nhiên LLM có một số nhược điểm:

- Dữ liệu huấn luyện rộng
- Độ chính xác không cao với miền cụ thể
- Có hiện tượng ảo giác (Hallucination)
- Dữ liệu chỉ cập nhật đến ngày train, dữ liệu cũ
- LLM không biết dữ liệu mới

- LLM không biết dữ liệu doanh nghiệp
- LLM không biết tài liệu người dùng
- Chi phí cho việc huấn luyện, tinh chỉnh lớn.

LLM CÓ MỘT SỐ NHƯỢC ĐIỂM



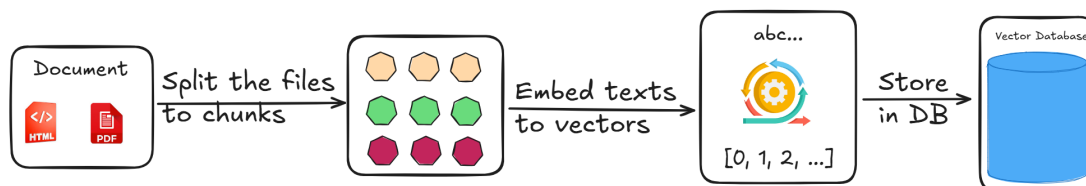
Hình 1.4: Một số nhược điểm của LLM

1.2.2 Tạo tăng cường truy xuất RAG (Retrieval Augmented Generation)

Mô hình ngôn ngữ lớn có thể đưa ra kết quả không chính xác trong thực tế, đối mặt với thách thức như hiểu được lập luận pháp lý phức tạp, đảm bảo tính minh bạch và tránh các phản hồi gây hiểu nhầm. Làm thế nào để mô hình ngôn ngữ lớn có thể trả lời các câu hỏi pháp luật một cách chính xác? Retrieval Augmented Generation (RAG) là một kỹ thuật được thiết kế để khắc phục hiện tượng ảo giác của LLM bằng cách cung cấp thêm nguồn tri thức bên ngoài trước khi sinh ra câu trả lời.

Ứng dụng trong đồ án

Trong phạm vi đồ án này, em sử dụng RAG để tăng cường khả năng truy xuất tri thức từ các văn bản pháp luật, giúp mô hình ngôn ngữ lớn đưa ra câu trả lời chính xác hơn, có căn cứ pháp lý rõ ràng và giảm thiểu hiện tượng ảo giác so với mô hình LLM truyền thống. Thay vì “đoán”, LLM sẽ dựa vào dữ liệu thật từ đó cung cấp nguồn dẫn chứng rõ ràng cho người dùng.



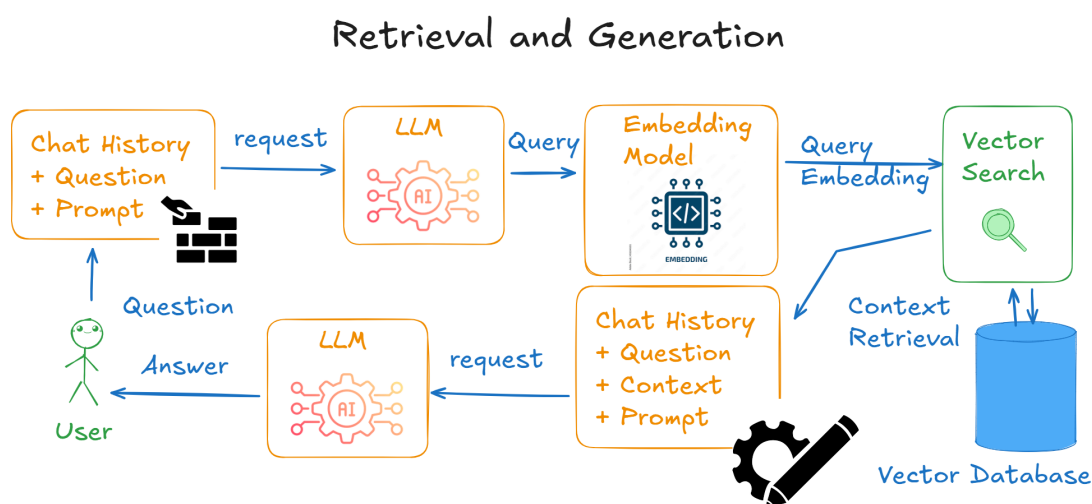
Extracting and Indexing

Hình 1.5: Sơ đồ RAG cơ bản (Extracting and Indexing)

Các bước cốt lõi để xây dựng một hệ thống RAG:

- Toàn bộ dữ liệu tri thức (tài liệu, văn bản luật, ...) được chia nhỏ thành các đoạn (chunks).
- Sử dụng các mô hình nhúng (embedding models) để chuyển đổi các đoạn văn bản này thành các biểu diễn vector.
- Lưu trữ các vector vào vector database để tối ưu hóa việc tìm kiếm không gian nhiều chiều.
- Hệ thống nhận yêu cầu từ người dùng dưới dạng ngôn ngữ tự nhiên.
- Câu truy vấn của người dùng cũng được mã hóa thành vector.
- Cơ chế truy xuất sẽ quét toàn bộ vector database để đo lường độ tương đồng ngữ nghĩa nhằm trích xuất ra các đoạn tài liệu liên quan nhất.
- Các đoạn văn bản liên quan vừa được truy xuất sẽ được ghép nối với câu truy vấn ban đầu của người dùng.
- Câu prompt đã được làm giàu ngữ cảnh sẽ được đưa vào LLM
- LLM sử dụng ngữ cảnh được cung cấp để sinh ra câu trả lời mạch lạc và có độ chính xác cao đối với nguồn dữ liệu.

Việc áp dụng kiến trúc RAG không chỉ giúp giảm đáng kể chi phí huấn luyện và bảo trì mô hình mà còn hạn chế hiện tượng tạo ra thông tin ảo giác thông qua việc dựa trên các tài liệu được truy xuất từ nguồn dữ liệu đáng tin cậy. Điều này giúp nâng cao độ chính xác, khả năng mở rộng và tính cập nhật của hệ thống.



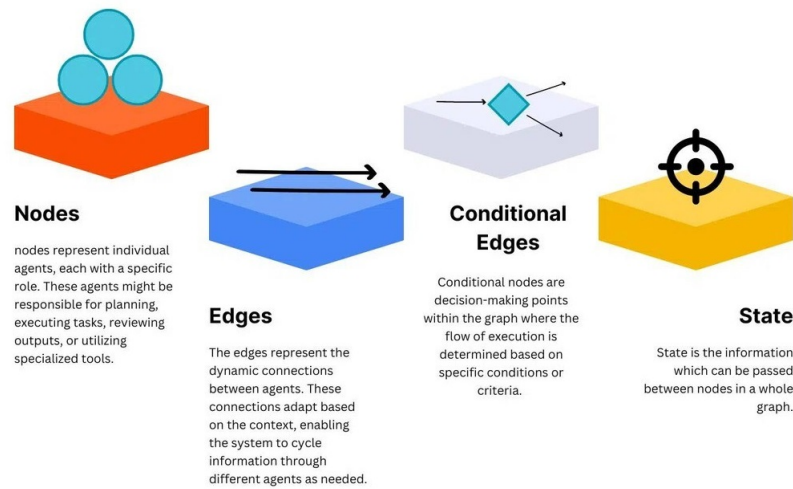
Hình 1.6: Sơ đồ RAG cơ bản (Retrieval and Generation)

Tuy nhiên, sử dụng RAG đơn giản thì độ chính xác chưa cao và vẫn có thể sai sót vì vậy, em sử dụng LangGraph xây dựng Agentic RAG để tăng độ chính xác và hạn chế ảo giác vì miền tư vấn pháp luật yêu cầu độ chính xác cao.

Thay vì thực thi các bước theo quy trình tuyến tính như LangChain, LangGraph mô hình hóa luồng công việc dưới dạng một đồ thị gồm các thành phần chính sau:

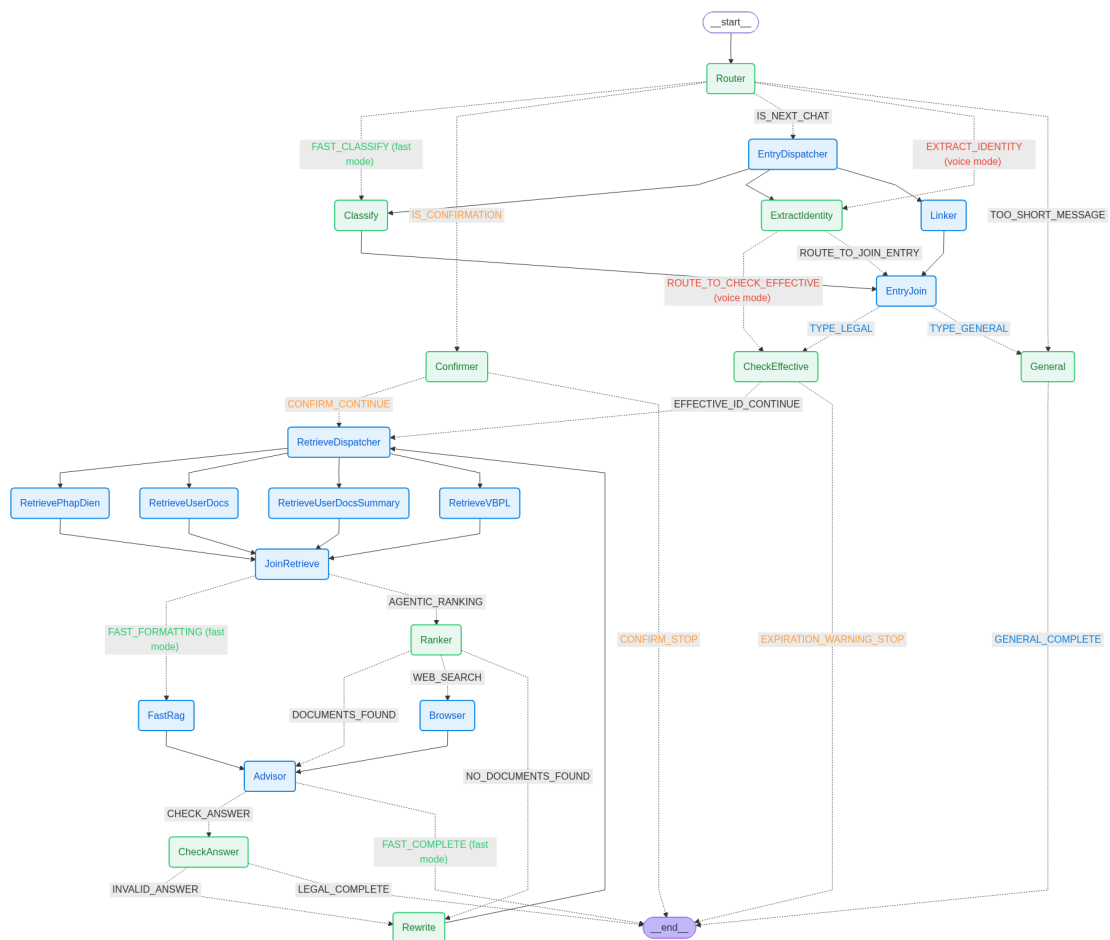
- Nodes (Nút): Đại diện cho các bước xử lý, tác vụ hoặc hành động cụ thể, chẳng hạn như gọi mô hình ngôn ngữ lớn (LLM), truy xuất cơ sở dữ liệu hoặc tương tác với người dùng.
- Edges (Cạnh): Xác định hướng chuyển tiếp giữa các nút. Các cạnh có thể là đường đi trực tiếp hoặc cạnh có điều kiện (conditional edges), cho phép lựa chọn nhánh thực thi dựa trên kết quả của nút trước đó.
- Cycles (Vòng lặp): Đây là đặc điểm nổi bật của LangGraph. Khác với kiến trúc đồ thị có hướng không chu trình, LangGraph cho phép tác nhân (agent) quay lại các nút đã thực hiện để đánh giá lại kết quả, tự sửa lỗi hoặc lặp lại quy trình nhiều lần cho đến khi đạt được mục tiêu mong muốn.

- State (Trạng thái): Đây là bộ nhớ dùng chung. Mọi nút (node) trong đồ thị đều có thể đọc dữ liệu từ State này và ghi kết quả mới của chúng vào đây.



Hình 1.7: Các khái niệm quan trọng của LangGraph

Sơ đồ Agentic RAG dự án này:



Hình 1.8: Sơ đồ LangGraph của dự án

Giải thích sơ đồ Agentic RAG:

- Hệ thống phân chia việc sử dụng thông minh giữa mô hình nhỏ cho luồng cơ bản và mô hình lớn cho tác vụ suy luận là chìa khóa để tối ưu cả chi phí, độ trễ và hiệu năng. (Nhìn vào sơ

đồ các Node màu xanh lá sử dụng mô hình nhỏ, các Node màu xanh dương sử dụng mô hình lớn)

- Mỗi tin nhắn đều đi qua **Router**. Node này kiểm tra điều kiện giúp định tuyến vào luồng chính xác theo từng trạng thái.
- Ở chế độ suy luận Agentic RAG đầy đủ thực hiện phân tích song song
 - **Linker** đọc URL nếu có.
 - **Classify** phân loại câu hỏi là pháp lý hay thông thường.
 - **Extract Identity** trích xuất số hiệu văn bản (VD: 100/2019/NĐ-CP).
- Kiểm tra hiệu lực pháp lý: Nếu câu hỏi thuộc nhóm **TYPE_LEGAL** và có số hiệu văn bản, hệ thống tra cứu trạng thái.
 - Văn bản đã hết hiệu lực hoàn toàn thì kết thúc ngay.
 - Văn bản hết hiệu lực một phần thì phát cảnh báo và chờ người dùng xác nhận ở lượt sau qua **Confirmer**.
- Truy xuất nguồn dữ liệu song song
 - Tài liệu người dùng (Milvus)
 - Tóm tắt tài liệu (Cloudflare R2)
 - Văn bản pháp luật (Pinecone)
 - Dữ liệu pháp điển (Qdrant)
- Tiếp tục tới **Ranker** để lọc tài liệu
 - Nếu có tài liệu hợp lý thì chuyển đến **Advisor** để tiến hành tạo câu trả lời.
 - Nếu không có tài liệu hợp lý thì chuyển đến **Rewrite** để tạo lại câu hỏi.
 - Sau khi thử tạo lại câu hỏi vẫn không có tài liệu hợp lý thì sẽ dùng **Browser** để lấy tài liệu tại nguồn thuvienphapluat.vn và luatvietnam.vn
- Vòng lặp tự kiểm tra chất lượng Sau khi **Advisor** trả lời, **Check Answer** đánh giá câu trả lời có hợp lý hay không?
 - Nếu không hợp lệ và chưa quá 2 lần thì xóa câu trả lời cũ, chuyển đến **Rewrite** để tạo lại câu hỏi và lặp lại toàn bộ luồng truy xuất.
 - Nếu câu trả lời hợp lệ hoặc đã quá số lần thử lại thì hệ thống kết thúc quá trình trả lời.
- Ở chế độ Fast RAG chỉ đi qua **Classify** để phân loại. Sau đó truy xuất dữ liệu lấy dữ liệu. Dùng **Advisor** trả lời và hệ thống kết thúc quá trình trả lời mà không cần bước kiểm tra.
- Ở chế độ giọng nói Voice, do đã có Agent nên không cần qua bước phân loại của LangGraph và người dùng không thể nhập URL nên hệ thống chuyển đến **Extract Identity** trích xuất số hiệu văn bản.

1.2.3 Kiến trúc vi dịch vụ MSA (Microservice Architecture)

Kiến trúc vi dịch vụ chia bài toán thành các thành phần nhỏ hơn được gọi là các dịch vụ.

- Mỗi dịch vụ tập trung vào một khả năng kinh doanh cụ thể.
- Các dịch vụ độc lập và giao tiếp với nhau thông qua hạ tầng mạng.

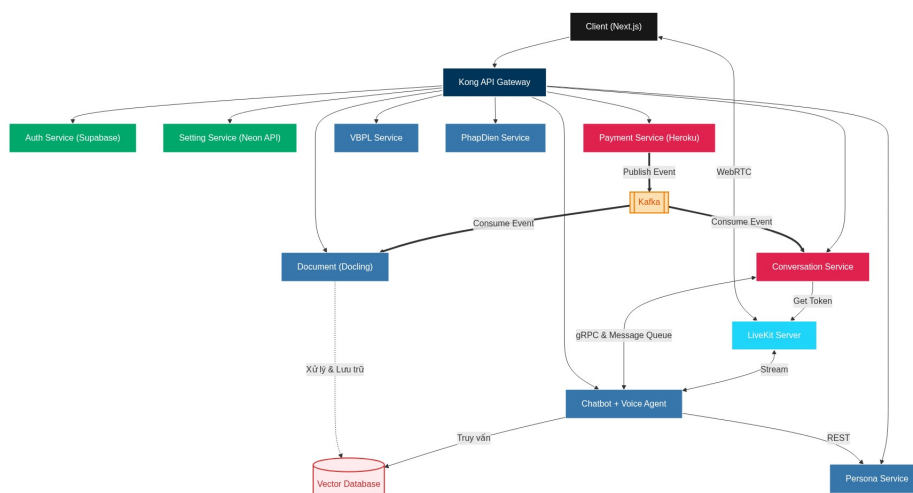


Hình 1.9: Giới thiệu Microservice

Một số lợi ích của Microservice:

- Mỗi dịch vụ có các biến bí mật (secret) riêng.
- Không cần cài các thư viện, các phụ thuộc với dịch vụ không dùng
- Đa dạng ngôn ngữ lập trình khác nhau theo từng logic của từng dịch vụ
- Phân chia tài nguyên cho từng dịch vụ (Ví dụ Heroku giới hạn RAM, AI cần GPU)
- Giảm thời gian quá trình kiểm thử vì mỗi dịch vụ có nghiệp vụ khác nhau
- Vì các dịch vụ độc lập, dễ dàng triển khai
- Tăng khả năng mở rộng hệ thống

Kiến trúc tổng quan các dịch vụ của dự án:



Hình 1.10: Kiến trúc tổng quan các dịch vụ của dự án

1.2.4 Thiết kế hướng miền DDD (Domain Driven Design)

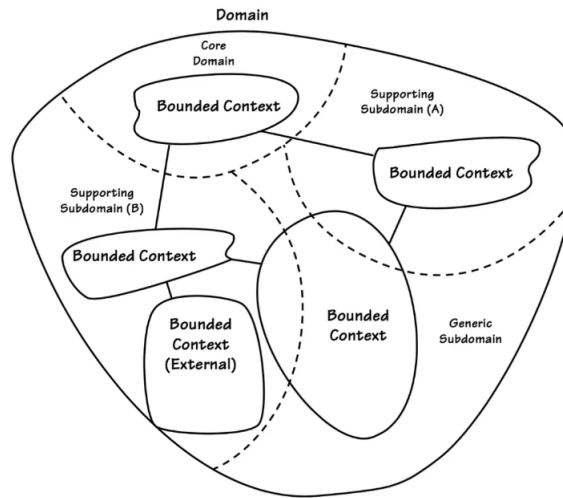
Thiết kế hướng miền là một phương pháp tiếp cận thiết kế phần mềm do Eric Evans giới thiệu, tập trung vào việc thấu hiểu và mô hình hóa sâu sắc lĩnh vực kinh doanh của tổ chức. Thay vì để logic nghiệp vụ bị lẫn lộn với các yếu tố kỹ thuật (như cơ sở dữ liệu, hạ tầng mạng), DDD đặt nghiệp vụ làm trung tâm để đưa ra giải pháp linh hoạt, dễ bảo trì và dễ mở rộng. Cốt lõi của DDD được chia thành hai nhóm mẫu chính: **Các mẫu chiến lược (Strategic Patterns)** và **Các mẫu kỹ thuật (Tactical Patterns)**.

Các mẫu chiến lược (Strategic Patterns)

Nhóm mẫu này giúp phân chia một hệ thống lớn và phức tạp thành các phần nhỏ hơn với ranh giới rõ ràng, tập trung vào kiến trúc tổng thể.

- **Miền (Domain) & Miền phụ (Sub-Domain):** Phạm vi kiến thức hoặc vấn đề mà phần mềm giải quyết. Một miền lớn được chia thành 3 loại miền phụ:

- Miền phụ cốt lõi (Core Subdomain)
- Miền phụ hỗ trợ (Supporting Subdomain)
- Miền phụ chung (Generic Subdomain)



Hình 1.11: Các loại miền phụ trong Domain Driven Design

- **Chuyên gia miền (Domain Expert):** Người hiểu sâu sắc nhất về quy trình kinh doanh thực tế, làm việc chặt chẽ với đội ngũ phát triển.
- **Ngôn ngữ chung (Ubiquitous Language):** Ngôn ngữ thống nhất được sử dụng xuyên suốt bởi cả đội ngũ kỹ thuật và chuyên gia miền, từ giao tiếp hàng ngày đến việc đặt tên trong mã nguồn.
- **Bối cảnh bị giới hạn (Bounded Context):** Xác định ranh giới rõ ràng cho một phần của hệ thống, giúp các nhóm hoạt động độc lập và giữ cho mô hình nhất quán trong phạm vi đó.
- **Bản đồ bối cảnh (Context Maps) & Các mối quan hệ:** Thể hiện trực quan cách các Bounded Context giao tiếp với nhau.

Các mẫu kỹ thuật (Tactical Patterns)

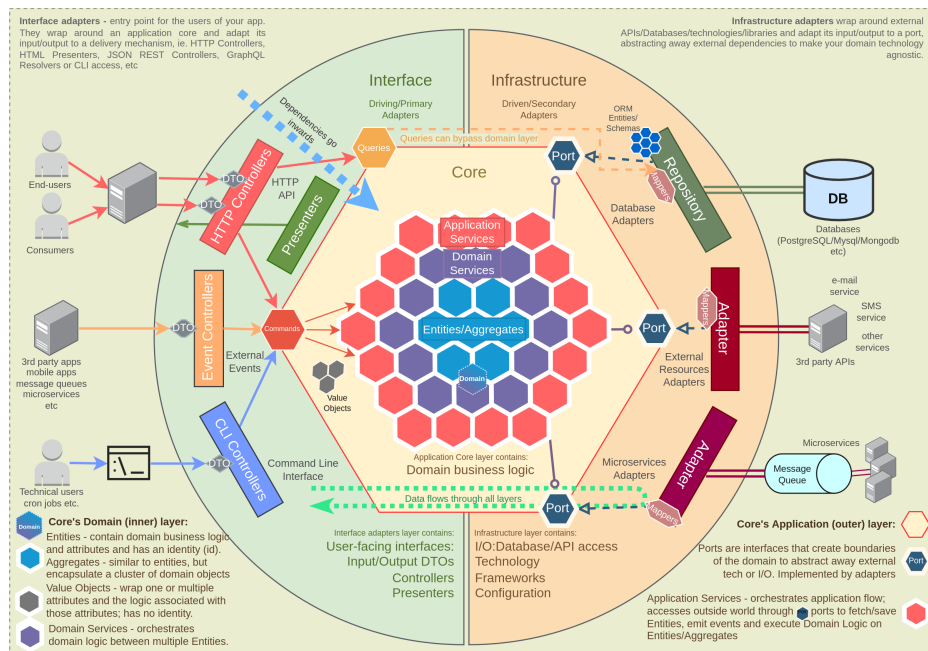
Nhóm mẫu này đóng vai trò hiện thực hóa các khái niệm nghiệp vụ vào thiết kế mã nguồn hệ thống.

- **Đối tượng thực thể (Entities Objects):** Các đối tượng có **định danh duy nhất (ID)** tồn tại xuyên suốt vòng đời. Thực thể quản lý trạng thái và có thể bị thay đổi theo thời gian.
- **Đối tượng giá trị (Value Objects):** Các đối tượng không có định danh duy nhất, dùng để đại diện cho một đặc điểm (ví dụ: Địa chỉ, Giá tiền). Chúng được so sánh qua các thuộc tính và thường được thiết kế ở dạng bất biến (immutable).
- **Tổng hợp (Aggregates):** Một nhóm các Thực thể và Đối tượng giá trị được gom lại thành một khối thống nhất để đảm bảo tính toàn vẹn dữ liệu. Giao tiếp với bên ngoài bắt buộc phải thông qua một thực thể gốc (Aggregate Root).
- **Nhà máy (Factory):** Đóng gói quy trình khởi tạo phức tạp của các đối tượng hoặc Aggregates để đảm bảo chúng luôn được tạo ra ở trạng thái hợp lệ.
- **Kho lưu trữ (Repository):** Trừu tượng hóa lớp tương tác với cơ sở dữ liệu. Nó hoạt động như một bộ sưu tập lưu trữ các đối tượng Aggregates trong bộ nhớ, giúp mô hình miền tách biệt hoàn toàn khỏi công nghệ lưu trữ.
- **Dịch vụ miền (Domain Service):** Nơi chứa các quy tắc hoặc hành vi nghiệp vụ phức tạp liên quan đến nhiều đối tượng, không phù hợp để đặt vào bên trong một Thực thể hay Đối tượng giá trị cụ thể.

Kiến trúc hệ thống trong DDD

Để bảo vệ lớp miền cốt lõi khỏi các công nghệ bên ngoài, báo cáo áp dụng các kiến trúc phân lớp:

- **Kiến trúc phân lớp (Layered Architecture):** Thường chia làm 4 lớp cơ bản: Giao diện người dùng (User Interface) → Ứng dụng (Application) → Lớp Miền (Domain - trung tâm) → Hạ tầng (Infrastructure).
- **Kiến trúc lục giác (Hexagonal Architecture / Ports and Adapters):** Đây là kiến trúc được áp dụng chính. Đặt lớp Miền (Domain) ở vị trí trung tâm, cách ly hoàn toàn với các yếu tố bên ngoài. Mọi giao tiếp với cơ sở dữ liệu, API, hoặc giao diện đều thông qua các **Cổng (Ports)** và **Bộ điều hợp (Adapters)**. Kiến trúc này giúp hệ thống cực kỳ dễ kiểm thử, linh hoạt thay đổi công nghệ (Database, Framework) mà không làm ảnh hưởng đến logic nghiệp vụ cốt lõi.
- **Nguyên tắc đảo ngược phụ thuộc (Dependency Inversion):** Đảo ngược phụ thuộc là nguyên tắc cốt lõi giúp bảo vệ và cô lập lớp Miền (Domain Layer). Cơ chế hoạt động: Lớp Miền sẽ định nghĩa các giao diện (chính là các Cổng - Ports trong Kiến trúc lục giác) để yêu cầu những gì nó cần. Các module cấp thấp (như CSDL, hàng đợi tin nhắn) sẽ triển khai các giao diện này (đóng vai trò là Bộ điều hợp - Adapters) và cắm vào lớp Miền như những “plugin” độc lập.



Hình 1.12: Tổng quan kiến trúc lục giác

1.3 Mục tiêu

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2 Phân tích thiết kế hệ thống

2.1 Bài toán nghiệp vụ

Hệ thống hỗ trợ người dùng hỏi đáp pháp luật Việt Nam, phân tích tài liệu và đàm thoại trực tiếp thời gian thực với trợ lý ảo pháp lý. Dựa trên thiết kế hướng miền (DDD), bài toán lớn được chia làm các miền phụ (Subdomain) với các chức năng như sau:



Hình 2.1: Sơ đồ tổng quan SubDomain

2.1.1 Miền phụ cốt lõi (Core Subdomain)

Miền phụ cốt lõi là phần quan trọng và có giá trị nhất của hệ thống. Miền phụ cốt lõi giúp phân biệt các doanh nghiệp và làm cho các doanh nghiệp có giá trị. Miền phụ cốt lõi tập trung vào mục tiêu và yêu cầu của khách hàng với doanh nghiệp, từ đó quyết định sự thành công của doanh nghiệp. Vì vậy, mỗi doanh nghiệp luôn tìm cách thực hiện những điều khác biệt trong các miền phụ cốt lõi này để đạt được lợi thế so với đối thủ cạnh tranh.

- Dịch vụ trò chuyện (conversation service)
 - Nhập văn bản bằng text hoặc micro
 - Chức năng đàm thoại giọng nói Voice (Chỉ dành cho người dùng đã thanh toán VIP)
 - Sử dụng chức năng RAG suy luận (Chỉ dành cho người dùng đã thanh toán VIP)
 - Đánh dấu cuộc trò chuyện theo thư mục và ghi chú
 - Tạo liên kết chia sẻ và thu hồi cuộc trò chuyện
- Dịch vụ chatbot (chatbot service)
 - Tương tác giao tiếp với conversation service
 - Tương tác giao tiếp với livekit server
 - Lấy thông tin về lịch sử trò chuyện
 - Lấy thông tin chi tiết trò chuyện

Miền phụ cốt lõi là nơi diễn ra các luồng xử lý phức tạp và quyết định đến mức độ hài lòng của người dùng.

2.1.2 Miền phụ hỗ trợ (Supporting Subdomain)

Các miền phụ cốt lõi phụ thuộc vào các miền phụ hỗ trợ. Miền phụ hỗ trợ cung cấp các dịch vụ để miền phụ cốt lõi hoạt động hiệu quả. Tuy nhiên, miền phụ hỗ trợ không đòi hỏi mức độ phức tạp cao về logic nghiệp vụ.

- Dịch vụ văn bản (document service)

- Tải lên file tài liệu văn bản (Chỉ dành cho người dùng đã thanh toán VIP)
- Lấy thông tin trạng thái văn bản
- Dịch vụ nhân vật (persona service)
 - Người dùng lựa chọn nhân vật để trò chuyện giọng nói
 - Quản lý nhân vật (cần vai trò ADMIN)
 - Quản lý giọng nói (cần vai trò ADMIN)
- Báo cáo Data pipeline
 - Báo cáo thông tin Data pipeline của văn bản pháp luật
 - Báo cáo thông tin Data pipeline của Pháp Điển

Miền phụ hỗ trợ giúp miền phụ cốt lõi không bị đình to. Tách nhỏ việc quản lý nhân vật, giọng nói và quá trình xử lý tài liệu tiêu tốn nhiều tài nguyên hệ thống.

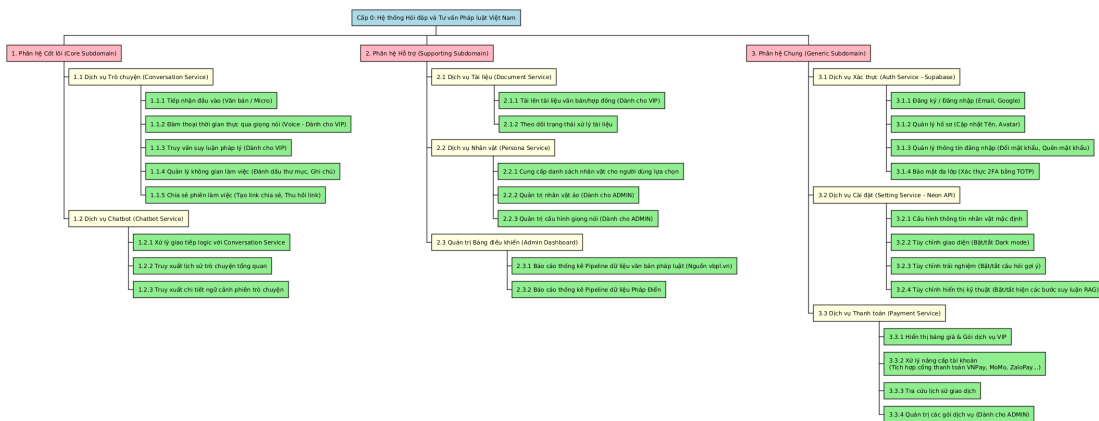
2.1.3 Miền phụ chung (Generic Subdomain)

Miền phụ chung cung cấp các giải pháp có sẵn mà doanh nghiệp có thể mua, thuê. Miền phụ chung có thể được tìm thấy trên nhiều ngành. Doanh nghiệp không thể đạt được bất kỳ lợi thế cạnh tranh so với đối thủ bằng cách thực hiện những điều khác biệt trong miền phụ chung.

- Dịch vụ xác thực (auth service - dùng supabase)
 - Người dùng đăng ký, đăng nhập bằng email hoặc đăng nhập bằng Google
 - Người dùng xem thông tin cá nhân và cập nhật tên, cập nhật ảnh avatar
 - Người dùng có thể đổi mật khẩu hoặc dùng quên mật khẩu
 - Xác thực 2FA bằng TOTP (bắt buộc vì dữ liệu pháp luật cần bảo mật)
- Dịch vụ cài đặt (setting service - dùng Neon API)
 - Lưu thông tin nhân vật giọng nói
 - Bật tắt giao diện Dark mode
 - Bật tắt câu hỏi gợi ý
 - Bật tắt Tự động hiển thị các bước suy luận RAG trong cuộc trò chuyện
- Dịch vụ thanh toán (payment service)
 - Hiển thị các gói VIP và mức giá
 - Quản lý các gói (cần vai trò ADMIN)
 - Người dùng mua gói VIP để nâng cấp tài khoản
 - Lưu lịch sử giao dịch

Việc áp dụng các giải pháp có sẵn như Supabase (Auth) hay Neon API (Settings) là một chiến lược thông minh để rút ngắn thời gian phát triển. Đặc biệt là việc tích hợp 2FA, điều kiện tiên quyết cho các hệ thống xử lý hợp đồng và dữ liệu pháp luật mang tính nhạy cảm.

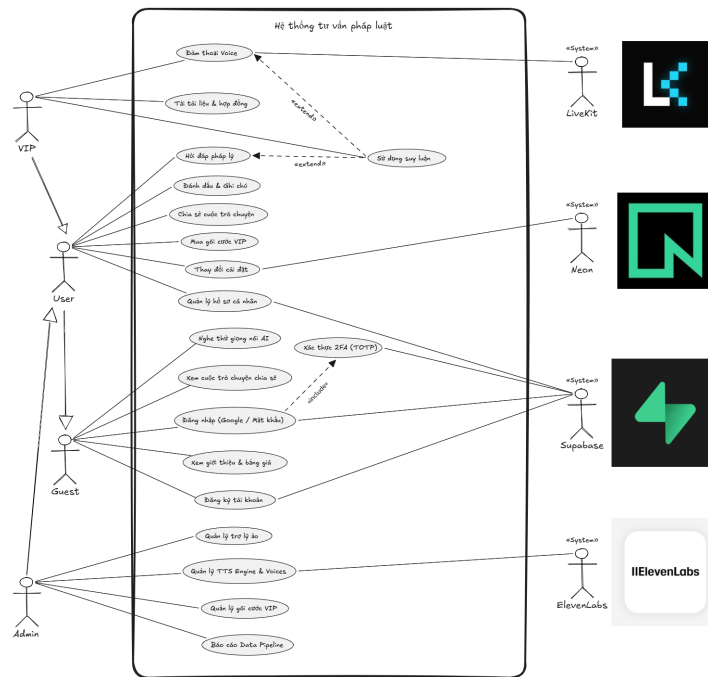
Sơ đồ phân rã chức năng



Hình 2.2: Sơ đồ phân rã chức năng

2.2 Phân tích thiết kế hệ thống

2.2.1 Sơ đồ Use Case



Hình 2.3: Sơ đồ Use Case

Hệ thống phân chia người dùng thành 4 nhóm đối tượng chính với các đặc quyền được kế thừa và mở rộng dần theo cấp độ:

- Khách vắng lai (Guest):
 - Đặc điểm: Là những người dùng chưa có tài khoản hoặc chưa đăng nhập vào hệ thống. Mục đích chính của họ là tìm hiểu thông tin cơ bản về ứng dụng và trải nghiệm thử các tính năng công khai.
 - Quyền hạn: Có thể xem các trang giới thiệu, bảng giá gói cước; nghe thử các mẫu giọng nói của các nhân vật; xem các đoạn trò chuyện tư vấn pháp luật được người dùng khác chia sẻ công khai. Họ có thể thực hiện đăng ký tài khoản mới hoặc đăng nhập (qua email hoặc Google).
- Người dùng phổ thông (User):
 - Đặc điểm: Là những cá nhân đã đăng ký và đăng nhập thành công vào hệ thống. Đây là tệp khách hàng tiêu chuẩn sử dụng các tính năng hỏi đáp pháp lý cơ bản bằng văn bản. Tác nhân này kế thừa toàn bộ quyền của Guest.
 - Quyền hạn: Có thể chat/hỏi đáp pháp lý cơ bản với AI; sử dụng các công cụ cá nhân hóa như đánh dấu (bookmark), tạo ghi chú, thay đổi cài đặt và quản lý hồ sơ cá nhân. Người dùng có thể chia sẻ cuộc trò chuyện của mình cho người khác, bảo mật tài khoản bằng xác thực 2 lớp (2FA/TOTP) và có thể thực hiện nâng cấp/mua gói cước VIP.
- Người dùng VIP (VIP):
 - Đặc điểm: Là những người dùng đã thanh toán và đăng ký gói cước VIP. Họ có nhu cầu giải quyết các vấn đề pháp lý phức tạp, cần phân tích tài liệu chuyên sâu hoặc muốn tương tác rảnh tay qua giọng nói. Tác nhân này kế thừa toàn bộ quyền của User.
 - Quyền hạn: Được sử dụng các tính năng cao cấp bao gồm: Đàm thoại trực tiếp với AI bằng giọng nói (Voice chat qua LiveKit); tải lên các tài liệu, hợp đồng cá nhân để AI phân tích; và được phép sử dụng các mô hình suy luận (reasoning) nâng cao để giải quyết các truy vấn pháp lý phức tạp (áp dụng cho cả chat văn bản và đàm thoại).

- Quản trị viên (Admin):

- Đặc điểm: Là nhân sự thuộc ban quản trị/vận hành hệ thống. Họ có kiến thức về quản lý hệ thống, theo dõi luồng dữ liệu (Data Pipeline) và cấu hình các dịch vụ tích hợp giọng nói. Tác nhân này kế thừa toàn bộ quyền cơ bản của User.
- Quyền hạn: Có toàn quyền truy cập trang quản trị để: Quản lý các nhân vật trợ lý ảo; thiết lập và quản lý hệ thống chuyển đổi văn bản thành giọng nói (TTS Engine và Voices kết nối với ElevenLabs); quản lý các gói cước VIP; và xem/theo dõi các báo cáo về luồng xử lý dữ liệu pháp luật (Data Pipeline).

2.2.2 Sơ đồ tuần tự (Sequence Diagram)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.3 Kiến trúc hệ thống

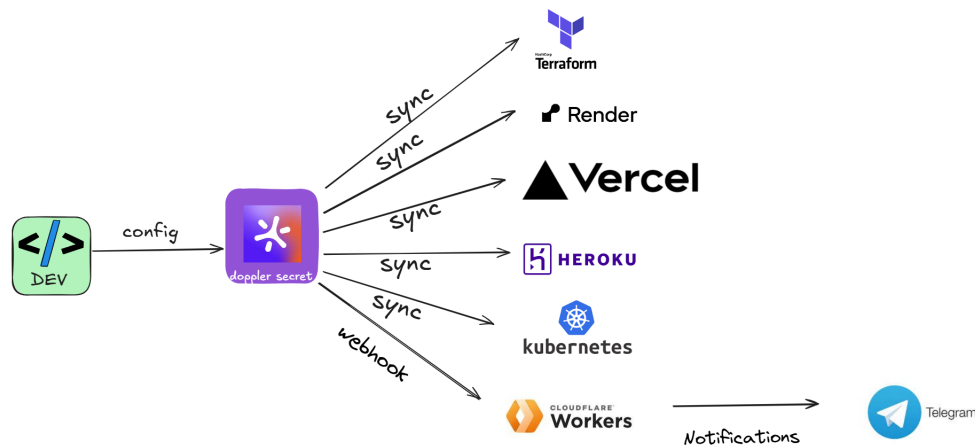
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.3.1 Quản lý các thông tin bí mật (doppler)

Quản lý các thông tin bí mật là một phần cực kỳ quan trọng. Đặc biệt khi hệ thống được thiết kế theo kiến trúc microservices. Khi dự án có nhiều service, việc sử dụng file .env độc lập cho từng service sẽ gây khó khăn trong việc quản lý, cấu hình, đồng bộ và bảo mật.

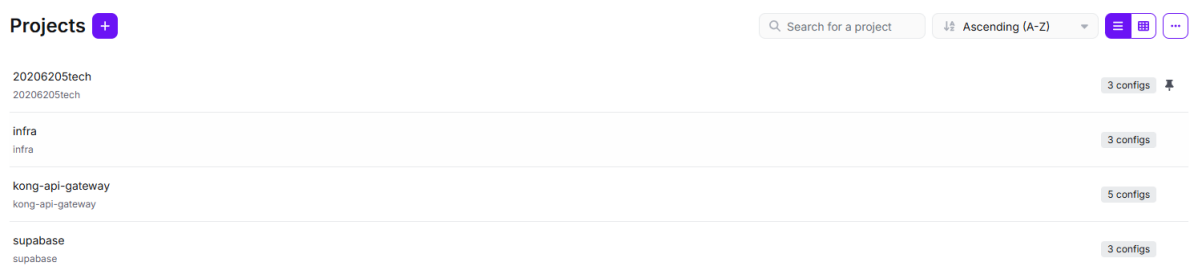
Ứng dụng trong đồ án

Trong đồ án này, em sử dụng Doppler – một nền tảng quản lý thông tin bí mật tập trung. Việc áp dụng Doppler mang lại một không gian lưu trữ an toàn, thống nhất (Single Source of Truth) cho toàn bộ hệ thống.

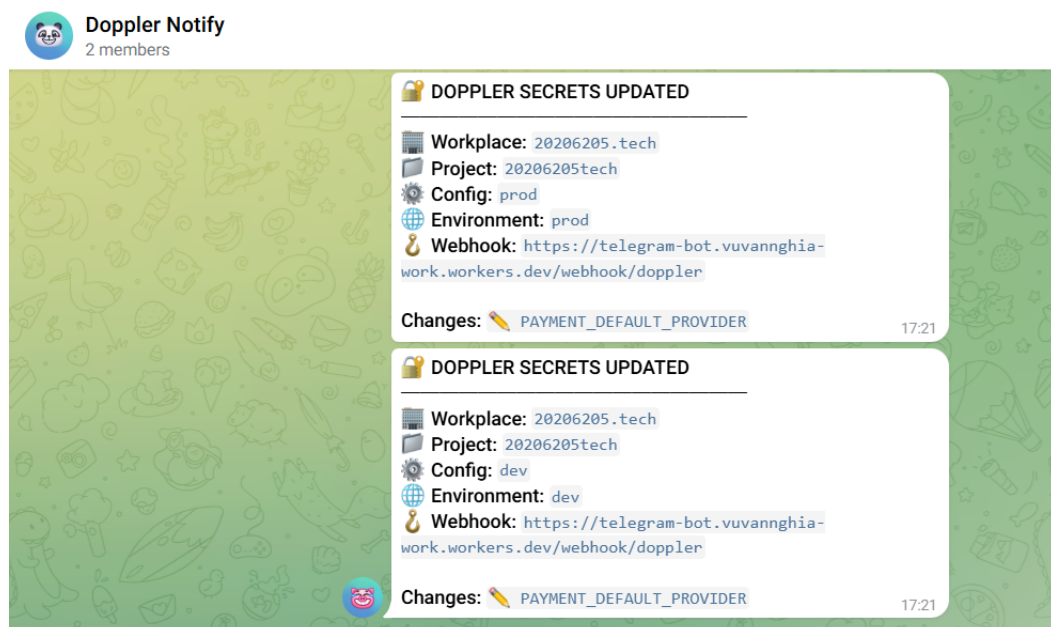


Hình 2.4: Sơ đồ quản lý secret bằng doppler

Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



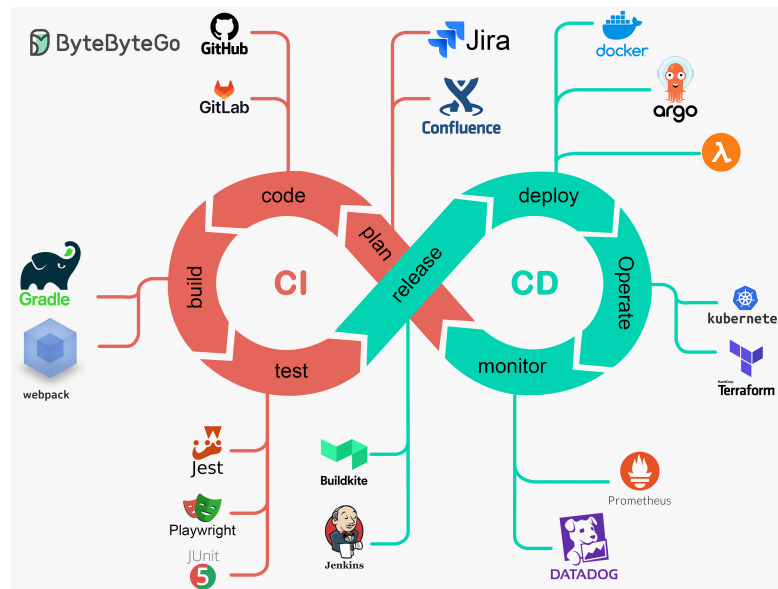
Hình 2.5: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



Hình 2.6: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

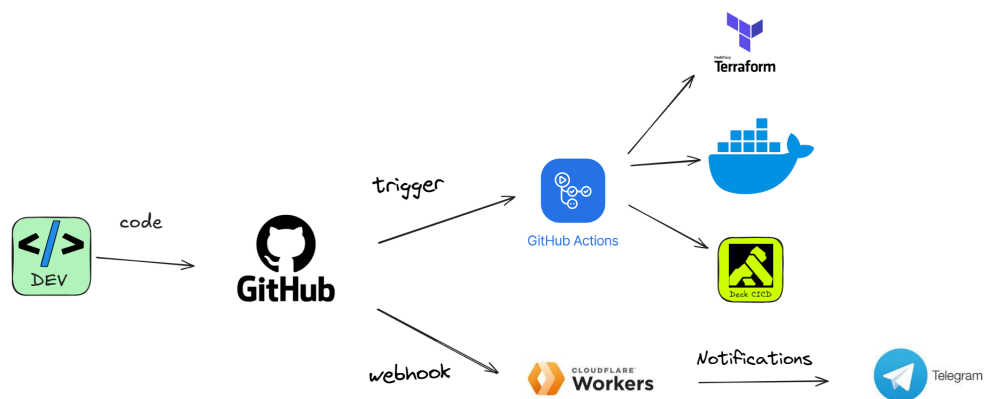
2.3.2 CI/CD (Github Action)

CI/CD là phương pháp tự động hóa quy trình tích hợp và triển khai phần mềm. Phương pháp này giúp đẩy nhanh tốc độ, giảm thiểu lỗi thủ công và duy trì độ ổn định cho hệ thống. GitHub Actions là công cụ CI/CD mạnh mẽ được tích hợp sẵn trên GitHub.



Hình 2.7: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Sơ đồ CI/CD bằng Github Action



Hình 2.8: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Ứng dụng trong đồ án

Trong đồ án này, hệ thống vừa sử dụng Github Action miễn phí của Github, vừa sử dụng Self-hosted runner trên VPS.

2.3.3 Cơ sở hạ tầng dưới dạng mã (terraform)

Buckets	Objects	Size
dev-documents	246	2.1 MB
dev-persona	270	1.17 GB
dev-share	13	7.21 kB
prod-documents	12	84.83 kB
prod-persona	270	1.17 GB
prod-share	5	3.77 kB

Hình 2.9: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

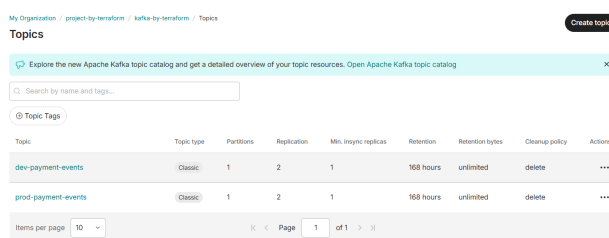
Trong công nghệ phần mềm việc cấu hình các cơ sở hạ tầng thủ công có thể gây sai sót, mất nhiều thời gian hoặc khi muốn tạo lại chính xác rất khó và khi cần xóa thì cũng sẽ phải ấn thủ công dẫn đến thiếu sót, gây rủi ro. Việc sử dụng Infrastructure as Code (IaC) sẽ khắc phục được vấn đề đó.

Ứng dụng trong đồ án

Trong đồ án này, em sử dụng Terraform để quản lý cấu hình một số hạ tầng của các nhà cung cấp có hỗ trợ terraform giúp định nghĩa và quản lý hạ tầng của dự án một cách tự động thay vì thao tác thủ công.

- langsmith: theo dõi RAG
- oracle: tạo vps
- cloudflare: cấu hình DNS, bucket R2 file, tunnel
- aiven: kafka
- supabase: auth google
- Neon: database postgres VBPL=>
- digitalocean xxxxxxxxxxxxxx: database postgres VBPL
- heroku: payment service

Kết quả kafka trong aiven được tạo tự động từ terraform



My Organization / project-by-terraform / kafka-by-terraform / Topics

Explore the new Apache Kafka topic catalog and get a detailed overview of your topic resources. Open Apache Kafka topic catalog

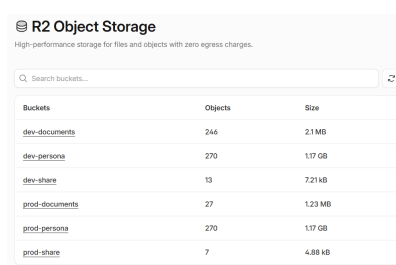
Search by name and tags...

Topic Tags

Topic	Topic type	Partitions	Replication	Min. Insync replicas	Retention	Retention bytes	Cleanup policy	Actions
dev-payment-events	Classic	1	2	1	168 hours	unlimited	delete	...
prod-payment-events	Classic	1	2	1	168 hours	unlimited	delete	...

Items per page: 10 Page 1 of 1

Hình 2.10: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



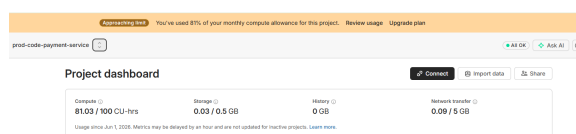
R2 Object Storage

High-performance storage for files and objects with zero egress charges.

Search buckets...

Buckets	Objects	Size
dev-documents	246	2.1 MB
dev-persona	270	1.17 GB
dev-share	13	7.21 kB
prod-documents	27	1.23 MB
prod-persona	270	1.17 GB
prod-share	7	4.88 kB

Hình 2.11: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



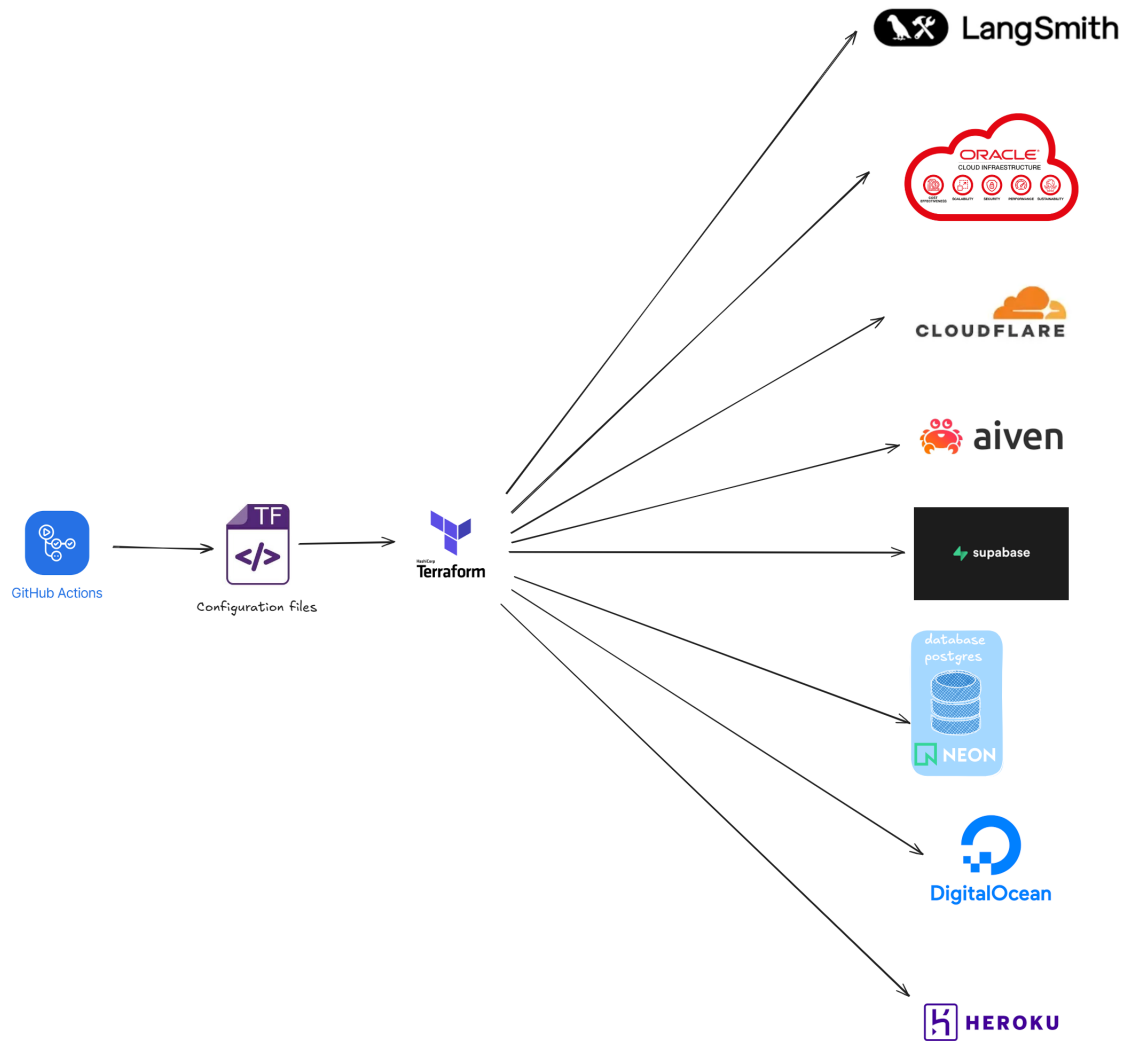
prod-code-payment-service

Project dashboard

Compute	Storage	Memory	Network transfer
81.03 / 100 CU-hrs	0.03 / 0.5 GB	0 GB	0.09 / 5 GB

Usage since Jun 1, 2024. Metrics may be delayed by an hour and may not be updated for inactive projects. Learn more.

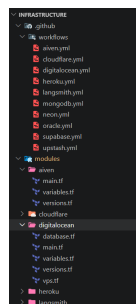
Hình 2.12: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



Hình 2.13: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Tại dự án <https://github.com/20206205Tech/infra-by-terraform> dùng root module lớn truyền biến vào các module nằm trong thư mục modules. Tuy nhiên việc chạy CI/CD có thể xảy ra lỗi vì provider bị lỗi, hoặc mất thời gian chỉ vì thay đổi nhỏ cần terraform init với nhiều provider không thay đổi. Vì vậy, trong đề án này, em sử dụng mỗi provider trong folder riêng Tại dự án <https://github.com/20206205Tech/infrastructure>. Kết hợp github action chỉ chạy terraform khi có sự thay đổi của folder.

Cấu trúc dự án terraform với các folder riêng



Hình 2.14: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Sử dụng Terraform Cloud để làm Remote Backend

Workspaces

All filters

0 Need attention
1 Errored
0 Running
0 On hold
10 Completed

No filters applied

Workspace name	Repository	Health	Project	Latest change	Manage
infra-given Applied	None	None	Default Project	8 days ago	⋮
infra-cloudflare Applied	None	None	Default Project	5 days ago	⋮
infra-digitalocean Applied	None	None	Default Project	a month ago	⋮
infra-heroku Applied	None	None	Default Project	a month ago	⋮

Hình 2.15: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

2.3.4 API Gateway trong Microservice (Kong API Gateway)

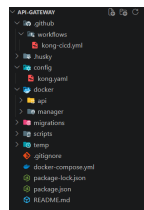
API Gateway được sử dụng để quản lý và kiểm soát việc truy cập vào các API. API Gateway cung cấp một lớp trung gian giữa client và các microservices. Có thể nói, API Gateway là một điểm vào độc lập cho nhiều client của một ứng dụng.

Kong API Gateway là một trong những API Gateway mã nguồn mở phổ biến có thể quản lý các API được triển khai ở bất kỳ đâu, từ cơ sở hạ tầng đơn giản đến môi trường đa đám mây phức tạp.



Hình 2.16: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Cấu trúc dự án Kong API Gateway

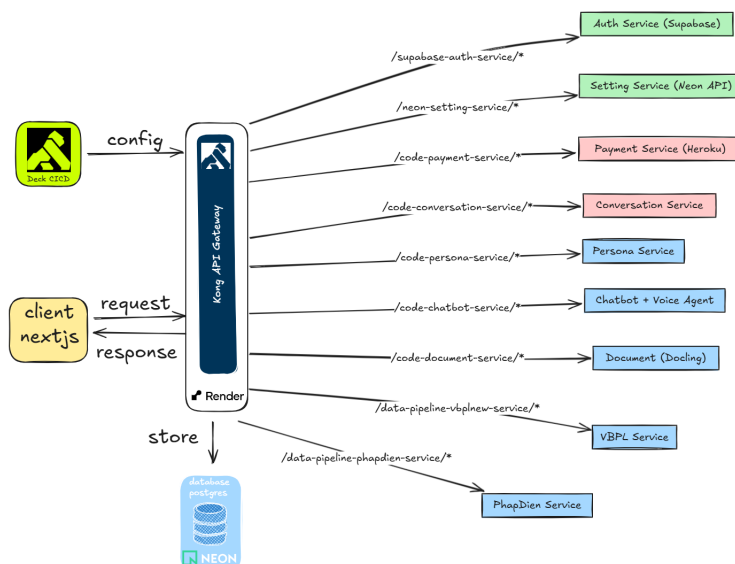


Hình 2.17: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

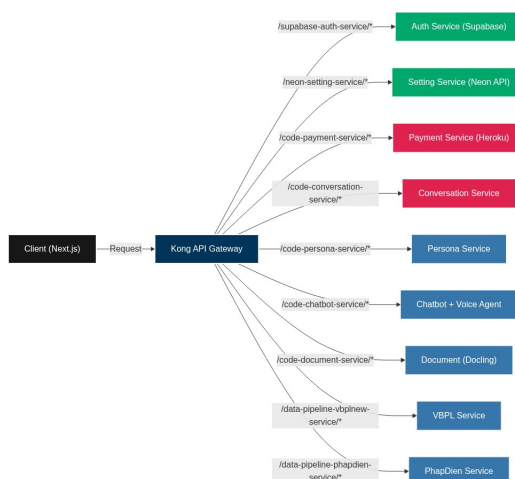
[illegible]

Hình 2.18: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Sơ đồ quản lý Kong API Gateway



Hình 2.19: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



Hình 2.20: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Có nhiều cách quản lý => ưu điểm của DECK

Các plugins trong dự án này:

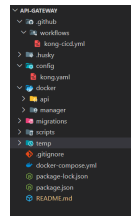
- Plugin bot-detection: phân tích User-Agent và các dấu hiệu khác để chặn các request từ các bot độc hại, scraper, hoặc các công cụ tự động không mong muốn.
- Plugin rate-limiting: giới hạn số lượng request, giúp bảo vệ các dịch vụ khỏi bị quá tải hoặc bị lạm dụng.
- Plugin request-size-limiting: chặn các request có kích thước vượt quá ngưỡng cho phép.
- Plugin http-log: thu thập các thông tin quan trọng về request/response và gửi qua giao thức http api NestJS và thực hiện lưu vào database postgres
- Plugin correlation-id tạo request-id: tự động tạo ra một mã định danh duy nhất và gắn vào header của mỗi request để truy vết (tracing).
- Plugin request-transformer thêm thông tin Request-Kong-Secret: gửi header chứa mã bí mật vào request trước khi đẩy xuống dịch vụ để các request phải được gọi từ Kong API Gateway.

Name	Applied To	Status
Request Transformer	Global	Enabled
Request Transformer	Public	Enabled
Request Termination	Public	Enabled
Request Transformer	Service	Enabled
Correlation ID	Global	Enabled
Request Size Limiting	Global	Enabled
Bot Detection	Global	Enabled
Rate Limiting	Global	Enabled
HTTP Log	Global	Enabled
Request Transformer	Public	Enabled
Request Transformer	Service	Enabled

Hình 2.21: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



Hình 2.22: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler



Hình 2.23: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

2.3.5 Container và GitOps (Docker, Kubernetes, ArgoCD)

Docker: Hệ thống chạy tốt trên máy của lập trình viên nhưng lỗi khi lên server do khác biệt về hệ điều hành hoặc thư viện. Docker đóng gói ứng dụng cùng với các thư viện, file cấu hình và môi trường cần thiết vào một hộp kín gọi là Container.

Kubernetes (K8s): Khi có nhiều container trên các server khác nhau, nếu một server down thì sao? Nếu đột nhiên lượng truy cập tăng vọt thì làm thế nào? Kubernetes là một hệ thống quản lý container (container orchestration). Giúp tự động hóa việc triển khai, mở rộng quy mô và quản lý các container.

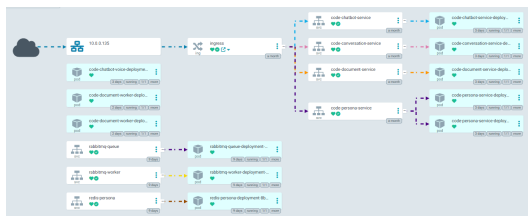
GitOps với ArgoCD: Việc cấu hình hệ thống bằng cách gõ lệnh tay rất dễ dẫn đến sai sót, khó theo dõi ai đã thay đổi gì, và cực kỳ khó khôi phục nếu hệ thống sập. GitOps là một phương pháp sử dụng manifest của Git (như GitHub, GitLab) là nguồn chân lý duy nhất (Single Source of Truth). Các file yaml của K8s đều được lưu trên Git. ArgoCD nhận diện thay đổi và tự động triển khai (deploy) hoặc tình chỉnh hạ tầng thực tế cho khớp với Git.

Kết quả các docker image trên Docker Hub

Repositories				
All repositories within the vuamghiaa namespace.				
Search by repository name	All content	Create a repository		
Name	Last Pushed	Contains	Visibility	Scout
vuamghiaa/code-conversation-service	about 3 hours ago	MANIFEST, CACHE, IMAGE	Public	Inactive
vuamghiaa/code-document-worker	about 7 hours ago	IMAGE	Public	Inactive
vuamghiaa/code-chatbot-service	about 13 hours ago	MANIFEST, CACHE, IMAGE	Private	Inactive
vuamghiaa/code-persona-service	about 13 hours ago	IMAGE	Public	Inactive
vuamghiaa/code-document-service	1 day ago	MANIFEST, CACHE, IMAGE	Public	Inactive
vuamghiaa/data-pipeline-vbpl-service	3 days ago	MANIFEST, CACHE, IMAGE	Public	Inactive

Hình 2.24: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

Kết quả giao diện ArgoCD chạy docker trong kubernetes



Hình 2.25: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

2.3.6 Giao tiếp giữa các services

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.4 Đường ống dữ liệu

Đường ống dữ liệu bao gồm các chuỗi quy trình xử lý tự động từ một hoặc nhiều nguồn (source) đến một đích đến (destination) để lưu trữ, phân tích hoặc phục vụ cho các ứng dụng khác.

Vì dự án này làm về miền nghiệp vụ pháp luật nên không cần thu thập dữ liệu từ nhiều nguồn khác nhau. Em sử dụng dữ liệu chính thống từ nhà nước là **Văn bản pháp luật** tại địa chỉ <https://vbpl.vn> và dữ liệu **Pháp Điển** tại địa chỉ <https://phapdien.moj.gov.vn/Pages/chi-tiet-bo-phap-dien.aspx>

Giao diện nguồn dữ liệu Văn bản pháp luật



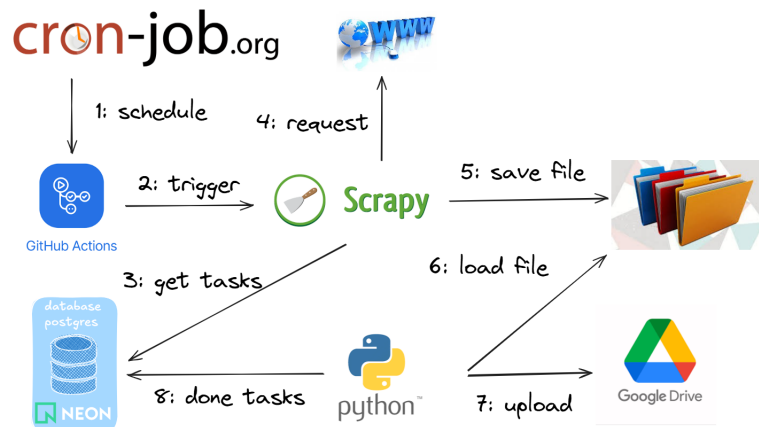
Hình 2.26: Giao diện nguồn dữ liệu Văn bản pháp luật

Giao diện nguồn dữ liệu Pháp Điển



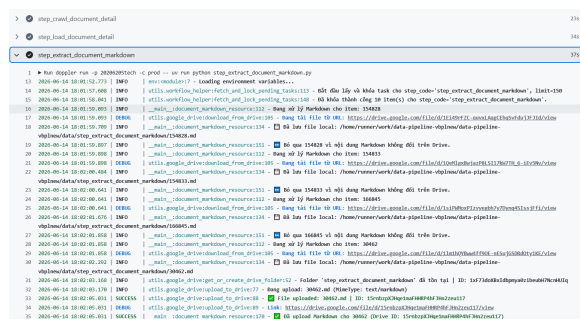
Hình 2.27: Giao diện nguồn dữ liệu Pháp Điển

Sơ đồ thu thập dữ liệu chung



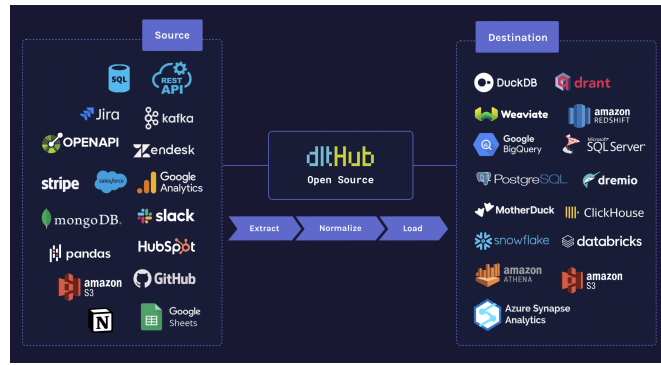
Hình 2.28: Sơ đồ thu thập dữ liệu chung

Tại các bước sẽ sử dụng logic so sánh hash để bỏ qua văn bản không thay đổi nội dung giúp tiết kiệm tài nguyên và thời gian xử lý dữ liệu.



Hình 2.29: Sơ đồ thu thập dữ liệu chung

Thư viện dlt (Data Load Tool) là một thư viện Python mã nguồn mở giúp trích xuất dữ liệu từ nhiều nguồn khác nhau và tải chúng vào các tập dữ liệu có cấu trúc rõ ràng.



Hình 2.30: Giao diện nguồn dữ liệu Văn bản pháp luật

Việc sử dụng Data Load Tool hỗ trợ tự động nội suy cấu trúc, kiểu dữ liệu và xử lý mượt mà các cấu trúc dữ liệu lồng nhau hỗ trợ tốt việc thay đổi nếu nguồn dữ liệu có sự thay đổi



Hình 2.31: Data Load Tool tự động thích ứng nguồn dữ liệu có sự thay đổi

2.4.1 Đường ống dữ liệu Văn bản pháp luật

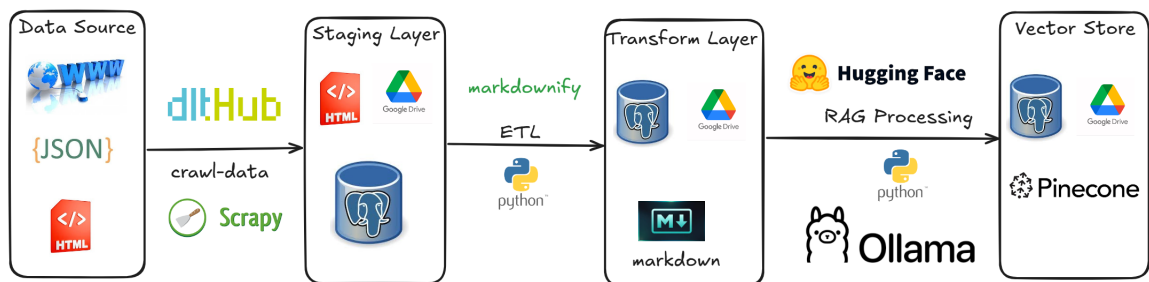
Khi làm việc với bài toán dữ liệu thực tế, việc thu thập dữ liệu có thách thức phụ thuộc vào nguồn dữ liệu. Vào ngày 24/05/2026, nguồn dữ liệu Văn bản pháp luật được thay đổi cả hệ thống, API và giao diện. Do sự thay đổi lớn về dữ liệu thu thập nên hệ thống cần xử lý lại các bước Data pipeline.

Giao diện nguồn dữ liệu Văn bản pháp luật mới



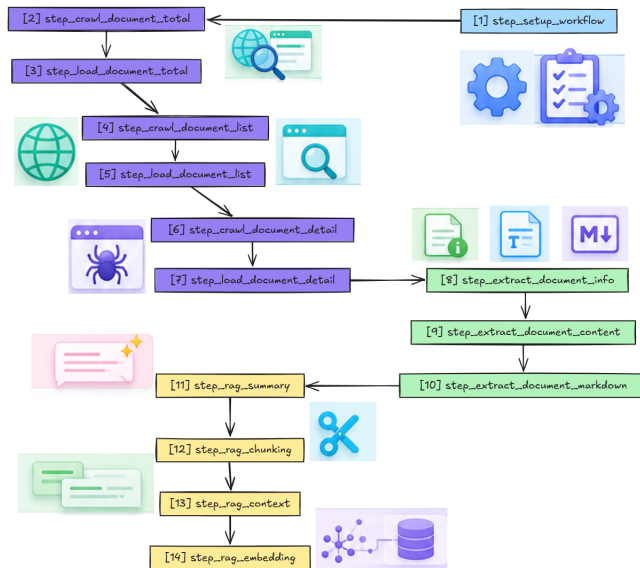
Hình 2.32: Sơ đồ thu thập dữ liệu chung

Sơ đồ tổng quan Data pipeline Văn bản pháp luật



Hình 2.33: Sơ đồ thu thập dữ liệu chung

Các bước thực hiện cũ



Hình 2.34: Sơ đồ thu thập dữ liệu chung

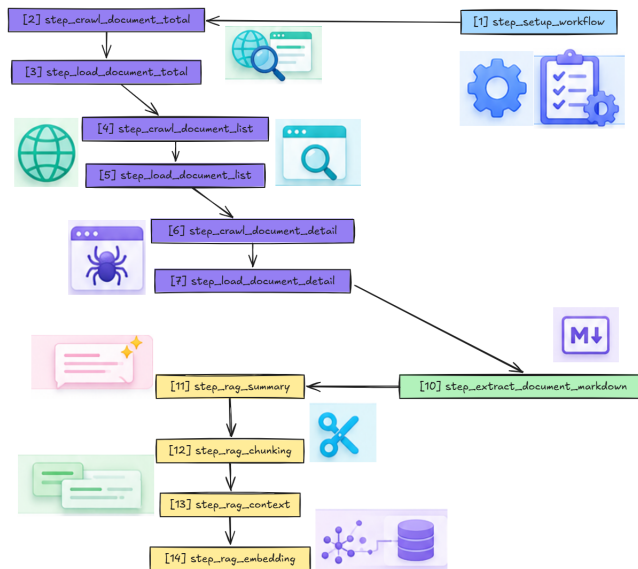
Mô tả các bước thực hiện cũ:

Quá trình Data pipeline được chia thành 3 giai đoạn: Thu thập dữ liệu, Tiền xử lý văn bản và Tích hợp RAG Pipeline.

- Giai đoạn Khởi tạo:
 - step_setup_workflow cập nhật danh sách và thứ tự các bước (workflow) vào cơ sở dữ liệu PostgreSQL
- Giai đoạn Thu thập Dữ liệu:
 - step_crawl_document_total và step_load_document_total: thu thập tổng số lượng văn bản pháp luật hiện có trên hệ thống web.
 - step_crawl_document_list và step_load_document_list: Dựa vào sự chênh lệch tổng số lượng, tính toán số trang cần quét và thu thập danh sách các item_id.
 - step_crawl_document_detail và step_load_document_detail: tải toàn bộ mã nguồn HTML trang chi tiết của từng văn bản và Upload các file HTML gốc này lên Google Drive, đồng thời lưu drive_id và mã băm MD5 hash vào database để quản lý thay đổi.
- Giai đoạn Tiền xử lý Dữ liệu:
 - step_extract_document_info: từ file HTML, bóc tách các siêu dữ liệu quan trọng (Metadata) như: Số hiệu, Cơ quan ban hành, Ngày có hiệu lực, Người ký, v.v., và lưu vào database.
 - step_extract_document_content: Làm sạch file HTML, cắt bỏ các phần thừa (header, footer, menu) và chỉ giữ lại phần nội dung (thẻ div#toanvancontent).
 - step_extract_document_markdown: Chuyển đổi file HTML đã làm sạch sang định dạng Markdown.
- Giai đoạn Tích hợp RAG pipeline
 - step_rag_summary: Sử dụng LLM viết một bản tóm tắt ngắn gọn mô tả bức tranh toàn cảnh của văn bản pháp luật đó.
 - step_rag_chunking: Chạy Semantic Chunking. LLM sẽ dựa vào bản tóm tắt và nội dung chi tiết để quyết định các "điểm cắt" tối ưu, chia nhỏ văn bản thành các đoạn (chunks) sao cho không bị đứt gãy ý nghĩa.
 - step_rag_context: Đây là bước chống mất ngữ cảnh. LLM sẽ sinh ra ngữ cảnh cho từng đoạn chunk nhỏ vừa được cắt, đảm bảo mỗi chunk đều có thể đứng độc lập khi được tìm kiếm.

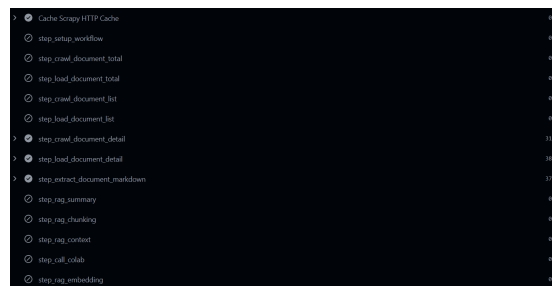
- `step_rag_embedding`: Sử dụng mô hình HuggingFace (`keepitreal/vietnamese-sbert`) để biến đổi các đoạn văn bản (đã có ngữ cảnh) thành vector đa chiều và lưu trữ vào VectorDB (Pinecone).

Các bước thực hiện mới



Hình 2.35: Sơ đồ thu thập dữ liệu chung

- Nhờ vào việc nguồn dữ liệu thay đổi sang hệ thống API mới trả về cấu trúc JSON giàu thông tin, hệ thống không còn cần các bước bóc tách thủ công từ mã nguồn HTML phức tạp, giúp tăng tốc độ xử lý và độ chính xác của dữ liệu.
- Hệ thống loại bỏ bước `step_extract_document_info` vì nội dung Metadata chi tiết đã có trong API
- Hệ thống loại bỏ bước `step_extract_document_content` vì nội dung văn bản đã được API trả trong thông tin `documentContent`

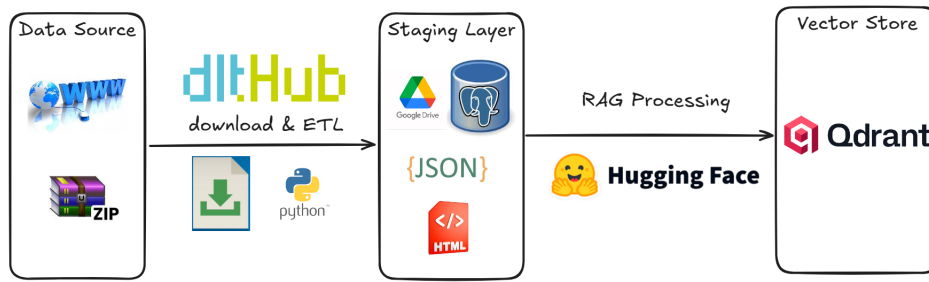


Hình 2.36: Sơ đồ thu thập dữ liệu chung

2.4.2 Đường ống dữ liệu Pháp Điển

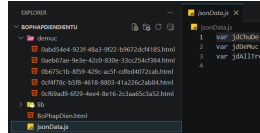
Pháp điển là tập hợp các quy phạm pháp luật đang còn hiệu lực của các văn bản quy phạm pháp luật do cơ quan nhà nước ở trung ương ban hành, từ Thông tư trở lên và trừ Hiến pháp.

Sơ đồ tổng quan Data pipeline Pháp Điển



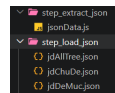
Hình 2.37: Sơ đồ thu thập dữ liệu chung

Tổng quan thông tin file zip



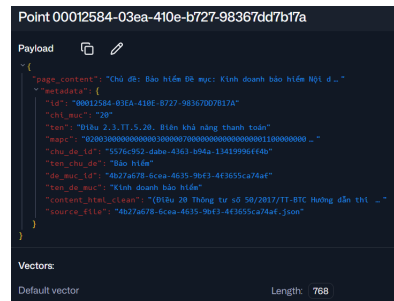
Hình 2.38: Sơ đồ thu thập dữ liệu chung

Kết quả file JSON đã trích xuất



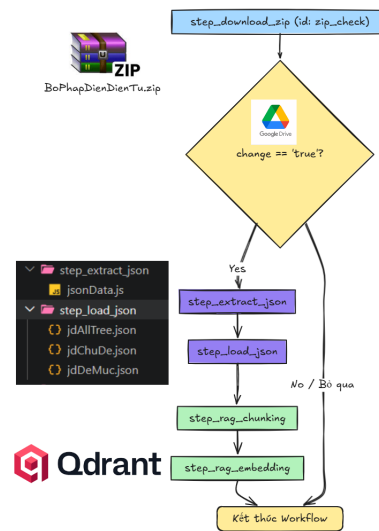
Hình 2.39: Sơ đồ thu thập dữ liệu chung

Kết quả vector được lưu trong vector database



Hình 2.40: Sơ đồ thu thập dữ liệu chung

Các bước thực hiện



Hình 2.41: Sơ đồ thu thập dữ liệu chung

Mô tả các bước thực hiện:

Quá trình Data pipeline được chia thành 3 giai đoạn: Thu thập dữ liệu, Tiền xử lý văn bản và Tích hợp RAG Pipeline.

- Giai đoạn Thu thập Dữ liệu:

- step_download_zip: Tự động tải file nén chứa toàn bộ Bộ pháp điển điện tử (BoPhapDienDi-enTu.zip) từ nguồn Cổng thông tin điện tử pháp điển. Hệ thống tính toán mã băm (MD5) và đồng bộ với Google Drive để kiểm tra sự thay đổi dữ liệu.

- Giai đoạn Tiền xử lý Dữ liệu:

- `step_extract_json` và `step_load_json`: Từ file `jsonData.js`. Tiến hành bóc tách (parse) các biến cấu trúc dữ liệu để lấy danh sách Chủ đề (`jdChuDe`), Đề mục (`jdDeMuc`), và Cây mục lục (`jdAll-Tree`), sau đó xuất ra định dạng JSON.

- Giai đoạn Tích hợp RAG Pipeline:

- `step_rag_chunking`: Thực hiện đối chiếu từ Cây mục lục (`jdAllTree`) với các thẻ bên trong từng file HTML chi tiết. Từ đó, bóc tách chính xác từng phần nội dung văn bản thành dạng JSON có các đoạn văn bản và siêu dữ liệu (Metadata).

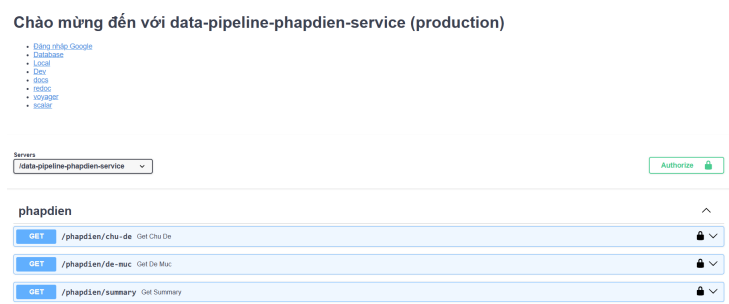
- step_rag_embedding: Làm giàu ngữ cảnh bằng cách nối trực tiếp tên Chủ đề và Đề mục vào phần đầu của mỗi đoạn văn bản chunk. Sau đó, nội dung được đưa qua mô hình HuggingFace (keepitreal/vietnamese-sbert) để biến đổi thành các vector cơ sở dữ liệu vector Qdrant.

```

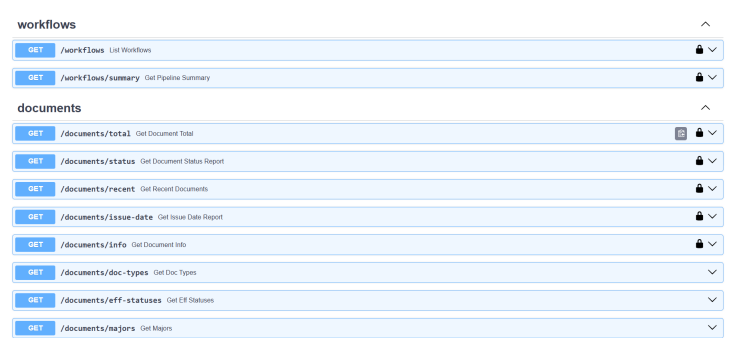
1  • Set up Python
2  • Install dependencies
3  • step_download_zip
4  • Run deployer run -p 30000 --watch -c good -- --run python step_download_zip.py
5  • main_download_zip
6  • step_download_zip
7  • step_download_zip
8  • step_download_zip
9  • step_download_zip
10 • step_download_zip
11 • step_download_zip
12 • step_download_zip
13 • step_download_zip
14 • step_download_zip
15 • step_download_zip
16 • step_download_zip
17 • step_download_zip
18 • step_download_zip
19 • step_download_zip
20 • step_download_zip
21 • step_download_zip
22 • step_download_zip
23 • step_download_zip
24 • step_download_zip
25 • step_download_zip
26 • step_download_zip
27 • step_download_zip
28 • step_download_zip
29 • step_download_zip
30 • step_download_zip
31 • step_download_zip
32 • step_download_zip
33 • step_download_zip
34 • step_download_zip
35 • step_download_zip
36 • step_download_zip
37 • step_download_zip
38 • step_download_zip
39 • step_download_zip
40 • step_download_zip
41 • step_download_zip
42 • step_download_zip
43 • step_download_zip
44 • step_download_zip
45 • step_download_zip
46 • step_download_zip
47 • step_download_zip
48 • step_download_zip
49 • step_download_zip
50 • step_download_zip
51 • step_download_zip
52 • step_download_zip
53 • step_download_zip
54 • step_download_zip
55 • step_download_zip
56 • step_download_zip
57 • step_download_zip
58 • step_download_zip
59 • step_download_zip
60 • step_download_zip
61 • step_download_zip
62 • step_download_zip
63 • step_download_zip
64 • step_download_zip
65 • step_download_zip
66 • step_download_zip
67 • step_download_zip
68 • step_download_zip
69 • step_download_zip
70 • step_download_zip
71 • step_download_zip
72 • step_download_zip
73 • step_download_zip
74 • step_download_zip
75 • step_download_zip
76 • step_download_zip
77 • step_download_zip
78 • step_download_zip
79 • step_download_zip
80 • step_download_zip
81 • step_download_zip
82 • step_download_zip
83 • step_download_zip
84 • step_download_zip
85 • step_download_zip
86 • step_download_zip
87 • step_download_zip
88 • step_download_zip
89 • step_download_zip
90 • step_download_zip
91 • step_download_zip
92 • step_download_zip
93 • step_download_zip
94 • step_download_zip
95 • step_download_zip
96 • step_download_zip
97 • step_download_zip
98 • step_download_zip
99 • step_download_zip
100 • step_download_zip

```

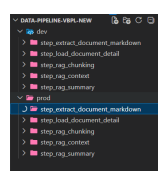
Hình 2.42: Sơ đồ thu thập dữ liệu chung



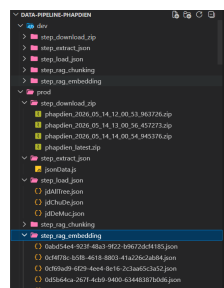
Hình 2.43: swagger



Hình 2.44: swagger



Hình 2.45: swagger

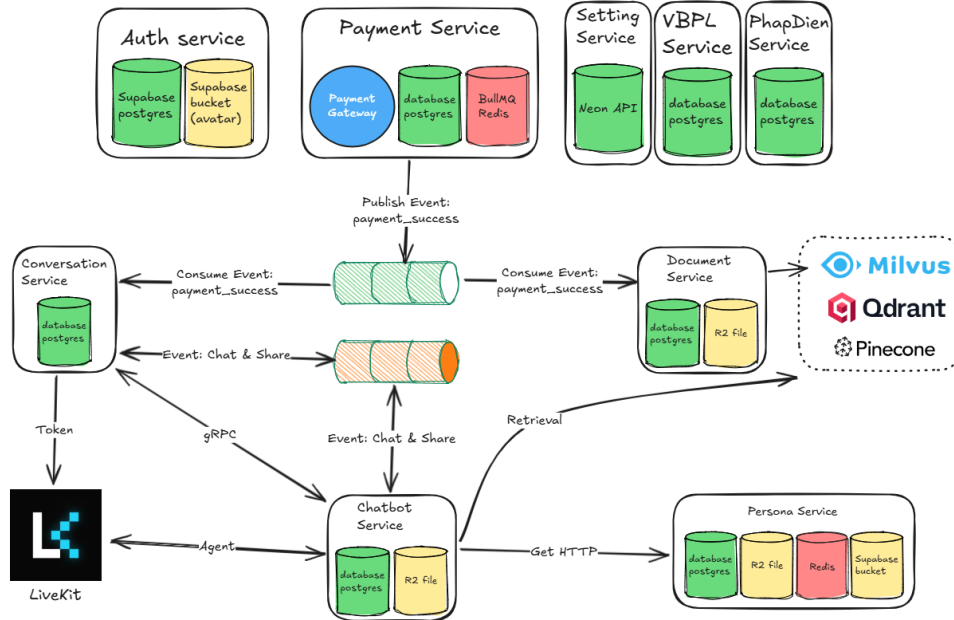


Hình 2.46: swagger

3 Triển khai hệ thống

3.1 Triển khai các dịch vụ

3.2 Tổng quan các Microservice của hệ thống



Hình 3.1: Sơ đồ tổng quan các Microservice

Tất cả các dịch vụ đều dùng `https://[PROJECT_REF].supabase.co/auth/v1/.well-known/jwks.json` để xác thực thông tin JWT của người dùng.

Tất cả các dịch vụ đều dùng cơ sở dữ liệu Postgres của Neon để làm cơ sở dữ liệu riêng theo đúng mẫu cơ sở dữ liệu cho mỗi dịch vụ (database per service pattern). Mỗi dịch vụ sẽ sở hữu và quản lý hoàn toàn cơ sở dữ liệu của riêng nó.

28 Projects

service

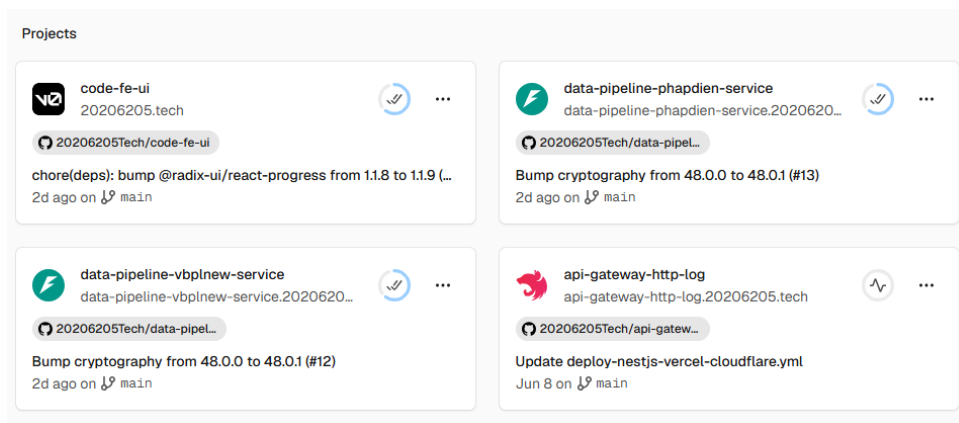
Name	Region	Created at	Storage	Compute last active at	Branches	Integrations
prod-code-persona-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.86 MB	An hour ago	1	Add +
dev-code-persona-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.85 MB	16 hours ago	1	Add +
dev-code-document-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.6 MB	11 days ago	1	Add +
prod-code-document-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.33 MB	12 hours ago	1	Add +
dev-code-chatbot-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.3 MB	2 days ago	1	Add +
prod-code-chatbot-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.93 MB	12 hours ago	1	Add +
prod-code-conversation-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.1 MB	A few seconds ago	1	Add +
dev-code-conversation-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.15 MB	2 days ago	1	Add +
dev-code-payment-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.99 MB	3 minutes ago	1	Add +
prod-code-payment-service	AWS Asia Pacific 1 (Singapore)	A month ago	33.04 MB	A few seconds ago	1	Add +

Items per page: 10

Hình 3.2: Bảng điều khiển cơ sở dữ liệu Neon

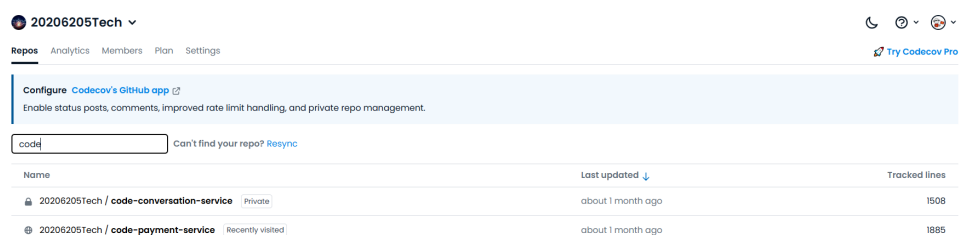
Các dịch vụ ngoại trừ auth và setting được cấu hình trong Kong API Gateway đơn giản chỉ cần tạo Service và Route để định tuyến request. Dịch vụ auth và setting sẽ được mô tả chi tiết về cách cấu hình trong Kong API Gateway trong nội dung trình bày của chính dịch vụ đó.

Trang thống kê thông tin đường ống dữ liệu (Data Pipeline) dành cho quản trị viên (ADMIN) có tần suất truy vấn thấp nên sẽ được triển khai trên Vercel.



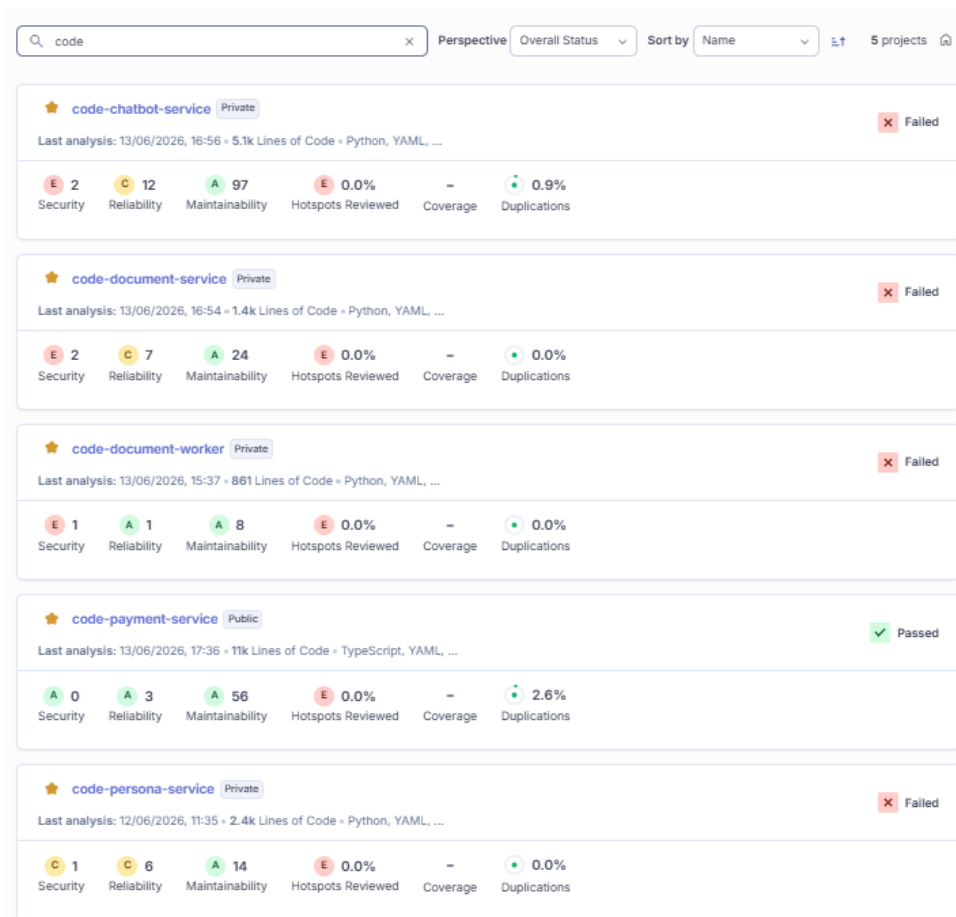
Hình 3.3: Dự án trên Vercel

Các dịch vụ NestJS sử dụng Codecov để theo dõi độ bao phủ kiểm thử (test coverage).



Hình 3.4: Thống kê độ bao phủ kiểm thử trên Codecov

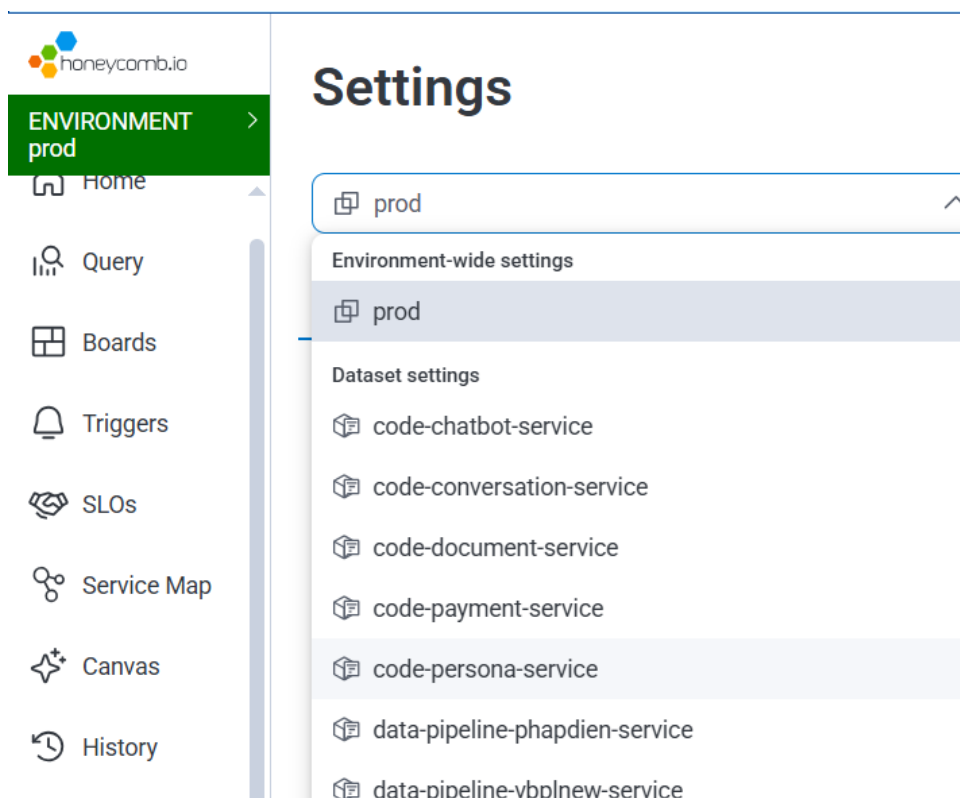
Tích hợp SonarQube để phân tích chất lượng mã nguồn và các lỗ hổng bảo mật.



Hình 3.5: Giao diện phân tích mã nguồn trên SonarQube

3.2.1 Tracing và Quan sát hệ thống (Observability)

Hệ thống sử dụng **OpenTelemetry (OTel)** tích hợp trong tất cả các dịch vụ để tự động thu thập dấu vết phân tán (distributed traces) và gửi dữ liệu về Honeycomb. Tích hợp Honeycomb để theo dõi vết (distributed tracing) giữa các dịch vụ.



Hình 3.6: Giao diện theo dõi vết distributed tracing trên Honeycomb

3.3 Dịch vụ xác thực (auth service - dùng Supabase)

Hệ thống sử dụng Supabase để xử lý các nhóm chức năng chính bao gồm: xác thực và phân quyền, xác thực 2 yếu tố (MFA), quản lý dữ liệu người dùng và quản lý tệp tin (lưu trữ các file ảnh đại diện do người dùng tải lên tại bucket `avatars`).

Do mỗi lần người dùng đăng nhập lại, Supabase/Google sẽ ghi đè lại thông tin gốc từ tài khoản liên kết, bảng `profiles` được thiết kế riêng để lưu trữ và duy trì thông tin cập nhật mới nhất của người dùng.

Mô tả chi tiết các chức năng:

3.3.1 I. Xác thực và Quản lý tài khoản

Hệ thống sử dụng các API Auth của Supabase để xử lý toàn bộ luồng đăng nhập, đăng ký và bảo mật tài khoản:

- **Đăng nhập bằng Google:**
 - Tạo URL và chuyển hướng người dùng đến trang ủy quyền của Google thông qua Supabase Auth Provider.
 - Endpoint sử dụng: `/auth/v1/authorize?provider=google&redirect_to={redirectUrl}`
- **Đăng nhập bằng Email và Mật khẩu:**
 - Cho phép người dùng đăng nhập bằng tài khoản email. Trả về thông tin Access Token và Refresh Token nếu thành công.
 - Endpoint sử dụng: `/auth/v1/token?grant_type=password`
- **Đăng ký tài khoản mới:**
 - Tạo tài khoản mới bằng Email và Mật khẩu, hỗ trợ gửi email xác nhận tài khoản thông qua đường dẫn chuyển hướng sau khi kích hoạt thành công (`redirect_to`).
 - Endpoint sử dụng: `/auth/v1/signup`
- **Khôi phục mật khẩu:**

- Gửi yêu cầu khôi phục mật khẩu tới email của người dùng. Hệ thống sẽ gửi một email chứa liên kết để xác thực và chuyển hướng người dùng về trang đổi mật khẩu.
- Endpoint sử dụng: `/auth/v1/recover`
- **Cập nhật mật khẩu mới:**
 - Cho phép người dùng đổi mật khẩu mới sau khi đã đăng nhập (hoặc sau khi click vào liên kết khôi phục mật khẩu).
 - Endpoint sử dụng: `/auth/v1/user`
- **Làm mới Access Token:**
 - Sử dụng `refresh_token` để lấy một `access_token` mới khi token cũ hết hạn mà không bắt người dùng phải đăng nhập lại.
 - Endpoint sử dụng: `/auth/v1/token?grant_type=refresh_token`
- **Đăng xuất:**
 - Hủy phiên làm việc hiện tại của người dùng trên hệ thống Supabase Auth.
 - Endpoint sử dụng: `/auth/v1/logout`

3.3.2 II. Xác thực 2 yếu tố (MFA)

Hệ thống hỗ trợ phương thức xác thực hai yếu tố TOTP (Time-based One-Time Password - như Google Authenticator hay Authy) bằng các API nâng cao của Supabase:

- **Đăng ký thiết bị xác thực MFA mới:**
 - Khởi tạo quá trình liên kết thiết bị xác thực mới. API trả về mã QR (`qr_code`), mã bí mật (`secret`), và chuỗi URI để người dùng quét trên ứng dụng Authenticator.
 - Endpoint sử dụng: `/auth/v1/factors`
- **Tạo thử thách xác thực:**
 - Tạo một thử thách xác thực (challenge) dựa trên ID thiết bị MFA (`factorId`) để chuẩn bị so khớp với mã TOTP người dùng nhập.
 - Endpoint sử dụng: `/auth/v1/factors/{factorId}/challenge`
- **Xác thực mã TOTP:**
 - Kiểm tra mã TOTP gồm 6 chữ số người dùng nhập có khớp với thử thách hiện tại không. Nếu đúng, Supabase sẽ nâng cấp phiên làm việc của người dùng (tăng cấp bảo mật cho Access Token lên mức độ xác thực `aal2`).
 - Endpoint sử dụng: `/auth/v1/factors/{factorId}/verify`
- **Lấy danh sách các yếu tố MFA:**
 - Lấy thông tin tài khoản người dùng để lọc ra danh sách các thiết bị/yếu tố xác thực đã liên kết, phân loại thành: tất cả thiết bị (`all`) và các thiết bị đã kích hoạt thành công (`active`).
 - Endpoint sử dụng: `/auth/v1/user`
- **Hủy liên kết/Xóa thiết bị MFA:**
 - Xóa một thiết bị xác thực MFA khỏi tài khoản người dùng.
 - Endpoint sử dụng: `/auth/v1/factors/{factorId}`
- **Cập nhật tên hiển thị của thiết bị MFA:**
 - Đổi tên hiển thị (`friendly_name`) của thiết bị xác thực (ví dụ: "Điện thoại cá nhân").
 - Endpoint sử dụng: `/auth/v1/factors/{factorId}`

3.3.3 III. Quản lý Hồ sơ người dùng (Supabase Database - PostgREST API)

Hệ thống sử dụng cơ chế RESTful API tự động sinh từ PostgreSQL của Supabase (PostgREST) để thực hiện các thao tác CRUD trên bảng cơ sở dữ liệu `profiles`:

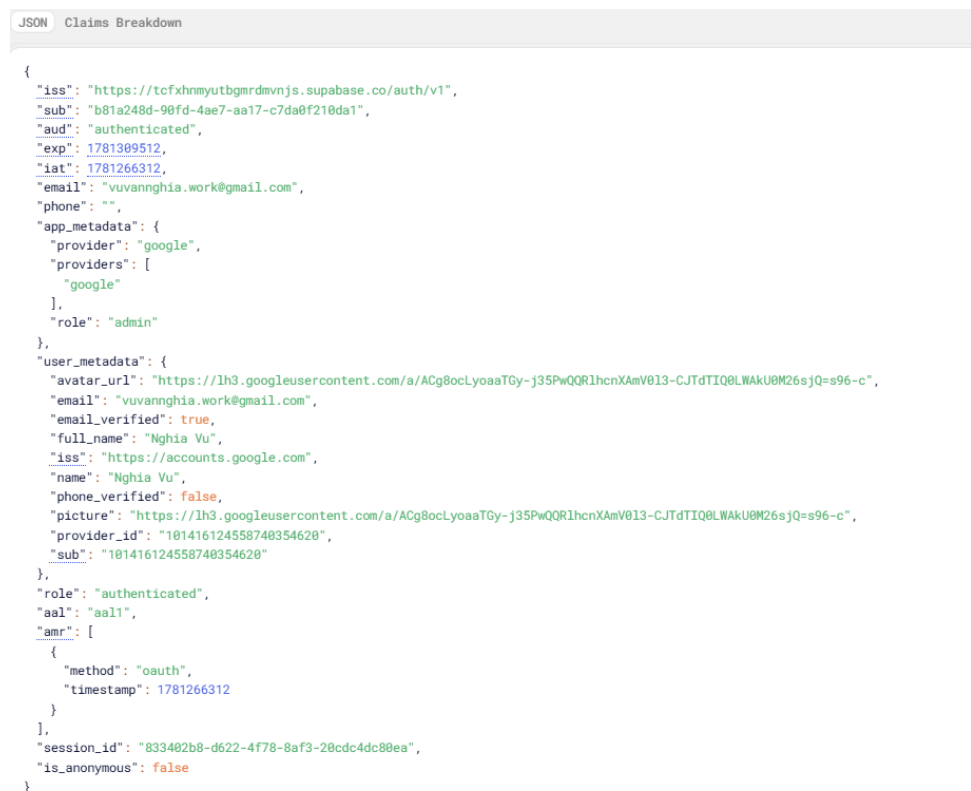
- **Lấy thông tin hồ sơ:**
 - Truy vấn thông tin chi tiết hồ sơ người dùng (như tên đầy đủ, ảnh đại diện) từ bảng `profiles` dựa vào `userId`.
 - Endpoint sử dụng: `/rest/v1/profiles?id=eq.{userId}`
- **Cập nhật thông tin hồ sơ:**
 - Cập nhật các trường thông tin hồ sơ như tên hiển thị (`full_name`) hoặc đường dẫn ảnh đại diện (`avatar_url`).
 - Endpoint sử dụng: `/rest/v1/profiles?id=eq.{userId}`

3.3.4 IV. Quản lý Ảnh đại diện (Supabase Storage)

Hệ thống sử dụng dịch vụ lưu trữ tệp (Storage) của Supabase để quản lý hình ảnh của người dùng:

- **Tải lên ảnh đại diện:**
 - Tải tệp tin ảnh của người dùng lên thư mục `avatars` trên Supabase Storage. Tên tệp được sinh ngẫu nhiên kết hợp với `userId` để tránh trùng lặp.
 - Endpoint tải lên: `/storage/v1/object/avatars/{fileName}`
 - Đường dẫn công khai nhận về: `/storage/v1/object/public/avatars/{fileName}`

Thông tin payload của JWT từ Supabase:



```

JSON Claims Breakdown

{
  "iss": "https://tcfxhnmvutbgrdmvnjs.supabase.co/auth/v1",
  "sub": "b81a248d-90fd-4ae7-aa17-c7da0f210da1",
  "aud": "authenticated",
  "exp": 1781309512,
  "iat": 1781266312,
  "email": "vuvanngia.work@gmail.com",
  "phone": "",
  "app_metadata": {
    "provider": "google",
    "providers": [
      "google"
    ]
  },
  "role": "admin",
  "user_metadata": {
    "avatar_url": "https://lh3.googleusercontent.com/a/ACg8ocLyooaTGy-j35PwQQRlhcXAmV013-CJTdTlQ0LWakU0M26sjQ=s96-c",
    "email": "vuvanngia.work@gmail.com",
    "email_verified": true,
    "full_name": "Nghia Vu",
    "iss": "https://accounts.google.com",
    "name": "Nghia Vu",
    "phone_verified": false,
    "picture": "https://lh3.googleusercontent.com/a/ACg8ocLyooaTGy-j35PwQQRlhcXAmV013-CJTdTlQ0LWakU0M26sjQ=s96-c",
    "provider_id": "101416124558748354620",
    "sub": "101416124558748354620"
  },
  "role": "authenticated",
  "aal": "aal1",
  "amr": [
    {
      "method": "oauth",
      "timestamp": 1781266312
    }
  ],
  "session_id": "833402b8-d622-4f78-8af3-20cdc4dc88ea",
  "is_anonymous": false
}

```

Hình 3.7: Thông tin JWT Payload từ Supabase

Tham khảo tài liệu của Supabase tại:

<https://supabase.com/docs/guides/auth/auth-mfa/totp>

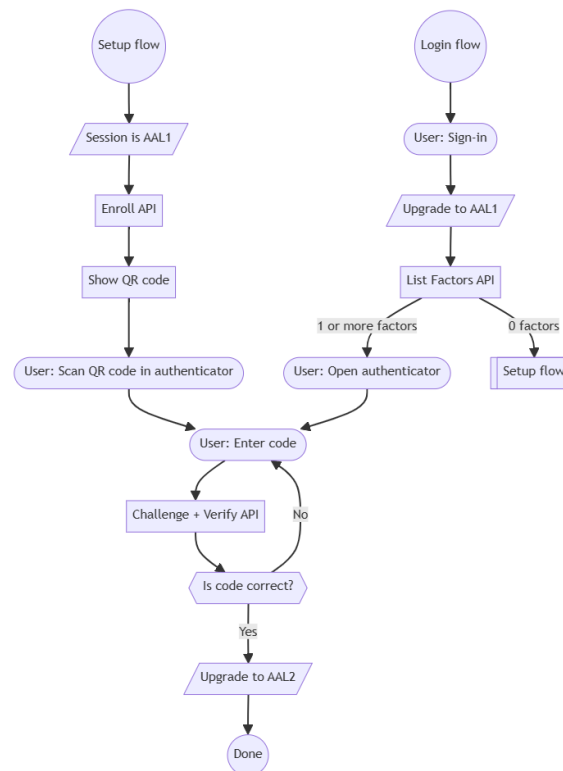
Trang tài liệu này hướng dẫn cách triển khai tính năng Xác thực Đa Yếu Tố (MFA) bằng ứng dụng Authenticator (TOTP) trong Supabase. Tính năng này hiện được cung cấp miễn phí và bật mặc định cho mọi dự án Supabase. Nội dung chính xoay quanh hai luồng hoạt động cơ bản:

Quy trình đăng ký yếu tố bảo mật (Enrollment Flow)

1. Khởi tạo quá trình đăng ký.
2. Hệ thống sẽ trả về một mã QR và một chuỗi mã bí mật.
3. Người dùng sẽ dùng ứng dụng xác thực (như Google Authenticator) để quét mã QR này.
4. Chuẩn bị hệ thống để tiếp nhận mã xác minh, đồng thời trả về một challenge ID.
5. Đối chiếu mã 6 chữ số mà người dùng nhập vào từ ứng dụng xác thực.
6. Nếu mã hợp lệ, yếu tố bảo mật MFA này sẽ chính thức được kích hoạt cho tài khoản.

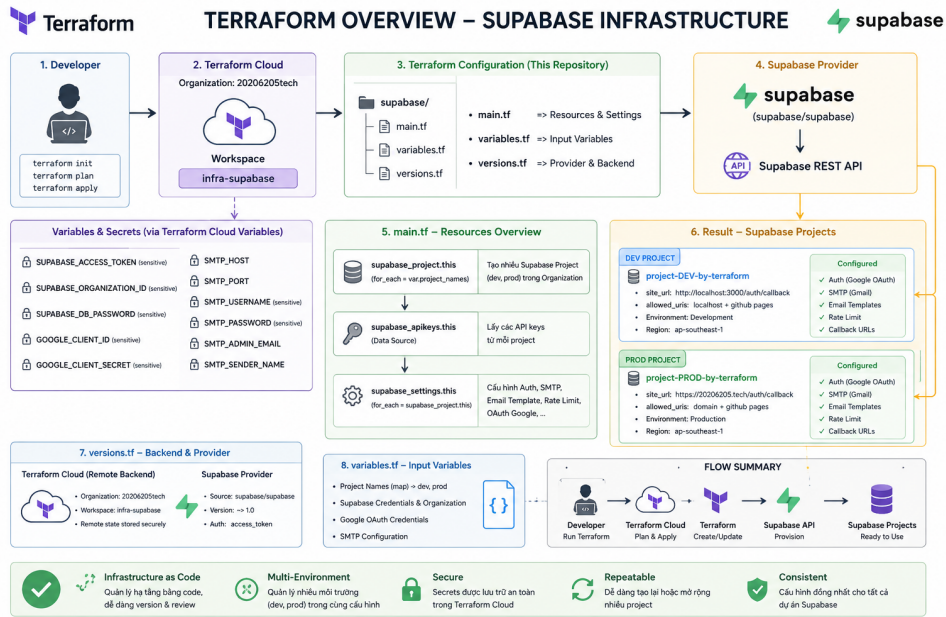
Quy trình kiểm tra khi đăng nhập (Challenge Flow)

1. Trích xuất danh sách các yếu tố MFA đã được người dùng đăng ký để lấy `factorId`.
2. Khi đã có `factorId`, ứng dụng tiếp tục gọi `challenge()` để mở phiên thử thách.
3. Dùng `verify()` cùng với mã OTP người dùng nhập vào để hoàn tất đăng nhập và nâng cấp phiên làm việc lên `aal2`.



Hình 3.8: Sơ đồ luồng xử lý xác thực MFA

Chi tiết về cấu hình Terraform dùng để quản lý hạ tầng Supabase:



Hình 3.9: Tổng quan cấu hình Terraform quản lý hạ tầng Supabase

Sử dụng Terraform Cloud với workspace `infra-supabase` để lưu trữ và quản lý trạng thái hạ tầng một cách tập trung.

Tự động tạo các dự án cho dev và prod dựa trên `project_names`.

Cấu hình mã vùng (region) là `ap-southeast-1` tại Singapore để tối ưu tốc độ kết nối cho người dùng tại Việt Nam.

Bỏ qua sự thay đổi của mật khẩu nhằm tránh việc Terraform yêu cầu cập nhật hoặc thay đổi mật khẩu cơ sở dữ liệu sau mỗi lần chạy cấu hình.

Tích hợp đăng nhập qua Google OAuth (Client ID và Client Secret).

Thiết lập giới hạn gửi email (rate limit).

Cấu hình Hệ thống Email (SMTP) đang để mặc định là Gmail.

Ghi đè các mẫu email HTML cho "Xác nhận tài khoản" và "Đặt lại mật khẩu" để đồng nhất email thương hiệu tương tự như trong dịch vụ thanh toán.

Xác nhận đăng ký tài khoản

Chào bạn,

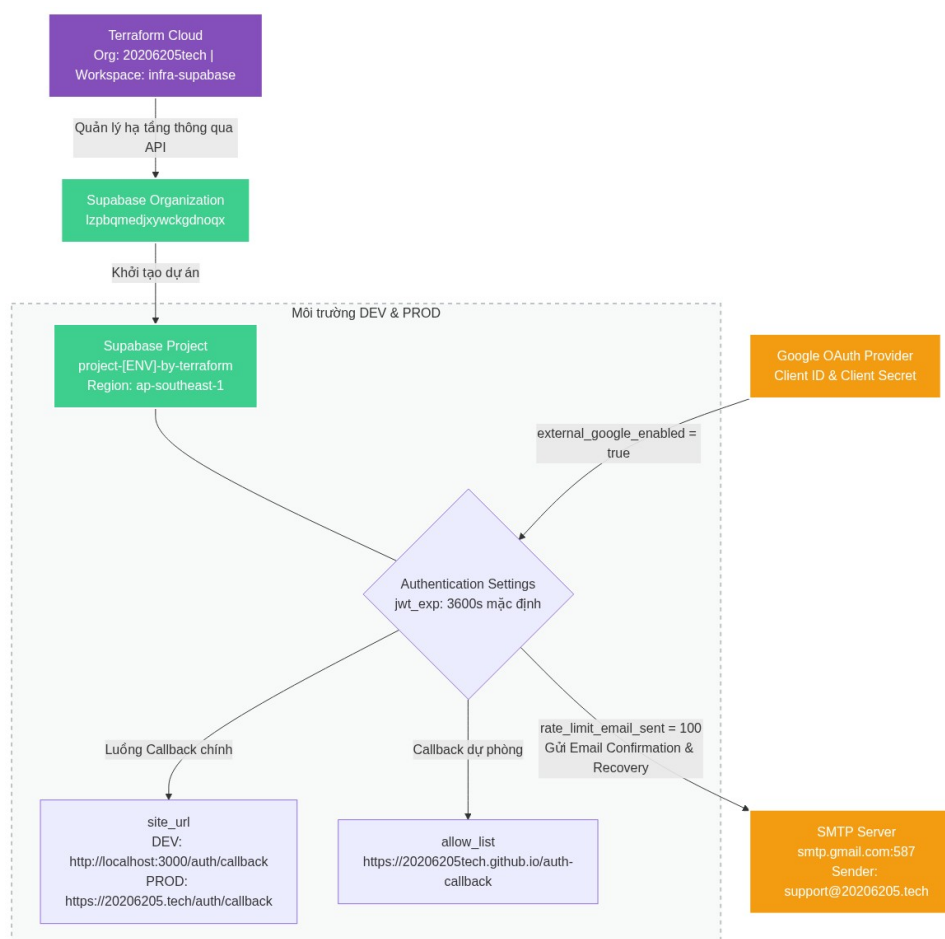
Cảm ơn bạn đã đăng ký tài khoản. Vui lòng nhấn vào nút bên dưới để xác nhận địa chỉ email của bạn:

[Xác nhận tài khoản](#)

Nếu nút bấm không hoạt động, vui lòng copy và dán đường dẫn sau vào trình duyệt:

https://cfxhnmuyutbgmrmdvnjs.supabase.co/auth/v1/verify?token=beb7299e0095ca978382210e09751769332ecc4b23a643f59c8a0b5d&type=signup&redirect_to=https://20206205.tech/auth/callback

Hình 3.10: Mẫu email HTML được cấu hình



Hình 3.11: Sơ đồ cấu hình Terraform cho Supabase

Chi tiết về cấu hình migration cơ sở dữ liệu Supabase:

Sử dụng Supabase CLI tạo dự án quản lý migration cơ sở dữ liệu bằng lệnh `npx supabase init`. Trong thư mục migrations, các file SQL được thiết kế kết hợp với chính sách Bảo mật cấp hàng RLS (Row-Level Security):

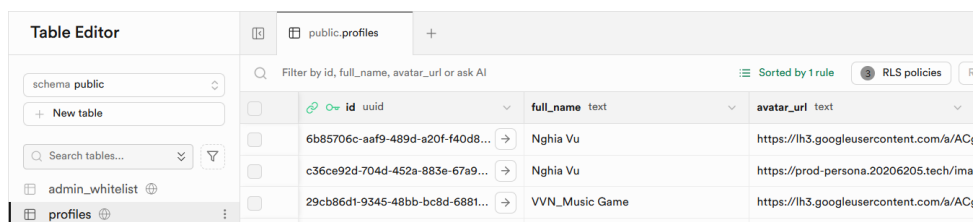
- Tạo bảng `admin_whitelist` lưu thông tin email của quản trị viên (ADMIN).
- Tạo bucket `avatars` lưu thông tin file ảnh đại diện người dùng.
- Tạo bảng `profiles` lưu thông tin của người dùng.
- Tạo bucket `persona_avatars` lưu thông tin file ảnh đại diện nhân vật.
- Tạo bucket `persona_audios` lưu thông tin file âm thanh lời chào nhân vật.

Kết quả bucket cấu hình trên Supabase:

NAME	POLICIES	FILE SIZE LIMIT	ALLOWED MIME TYPES
persona_audios PUBLIC	2	Unset (50 MB)	Any
persona_avatars PUBLIC	0	Unset (50 MB)	Any
avatars PUBLIC	4	Unset (50 MB)	Any

Hình 3.12: Cấu hình các Storage Bucket trên Supabase

Kết quả cơ sở dữ liệu trên Supabase:

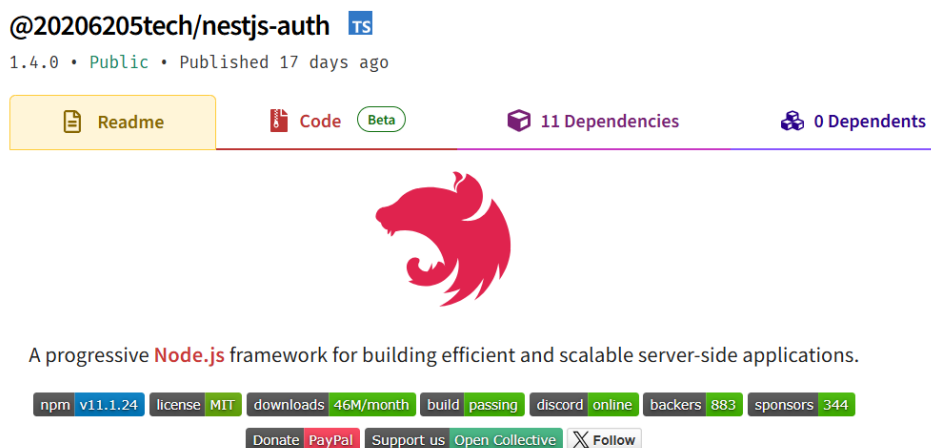


id	full_name	avatar_url
6b85706c-aaf9-489d-a20f-f40d8...	Nghia Vu	https://lh3.googleusercontent.com/a/ACQ...
c36ce92d-704d-452a-883e-67a9...	Nghia Vu	https://prod-persona.20206205.tech/ima...
29cb86d1-9345-48bb-bc8d-6881...	VVN_Music Game	https://lh3.googleusercontent.com/a/ACQ...

Hình 3.13: Cấu hình bảng cơ sở dữ liệu trên Supabase

Để tối ưu hóa quá trình phát triển, các gói thư viện dùng chung (Shared Packages) đã được xây dựng để các dịch vụ khác dễ dàng tái sử dụng và quản lý:

Hiện tại trong dự án, có 2 gói thư viện dùng chung được xây dựng và đăng ký trên npm/PyPI là NestJS (20206205tech-nestjs-auth) và FastAPI trong Python (20206205tech-python-auth). Các thư viện này thực hiện các bước kiểm duyệt bảo mật tự động bao gồm:



Hình 3.14: Quy trình kiểm tra bảo mật trong gói thư viện dùng chung

- Xác thực Request-Id được chuyển tiếp từ Kong API Gateway.
- Xác thực mã bảo mật Request-Kong-Secret để đảm bảo request thực sự đi qua API Gateway.
- Kiểm tra tính hợp lệ của chữ ký JWT của người dùng thông qua Supabase.
- Xác thực trạng thái MFA (yêu cầu bắt buộc để nâng cao bảo mật dữ liệu).
- Kiểm tra và phân quyền vai trò người dùng (ví dụ: yêu cầu vai trò quản trị viên ADMIN đối với các API quản lý).

3.3.5 Cấu hình trong Kong API Gateway

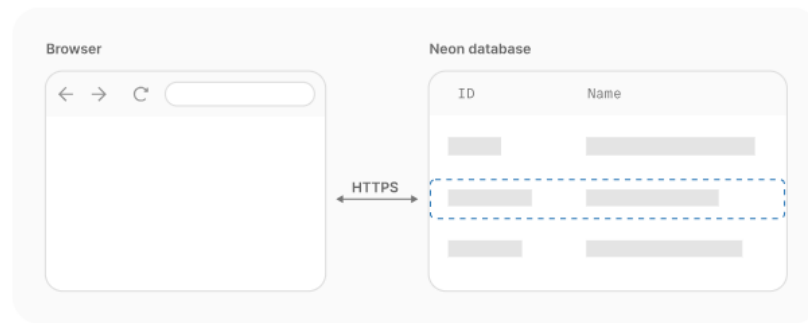
Tạo Service trong Kong trở về URL của dự án Supabase ([https://\[PROJECT_REF\].supabase.co](https://[PROJECT_REF].supabase.co)). Thiết lập Route: Tạo một Route liên kết với Service vừa tạo. Tham số `strip_path=true` đảm bảo Kong sẽ xóa tiền tố khỏi URI trước khi chuyển tiếp request đến Supabase.

Dùng plugin `request-transformer` để tự động thêm `apikey` vào header trước khi gửi đến Supabase.

3.4 Dịch vụ cài đặt (setting service - dùng Neon Data API)

Mục đích: sử dụng Neon Data API (REST API kết nối cơ sở dữ liệu Postgres) để người dùng có thể thực hiện các yêu cầu GET và POST nhằm lưu trữ và truy xuất các cấu hình cài đặt (dưới dạng key-value).

Kích hoạt thủ công Data API trên bảng điều khiển (Console) của Neon:



Data API

Instantly generate a REST API on top of your database. Query tables, views, and functions via standard HTTP requests, with zero backend code required. [Learn more](#)

☒ Use Neon Auth

Manages sign-up, login and account access for the Data API. It issues the JWTs required for API requests.

ⓘ Anyone on the web can sign up for your app. Support for restricted signups is coming soon.

☐ Grant public schema access

Applies grants ⓘ so authenticated users can read and write to tables in the public schema. Once enabled, add Row-Level Security (RLS) policies to define which rows they can access.

Enable Data API

Hình 3.15: Kích hoạt Data API trên Neon Console

Lấy thông tin URL của REST API trên giao diện web của Neon:

Data API

🏠 main

API Settings

API

Access your database through auto-generated REST API endpoints.

Security

To build apps that use the Data API, configure authentication for your users and [Row-Level Security in Postgres](#).

API URL

Provides read-write access to tables without Row Level Security enabled.

`https://ep-withered-band-aos89d8l.apirest.c-2.ap-southeast-1.aws.neon.tech/neondb/rest/v1` 🔗

To read and write data with the Data API, set up [authentication](#) and RLS policies. [Learn more](#)

Refresh schema cache

If you have made changes to the database schema, you can refresh the schema cache to apply them immediately.

Neon Auth is ready

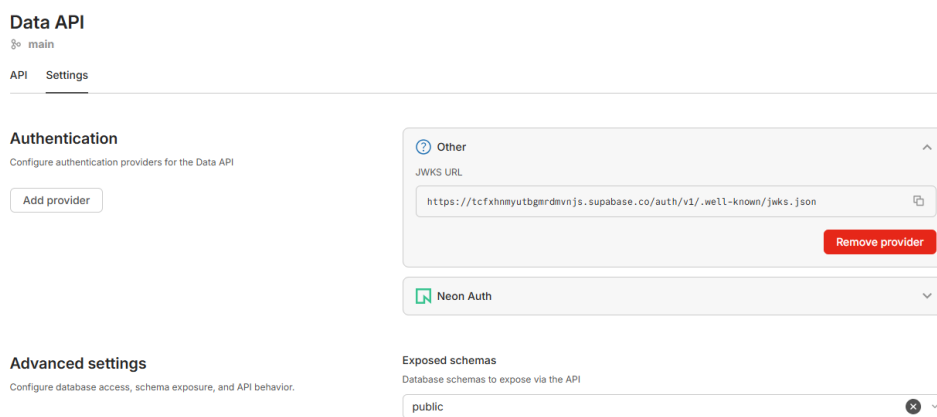
Go to the Neon Auth page for instructions on configuring authentication for your application. 🔗 Neon Auth

Data is protected

All of your tables have [Row-Level Security](#) enabled. Configure policies to control which rows users can access.

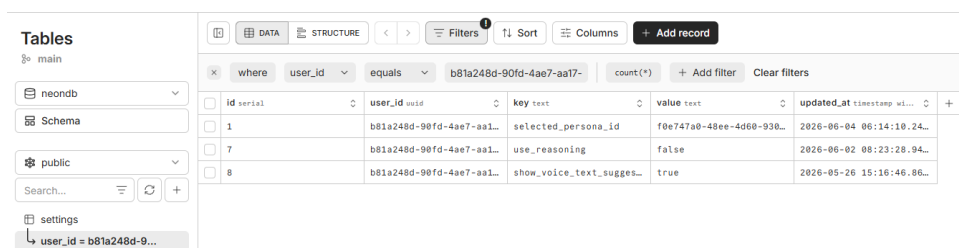
Hình 3.16: Thông tin URL REST API trên Neon Console

Neon hỗ trợ tích hợp cấu hình thông tin xác thực từ Supabase:



Hình 3.17: Tích hợp cấu hình xác thực Supabase trong Neon Console

Sử dụng SQL để tạo bảng **settings** gồm các trường: **id**, **user_id**, **key**, **value**, **updated_at**. Kích hoạt bảo mật cấp hàng RLS (Row-Level Security) để đảm bảo chỉ chủ sở hữu tài khoản mới có quyền thay đổi dữ liệu cài đặt của họ:



Hình 3.18: Bật RLS cho bảng settings trên Neon Console

3.4.1 Cấu hình trong Kong API Gateway

Tạo Service trong Kong trỏ trực tiếp đến endpoint REST API của bảng **settings** trên Neon. Thay vì tạo một Route chung cho tất cả các request, hệ thống được cấu hình để phân loại riêng biệt luồng đọc (GET) và luồng ghi (POST). Tham số **strip_path=true** giúp Kong loại bỏ tiền tố URI trước khi chuyển tiếp request đến Neon.

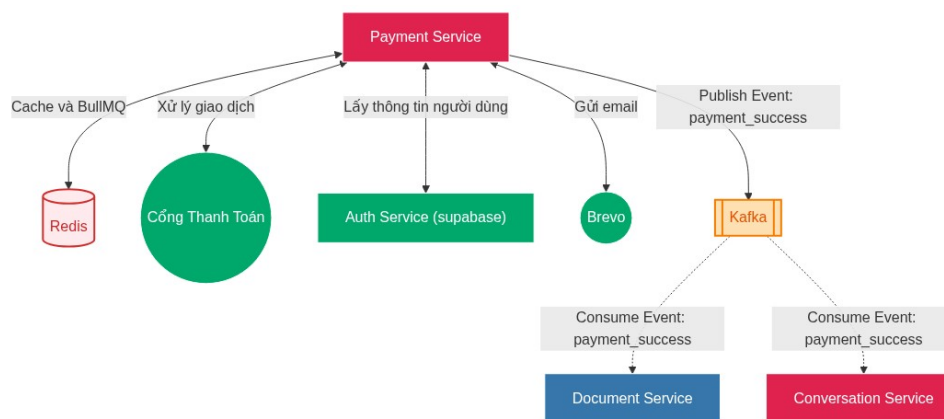
Cấu hình Plugin xử lý Upsert tự động:

Trong REST API dựa trên Postgres, để thực hiện hành động Upsert (chèn hoặc cập nhật), client cần gửi kèm các tham số cụ thể trên URL (**on_conflict=user_id,key**) và Header (**Prefer: resolution=merge-duplicates**).

Sử dụng plugin **request-transformer** để Kong API Gateway tự động đính kèm các tham số và header phức tạp này trước khi gửi đến Neon.

3.5 Dịch vụ thanh toán (payment service)

Sơ đồ tổng quan về Dịch vụ thanh toán (payment service):



Hình 3.19: Sơ đồ tích hợp thanh toán và sự kiện Kafka

Mục đích: Dịch vụ thanh toán (payment service) chịu trách nhiệm quản lý thông tin các gói dịch vụ (plans), quản lý thuê bao thành viên (subscriptions) của người dùng và tích hợp các cổng thanh toán.

3.5.1 I. Quản lý Gói dịch vụ (Plans)

Cho phép hệ thống định nghĩa và cung cấp các gói dịch vụ khác nhau tới người dùng.

- **Tạo mới gói dịch vụ:**
 - Cho phép Quản trị viên (ADMIN) thiết lập cấu hình gói dịch vụ mới bao gồm các thông số: tên gói (ví dụ: VIP 1 tháng), thời hạn sử dụng, giá tiền (VND), trạng thái hoạt động và các tính năng đi kèm (ví dụ: Quyền sử dụng mô hình suy luận AI, Voice, Xử lý tài liệu nâng cao).
- **Truy xuất danh sách gói:**
 - Lấy danh sách toàn bộ các gói dịch vụ hiện có trên hệ thống để hiển thị cho người dùng. Chức năng này được tối ưu hóa hiệu năng bằng cơ chế phân trang thông qua hai tham số skip và limit.
- **Cơ chế Cache với Redis:**
 - Lưu thông tin cache danh sách các plan vào Redis.
 - Khi người dùng yêu cầu xem danh sách gói đăng ký, hệ thống sẽ ưu tiên đọc từ Redis.
 - Nếu cache trống (cache miss), hệ thống sẽ truy xuất cơ sở dữ liệu, trả về kết quả cho client và lưu lại kết quả đó vào Redis cho các yêu cầu sau.
- **Lưu trữ/Vô hiệu hóa gói:**
 - Cho phép quản trị viên ẩn hoặc tạm dừng cung cấp các gói dịch vụ cũ.

3.5.2 II. Quản lý Thuê bao (Subscriptions) và Giao dịch

Quản lý thông tin đăng ký dịch vụ của người dùng và các giao dịch phát sinh:

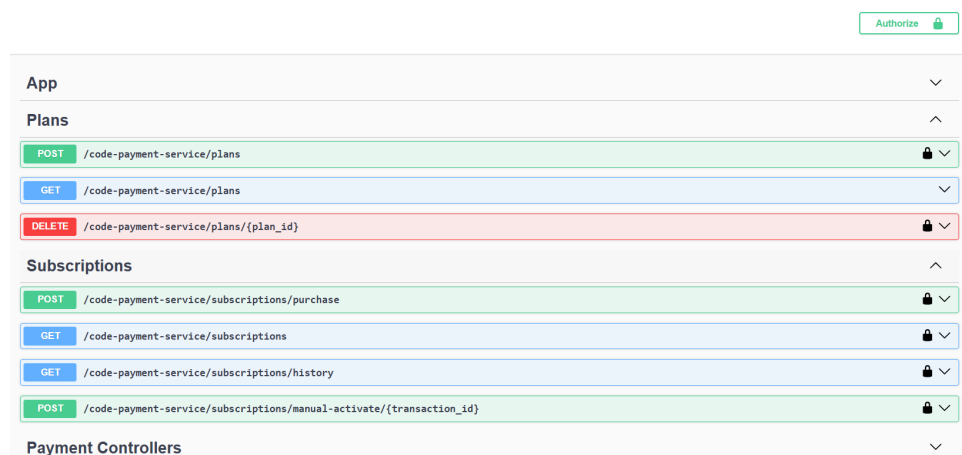
- **Mua gói:**
 - Người dùng khởi tạo yêu cầu thanh toán cho một gói dịch vụ cụ thể. Hệ thống tiếp nhận mã gói (plan_id) và đường dẫn chuyển hướng (redirect_url) để tạo link thanh toán và đưa người dùng tới cổng thanh toán.
 - Tích hợp chức năng tự động hủy giao dịch nếu hết thời gian chờ thanh toán (timeout) cấu hình trong Redis.
 - Gửi sự kiện thanh toán thành công lên Kafka.
 - Gửi email thông báo xác nhận thanh toán thành công cho người dùng.
- **Tra cứu thông tin Thuê bao:**

- Cho phép người dùng xem thông tin chi tiết và thời hạn của gói dịch vụ hiện tại mà họ đang sở hữu.
- **Tra cứu lịch sử giao dịch:**
 - Cung cấp danh sách lịch sử các giao dịch thanh toán trước đó của người dùng, hỗ trợ phân trang để dễ dàng theo dõi.
- **Kích hoạt giao dịch thủ công:**
 - Tính năng dành cho ADMIN để can thiệp kích hoạt thủ công một giao dịch (`transaction_id`) thành công, dùng trong các trường hợp đối soát thủ công hoặc xử lý sự cố.

3.5.3 III. Tích hợp Cổng thanh toán

- **Xử lý Callback (IPN):**
 - Cung cấp các endpoint (nhận cả phương thức `GET` và `POST`) để cổng thanh toán có thể âm thầm gọi về hệ thống (giao tiếp server-to-server) nhằm cập nhật trạng thái giao dịch một cách chính xác và bảo mật nhất.
- **Xử lý Return URL:**
 - Cung cấp các endpoint (nhận cả `GET` và `POST`) để tiếp nhận và điều hướng giao diện người dùng ngay sau khi họ hoàn tất thao tác thanh toán trên trang của đối tác.

Các API chính trong tài liệu Swagger API của Dịch vụ thanh toán (payment service):



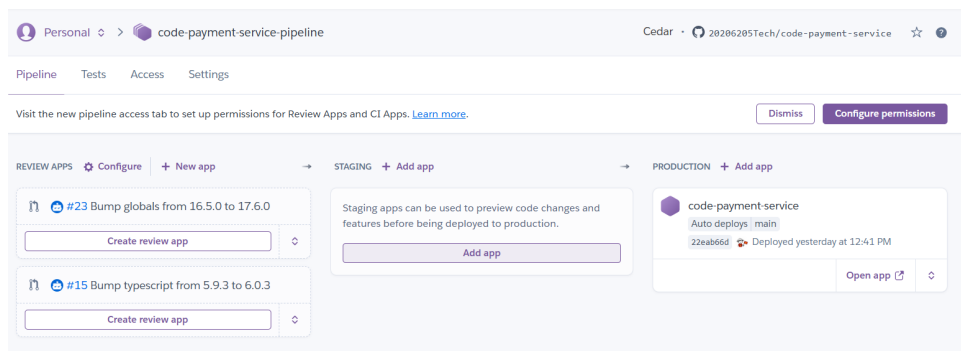
Hình 3.20: Tài liệu Swagger API Dịch vụ thanh toán

Trong quá trình phát triển dự án, do các dịch vụ thanh toán sẵn có chưa hỗ trợ tốt các cổng thanh toán phổ biến tại Việt Nam (như VNPay, ZaloPay, MoMo, SePay, ...), dịch vụ thanh toán này đã được tự thiết kế, phát triển và triển khai độc lập trên nền tảng Heroku.

Quy trình đóng gói và vận hành:

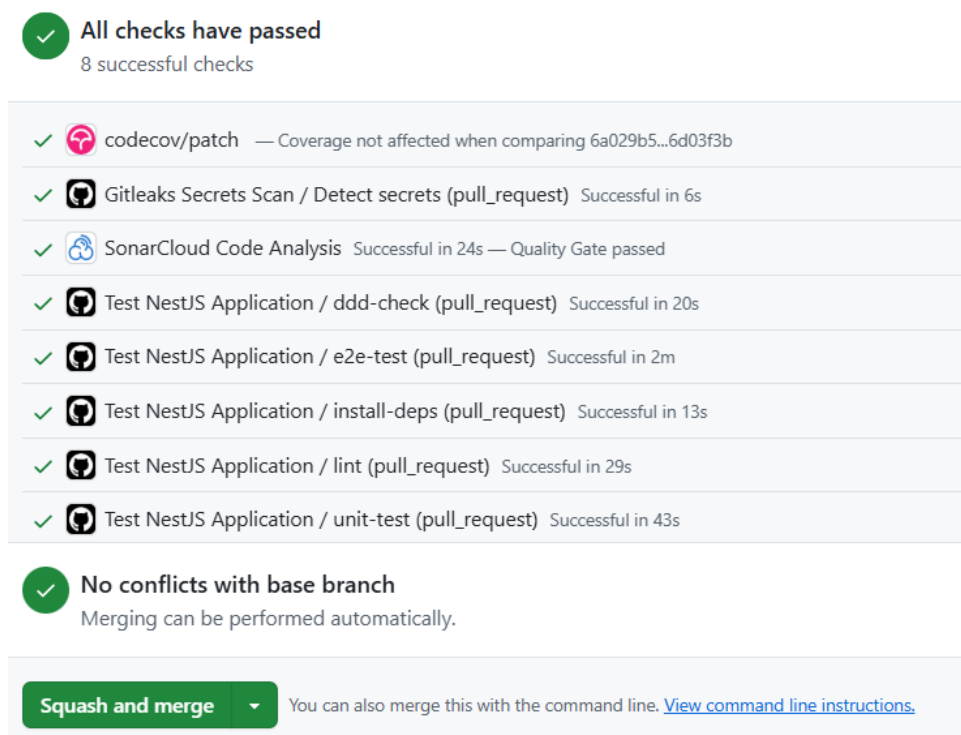
- Dịch vụ được đóng gói tự động bằng Docker.
- Tự động chạy và triển khai lên Heroku khi vượt qua các bước kiểm thử (tests) trong CI/CD của GitHub Actions.
- Kết nối trực tiếp với cơ sở dữ liệu Postgres (Neon), Kafka và Redis để vận hành hàng đợi BullMQ.

Triển khai Dịch vụ thanh toán (payment service) độc lập trên Heroku:



Hình 3.21: Trạng thái deploy dịch vụ thanh toán trên Heroku

Thông tin kiểm tra tự động của Pull Request trên GitHub:

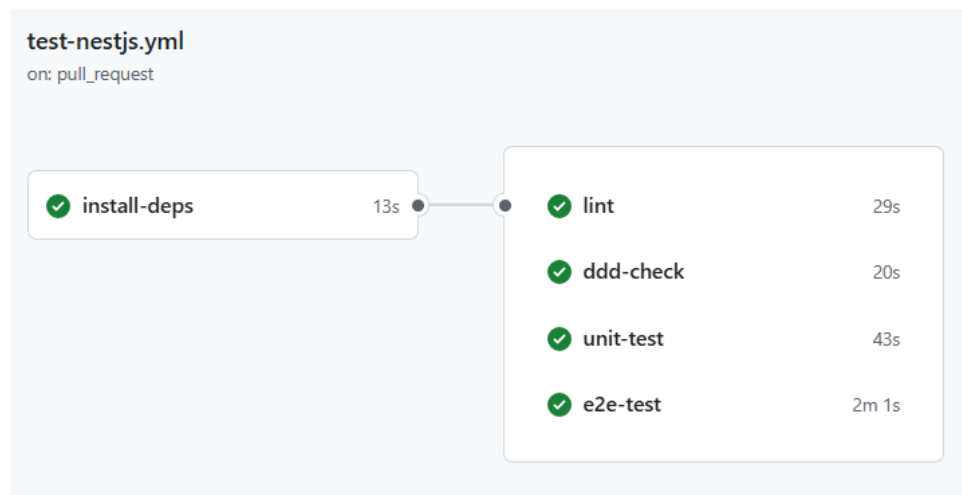


Hình 3.22: Kiểm tra Pull Request trên GitHub

Các bước CI/CD tự động chạy kiểm thử trên GitHub Actions:

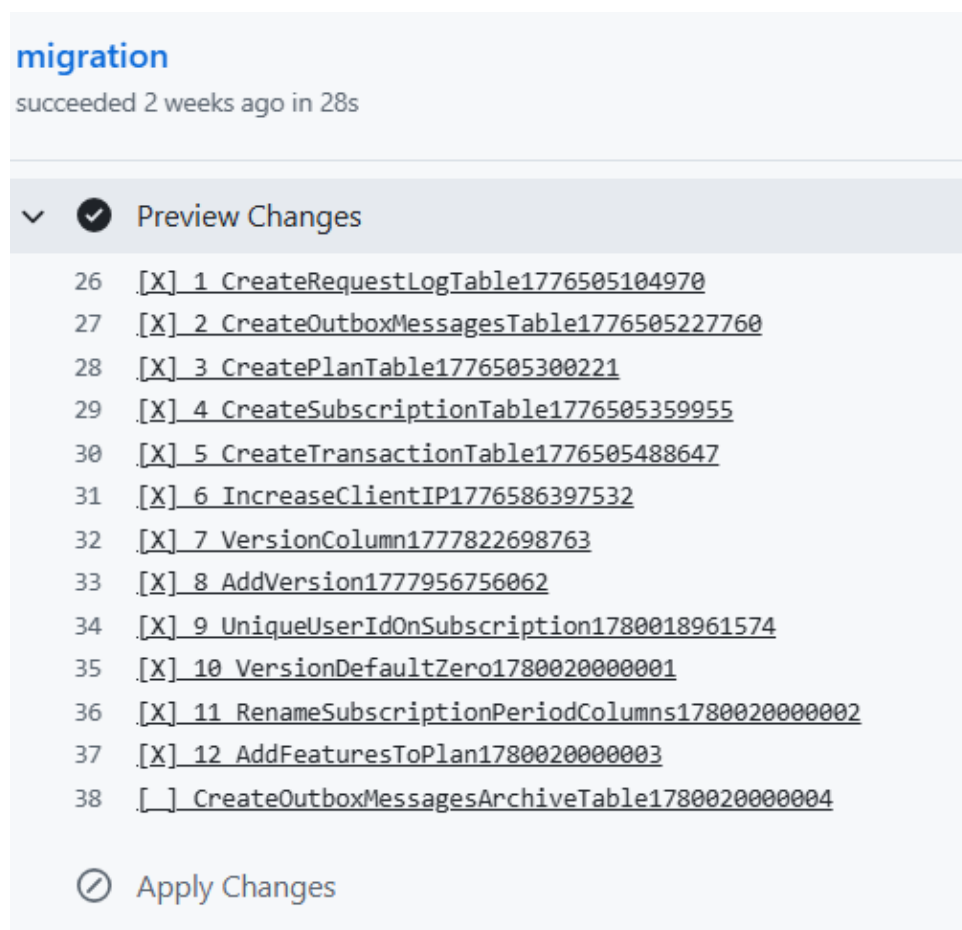
Hệ thống sử dụng **Testcontainers** (cho Postgres, Redis và Kafka) để tự động khởi chạy các container Docker độc lập ngay trong quá trình chạy integration tests, đảm bảo tính đúng đắn của mã nguồn trước khi deploy.

Hàng đợi tác vụ bất đồng bộ **BullMQ** (vận hành trên Redis) được tích hợp để quản lý và tự động xử lý hủy các giao dịch thanh toán quá hạn nếu hết thời gian chờ.



Hình 3.23: Quy trình chạy Test trong GitHub Actions

Quản lý migrations cơ sở dữ liệu tự động trong GitHub Actions:



Hình 3.24: Quản lý database migrations trên GitHub Actions

Các bảng trong cơ sở dữ liệu của Dịch vụ thanh toán (payment service):

Tables

go main

neondb

Schema

public

Search...

migrations

outbox_messages

outbox_messages_archive

plan

request_logs

subscription

transaction

DATA

STRUCTURE

< >

Filters

Sort

Columns

+ Add record

	id serial	timestamp bigint	name varchar	
☐	1	1776505104970	CreateRequestLogTable1...	
☐	2	1776505227760	CreateOutboxMessagesTa...	
☐	3	1776505300221	CreatePlanTable1776505...	
☐	4	1776505359955	CreateSubscriptionTabl...	
☐	5	1776505488647	CreateTransactionTable...	
☐	6	1776506397532	IncreaseClientIP177658...	
☐	7	1777822698763	VersionColumn17782269...	
☐	8	1777956756062	AddVersion1777956756062	
☐	9	1780018961574	UniqueUserIdOnSubscrip...	
☐	10	1780020000001	VersionDefaultZero1780...	
☐	11	1780020000002	RenameSubscriptionPeri...	
☐	12	1780020000003	AddFeaturesToPlan17800...	
☐	13	1780020000004	CreateOutboxMessagesAr...	

Hình 3.25: Cấu hình các bảng cơ sở dữ liệu của dịch vụ thanh toán

3.5.4 Sự kiện Thanh toán thành công (Subscription Purchased Event)

Khi giao dịch thanh toán gói dịch vụ (như gói VIP) thành công, Dịch vụ thanh toán sẽ tạo và phát hành (publish) sự kiện lên Kafka để đồng bộ quyền hạn người dùng.

1. Chi tiết sự kiện (Event Details)

- **Tên Sự kiện:** SubscriptionPurchasedEvent
- **Topic:**
 - dev-payment-events (môi trường phát triển - development)
 - prod-payment-events (môi trường vận hành - production)
- **Mã hóa (Compression):** GZIP
- **Phân vùng khóa (Message Key):** userId (đảm bảo tính tuần tự của các sự kiện liên quan đến cùng một người dùng trên cùng một partition).

2. Cấu trúc dữ liệu sự kiện (Event Payload Schema) Dữ liệu sự kiện được gửi dưới dạng chuỗi JSON có cấu trúc như sau:

Thuộc tính	Kiểu dữ liệu	Mô tả	Ví dụ
subscriptionId	String (UUID)	Mã định danh duy nhất của thuê bao mới tạo	"550e8400-e29b-41d4-a716-446655440000"
userId	String (UUID)	Mã định danh người dùng mua gói	"a1b2c3d4-e5f6-7a8b-9c0d-e1f2a3b4c5d6"
planId	String (UUID)	Mã định danh gói dịch vụ đã mua	"c9a64e1f-9c02-4c28-94ef-23e595a8947f"
periodStart	String (ISO 8601)	Thời điểm bắt đầu hiệu lực gói dịch vụ	"2026-06-15T02:16:31.000Z"
periodEnd	String (ISO 8601)	Thời điểm kết thúc hiệu lực gói dịch vụ	"2026-07-15T02:16:31.000Z"
version	Number	Số phiên bản của sự kiện (dùng để quản lý xung đột)	1
occurredAt	String (ISO 8601)	Thời điểm sự kiện phát sinh thực tế trên máy chủ	"2026-06-15T02:16:32.124Z"

Bảng 1: Cấu trúc dữ liệu sự kiện SubscriptionPurchasedEvent

Ví dụ JSON Payload:

```
{
  "subscriptionId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "a1b2c3d4-e5f6-7a8b-9c0d-e1f2a3b4c5d6",
  "planId": "c9a64e1f-9c02-4c28-94ef-23e595a8947f",
  "periodStart": "2026-06-15T02:16:31.000Z",
  "periodEnd": "2026-07-15T02:16:31.000Z",
  "version": 1,
  "occurredAt": "2026-06-15T02:16:32.124Z"
}
```

3. Cách các Service khác tiếp nhận và xử lý sự kiện Sự kiện này được tiêu thụ (consume) bởi hai dịch vụ: **Dịch vụ trò chuyện (conversation service)** và **Dịch vụ văn bản (document service)** để đồng bộ quyền lợi gói VIP của người dùng.

A. Dịch vụ văn bản (document service - Python)

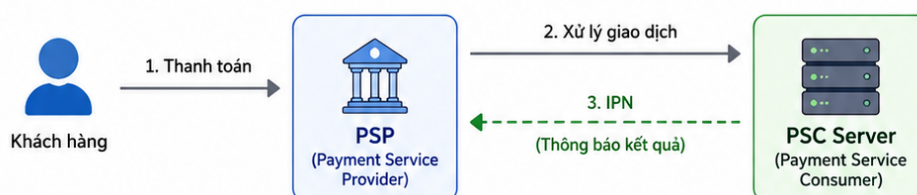
- **Tên Consumer Group:** document-service-group
- **Thư viện sử dụng:** aiokafka (Python asyncio-based Kafka client)
- **Luồng xử lý:**
 1. Consumer lắng nghe sự kiện từ topic, phân tích JSON payload sang kiểu dict.
 2. Hệ thống kiểm tra phiên bản (**Version Control**):
 - Lấy thông tin thuê bao hiện tại của người dùng trong cơ sở dữ liệu bảng `user_subscriptions`.
 - Nếu `version_nhận_được > version_hiện_tại`: Tiến hành cập nhật thông tin thuê bao (`period_start`, `period_end`, `subscription_id`, `version`) và đặt trạng thái `is_active = True`.
 - Nếu `version_nhận_được ≤ version_hiện_tại`: Bỏ qua (SKIP) sự kiện vì đã tồn tại bản cập nhật mới hơn trong DB (ngăn chặn lỗi out-of-order event).
 3. Người dùng chính thức được nâng quyền sử dụng dịch vụ bóc tách và tóm tắt văn bản dành riêng cho VIP.

B. Dịch vụ trò chuyện (conversation service - NestJS)

- **Tên Consumer Group:** conversation-service-group
- **Thư viện sử dụng:** kafkajs (kết nối mTLS)
- **Luồng xử lý:**
 1. Consumer lắng nghe và giải mã JSON payload nhận được.
 2. Khởi tạo một giao dịch cơ sở dữ liệu (**Database Transaction**) để thực hiện cập nhật bảng `UserSubscriptionEntity`.
 3. Kích hoạt khóa ghi bi quan (**Pessimistic Write Lock**) trên dòng dữ liệu của `userId` đó để ngăn chặn tranh chấp dữ liệu (Race Condition) trong trường hợp có nhiều sự kiện của cùng một user được xử lý song song.
 4. Thực hiện kiểm tra phiên bản (**Version Control**):
 - So sánh `version` của sự kiện nhận được với `version` hiện tại trong DB.
 - Nếu `version` nhận được cao hơn: Cập nhật thông tin thuê bao và lưu trạng thái kích hoạt `isActive = true`.
 - Nếu thấp hơn hoặc bằng: Bỏ qua (SKIP) tác vụ cập nhật.
 5. Mở khóa người dùng để kích hoạt tính năng trò chuyện thoại nâng cao và mô hình suy luận dành cho VIP.

3.5.5 Cơ chế Instant Payment Notification (IPN)

IPN (Instant Payment Notification) được sử dụng để Đơn vị cung cấp dịch vụ thanh toán - Payment Service Provider (PSP) thông báo kết quả giao dịch một cách không đồng bộ cho Đơn vị sử dụng dịch vụ - Payment Service Consumer (PSC).



Hình 3.26: Sơ đồ nguyên lý hoạt động của IPN

Sau khi khách hàng hoàn tất thanh toán, kết nối mạng của họ có thể bị gián đoạn hoặc họ có thể tắt trình duyệt trước khi được chuyển hướng về trang Web của hệ thống (Return URL). Cơ chế IPN đảm bảo rằng máy chủ của hệ thống vẫn nhận được kết quả cuối cùng một cách độc lập từ đối tác để cập nhật trạng thái đơn hàng một cách chính xác.



Hình 3.27: Mô phỏng luồng cập nhật trạng thái qua IPN

3.5.6 Thiết kế hệ thống theo Domain-Driven Design (DDD)

Dự án được thiết kế theo mô hình Phát triển hướng tên miền (Domain-Driven Design - DDD) kết hợp với kiến trúc Lục giác (Hexagonal Architecture).

Để demo khả năng thay đổi linh hoạt các dịch vụ tích hợp bên ngoài, hệ thống cho phép cấu hình cổng thanh toán mặc định thông qua biến môi trường `PAYMENT_DEFAULT_PROVIDER`. Nhờ áp dụng Dependency Inversion thông qua các Interface, việc chuyển đổi hoặc bổ sung các cổng thanh toán mới được thực hiện vô cùng dễ dàng mà không làm ảnh hưởng đến logic nghiệp vụ cốt lõi.

Giao diện demo thanh toán qua VNPAY:

Hình 3.28: Giao diện thanh toán VNPAY

Giao diện demo thanh toán qua MoMo:

Thông tin đơn hàng

Nhà cung cấp

 **MoMo Payment**

Mã đơn hàng

260613_DAAC56

Mô tả

Thanh toán đang ky: VI...

Số tiền

10.000đ

Đơn hàng sẽ hết hạn sau:

01

Giờ

39

Phút

42

Giây

Hướng dẫn thanh toán an toàn

Quay về

 **Nhập thông tin thẻ để thanh toán**



Vui lòng dùng thẻ quốc tế được phát hành bởi các Ngân hàng tại Việt Nam

Số thẻ

5200 0000 0000 1096

✓

Ngày hết hạn

05/26

✓

Tên chủ thẻ

NGUYEN VAN A

✓

CVC

111

✓

Số điện thoại

0397562283

✓

Thanh Toán

Hình 3.29: Giao diện thanh toán MoMo

Giao diện demo thanh toán qua ZaloPay:

Zalopay
GATEWAY



Thông tin đơn hàng

 **Merchant Demo V2**

Giá trị đơn hàng

₫10.000

Số tiền thanh toán

₫10.000

Mã giao dịch

260613_A82909

Nội dung

Thanh toán đang ky: VIP 1 #260613_A82909

 **Mã khuyến mãi**

Nhập mã khuyến mãi

Áp dụng

*Áp dụng khi quét QR bằng ứng dụng ngân hàng

Giao dịch kết thúc trong **19 : 50**

Quét QR để thanh toán



Ngân hàng Bản Việt

ZLPDEMO

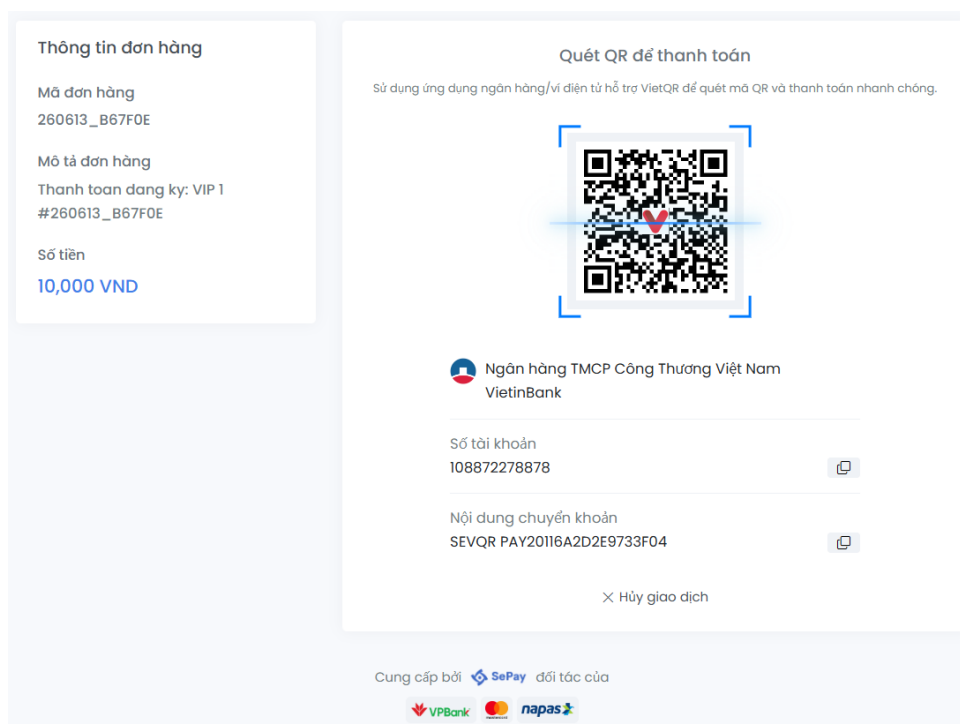
99ZP26164003276496

Mở ứng dụng có VietQR để thanh toán đơn hàng

Hình 3.30: Giao diện thanh toán ZaloPay

Giao diện demo thanh toán qua SePay:



Hình 3.31: Giao diện thanh toán SePay

Mẫu Email gửi cho người dùng khi thanh toán thành công:

Thanh toán thành công!

Chào **Nghia Vu**,

Cảm ơn bạn đã tin dùng dịch vụ của chúng tôi. Giao dịch mua gói **VIP 1** của bạn đã được xử lý thành công.

Mã giao dịch: 260613_BA5852

Tài khoản của bạn đã được cập nhật. Hãy truy cập ngay vào AI Chatbot để bắt đầu trải nghiệm nhé.

[Truy cập AI Chatbot](#)

Nếu nút bấm không hoạt động, vui lòng copy và dán đường dẫn sau vào trình duyệt:

<https://20206205.tech>

Trân trọng,
Đội ngũ AI Chatbot

Hình 3.32: Email thông báo thanh toán thành công

Dữ liệu giao dịch được lưu trữ và quản lý trong Redis:

The screenshot shows a Redis interface with a list of keys on the left and a detailed view of a specific key on the right.

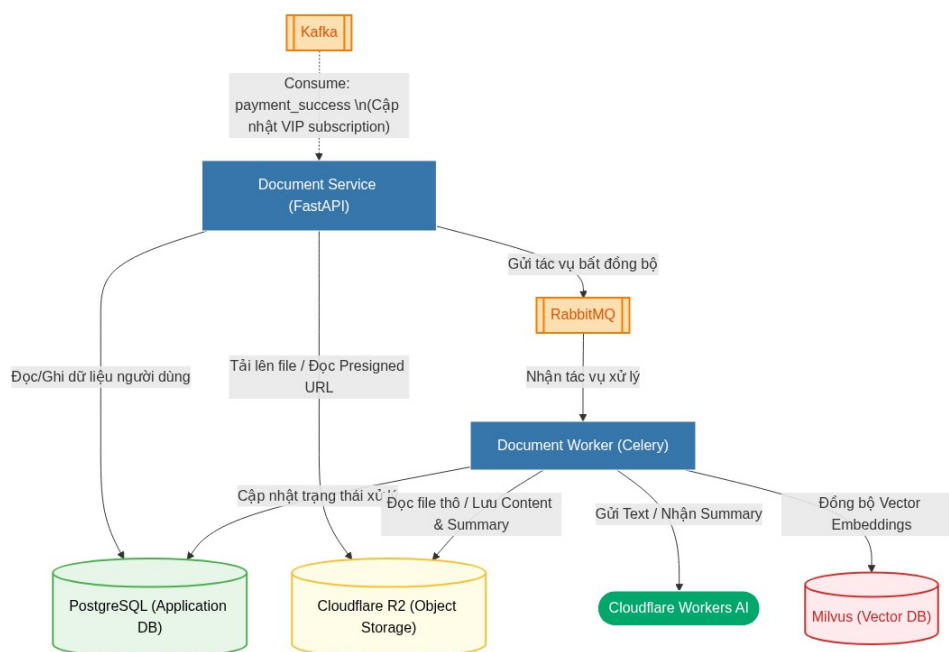
Key	Type	Value	Expires	Size
bull	String	64%	18	
development_payment	String	7%	2	
production_payment	String	7%	2	
plans	String	7%	2	
active	String	4%	1	
0	String	4%	1	
by-id	String	4%	1	
df82eb45-8517-4578-81ca-58078bdfb9c	String	29 d	320 B	

The detailed view of the key 'df82eb45-8517-4578-81ca-58078bdfb9c' shows a JSON object representing a payment plan with details like duration, price, and features.

Hình 3.33: Dữ liệu thanh toán lưu trữ trong Redis

3.6 Dịch vụ văn bản (document service)

Sơ đồ tổng quan về Dịch vụ văn bản (document service):



Hình 3.34: Sơ đồ tổng quan Dịch vụ văn bản

Mục đích: Tiếp nhận, lưu trữ và quản lý luồng xử lý tài liệu của người dùng. Dịch vụ này không chỉ lưu trữ tệp tin mà còn hỗ trợ quy trình trích xuất nội dung sang định dạng Markdown, tự động tạo bản tóm tắt và thực hiện vector hóa tài liệu.

Chào mừng đến với code-document-service (production)

- Đăng nhập Google
- Database
- Local
- Day
- doca
- redis
- voyager
- scalar

The screenshot shows the Swagger API interface for the 'code-document-service'. It lists several endpoints for document management.

Method	Endpoint	Description
POST	/documents/upload	Upload Document
GET	/documents/{doc_id}	Get Document Status
GET	/documents#	List User Documents
POST	/documents/{doc_id}/retry	Retry Document Processing
GET	/	Root

Hình 3.35: Tài liệu Swagger API Dịch vụ văn bản

Vì chức năng tải lên tài liệu chỉ dành riêng cho người dùng sở hữu gói VIP, dịch vụ văn bản (document service) thực hiện lắng nghe sự kiện thành toán thành công từ Kafka, lưu thông tin vào cơ sở dữ liệu để kiểm tra và xác thực quyền VIP của người dùng hiện tại.

- **Tải lên tài liệu mới:**
 - Người dùng gửi tệp tài liệu lên hệ thống. Sau khi tiếp nhận thành công, hệ thống sẽ trả về mã định danh duy nhất (`doc_id`), tên tệp gốc và trạng thái khởi tạo ban đầu, đồng thời đưa tác vụ vào hàng đợi xử lý.
- **Truy xuất trạng thái xử lý:**
 - Dựa vào `doc_id`, người dùng có thể truy vấn trạng thái xử lý hiện tại của tài liệu. Hệ thống sẽ báo cáo chi tiết về sự tồn tại của các thành phần dữ liệu thông qua các cờ trạng thái: `has_file`, `has_content`, `has_summary`.
- **Thử lại tác vụ lỗi:**
 - Trong trường hợp quá trình bóc tách hoặc tóm tắt gặp sự cố, hệ thống cung cấp một API chuyên biệt để kích hoạt lại quy trình xử lý cho tài liệu đó mà không cần người dùng phải tải lên lại tệp gốc.

3.6.1 Document Worker

Dịch vụ văn bản kết hợp với **Document Worker** (nhận các tác vụ nền từ Celery thông qua RabbitMQ) để thực hiện quy trình xử lý tài liệu phục vụ hệ thống RAG:

1. **Tải tệp tin:** Tải tệp tin từ Cloudflare R2 và sử dụng thư viện `Docling` để bóc tách cấu trúc và nội dung văn bản.
2. **Tóm tắt văn bản:** Sử dụng mô hình ngôn ngữ trên Cloudflare Workers AI để tự động tạo bản tóm tắt nội dung tài liệu bằng tiếng Việt.
3. **Chia nhỏ văn bản (Chunking):** Sử dụng bộ chia `RecursiveCharacterTextSplitter` của `LangChain` để chia nhỏ văn bản thành các đoạn phù hợp.
4. **Trích xuất đặc trưng (Embedding):** Tạo vector biểu diễn ngữ nghĩa cho từng đoạn văn bản bằng mô hình `keepitreal/vietnamese-sbert`.
5. **Lưu trữ Vector:** Quản lý và lưu trữ các vector embedding vào cơ sở dữ liệu vector `Milvus` để phục vụ tìm kiếm ngữ nghĩa sau này.

3.6.2 Cơ chế tối ưu & Độ bền bỉ (Optimization & Resilience)

- **Tái sử dụng kết quả (Deduplication):** Để tối ưu chi phí và hiệu năng, trước khi chạy parser và AI, worker kiểm tra xem người dùng đã từng tải lên tệp tin có cùng dung lượng (`file_size`) và băm (`file_hash`) hay chưa. Nếu đã có tài liệu trùng khớp đã xử lý thành công, hệ thống sao chép trực tiếp kết quả bóc tách và tóm tắt trên Cloudflare R2 sang đường dẫn của tài liệu mới mà không cần chạy lại mô hình AI.
- **Cấu hình hàng đợi Celery bền bỉ:**
 - `task_acks_late=True`: Task chỉ được xóa khỏi RabbitMQ khi worker đã thực sự hoàn thành, ngăn mất tác vụ nếu worker gặp sự cố giữa chừng.
 - `worker_prefetch_multiplier=1`: Ép worker chỉ nhận 1 tài liệu tại một thời điểm để cân bằng tải tốt hơn cho các tác vụ xử lý tệp nặng.
 - Tự động thử lại (retry) tối đa 3 lần với thời gian chờ 60 giây giữa mỗi lần nếu gặp sự cố kết nối ngoại vi (R2/Milvus/LLM timeout).
- **Giám sát Langfuse:** Toàn bộ luồng tóm tắt văn bản bằng AI (Cloudflare Workers AI) được theo dõi vết và giám sát chi tiết thông qua nền tảng **Langfuse** để kiểm soát chất lượng và độ trễ.

Giao diện quản lý vector trên Milvus:

Serving Clusters
+ -

- ▼ Free-01
 - 📁 prod_user_documents
 - 📁 dev_user_documents

Overview
Data
API Playground
Vector Search
Indexes
Partitions
Metrics
Actions ▼

📄 Import Data
📄 Sample Data

Data

Filter Ⓐ

Query
Reset

text Ⓐ	pk	Actions
CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập – Tự do – Hạnh phúc **GIẤY ỦY QUYỀN** Hôm ...	4662434980633	Q ↗
Họ và tên : : ----- ----- -----	4662434980633	Q ↗
Địa chỉ thường trú : :	4662434980633	Q ↗
NỘI DUNG ỦY QUYỀN: Bản ủy quyền đồng ý ủy quyền cho bà **Vũ Thị Thủy** thay mặt công ty li...	4662434980633	Q ↗

Hình 3.36: Dữ liệu vector lưu trữ trên Milvus

Quản lý các file tài liệu gốc trên Cloudflare R2:

[dev-documents](#) / a8dccb40-ab21-4ec2-becd-903d58863e8f / 

☐	Objects	Type
☐	 content/	Folder
☐	 document/	Folder
☐	 summary/	Folder

Hình 3.37: Tập tài liệu gốc lưu trữ trên Cloudflare R2

Quản lý nội dung tóm tắt lưu trữ trên Cloudflare R2:

[dev-documents](#) / [a8dccf40-ab21-4ec2-becd-903d58863e8f](#) / [summary](#) / 

☐ Objects	Type	Storage Class
☐ 06314f50-b3a7-4a7c-b5dd-f1ce5437e682	text/plain; charset=utf-8; ...	Standard
☐ 1176b5dd-8ab6-43bf-ade9-2d7772562d48	text/plain; charset=utf-8; ...	Standard
☐ 15151a83-2085-419a-8671-f434980e7fff	text/plain; charset=utf-8; ...	Standard
☐ 19adec44-4082-4b51-bbf5-f0375063362e	text/plain; charset=utf-8; ...	Standard
☐ 19ed0389-931f-4d7a-99ec-03d6c3a35152	text/plain; charset=utf-8; ...	Standard

Hình 3.38: Bản tóm tắt lưu trữ trên Cloudflare R2

Quản lý migration cơ sở dữ liệu của dịch vụ văn bản:

```

migration
succeeded 2 weeks ago in 24s

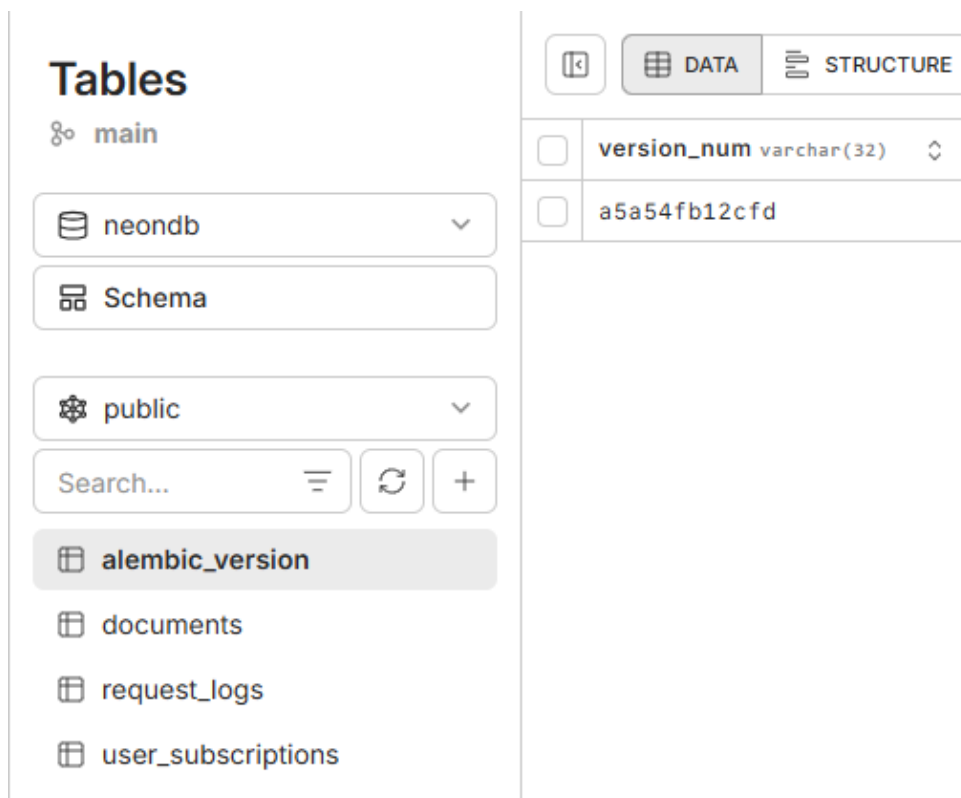
Preview Changes
33 * [scalar](http://localhost:52002/code-document-service/scalar)
34 INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
35 INFO [alembic.runtime.migration] Will assume transactional DDL.
36 f0f9ae3ead35
37 === Lịch sử Migration ===
38 f0f9ae3ead35 -> a5a54fb12cfd (head), rename dates and add version to user subscriptions
39 6d6b8e09b441 -> f0f9ae3ead35, add_file_hash_and_size
40 c73d83552e17 -> 6d6b8e09b441, create_user_subscriptions
41 4675d3821fd6 -> c73d83552e17, Boolean
42 97a9fddff7bc -> 4675d3821fd6, move_document_text_to_r2_urls
43 d4c0ecd18270 -> 97a9fddff7bc, create documents table
44 d08f3fc2b8e3 -> d4c0ecd18270, create request_logs
45 <base> -> d08f3fc2b8e3, Initial database
46 === Các thay đổi chưa áp dụng ===
47 2026-05-30 10:01:02.738 | INFO | env:<module>:7 - Loading environment variables...
48 2026-05-30 10:01:02.739 | SUCCESS | env:<module>:47 - DESCRIPTION:
49 # Chào mừng đến với code-document-service (production)
50
51 * [Đăng nhập Google](https://tcfxhnmvutbgmrdmwnjs.supabase.co/auth/v1/authorize?provider=google&redirect_to=
52 * [Database](https://console.neon.tech/app/org-still-feather-82034197/projects?q=code-document-service)
53 * [Local](http://localhost:52002)
54 * [Dev](https://dev-code-document-service.20206205.tech)
55 * [docs](http://localhost:52002/code-document-service/docs)
56 * [redoc](http://localhost:52002/code-document-service/redoc)
57 * [voyager](http://localhost:52002/code-document-service/voyager)
58 * [scalar](http://localhost:52002/code-document-service/scalar)
59 INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
60 INFO [alembic.runtime.migration] Will assume transactional DDL.
61 f0f9ae3ead35 -> a5a54fb12cfd (head), rename dates and add version to user subscriptions
62 6d6b8e09b441 -> f0f9ae3ead35, add_file_hash_and_size

Apply Changes

```

Hình 3.39: Quản lý migration cơ sở dữ liệu của dịch vụ văn bản

Sơ đồ cấu trúc các bảng trong cơ sở dữ liệu document:



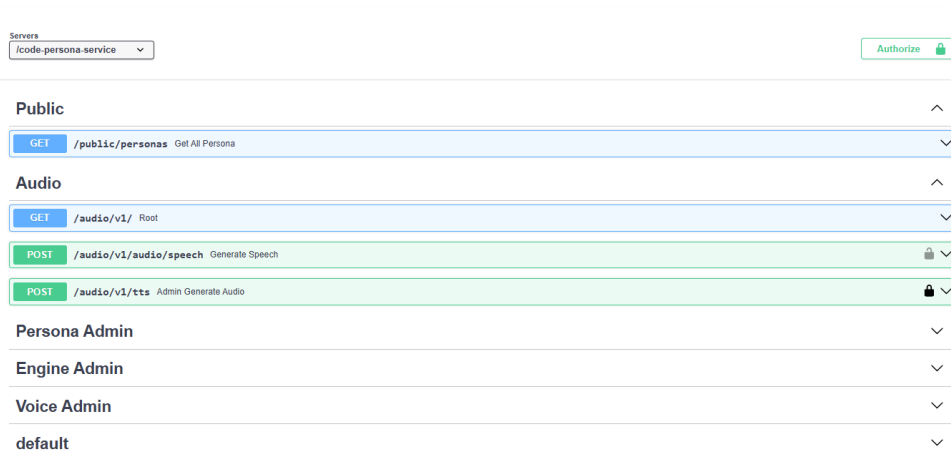
Hình 3.40: Cấu hình các bảng trong cơ sở dữ liệu document

3.7 Dịch vụ nhân vật (persona service)

Mục đích: Chịu trách nhiệm quản lý hệ thống nhân vật (personas), kho giọng đọc (Voices), các nền tảng tổng hợp giọng nói (TTS Engines) và xử lý trực tiếp các yêu cầu chuyển đổi văn bản thành âm thanh (Text-to-Speech).

Chào mừng đến với code-persona-service (production)

- Đăng nhập Google
- Database
- Local
- Dev
- docs
- redis
- voyager
- scalar



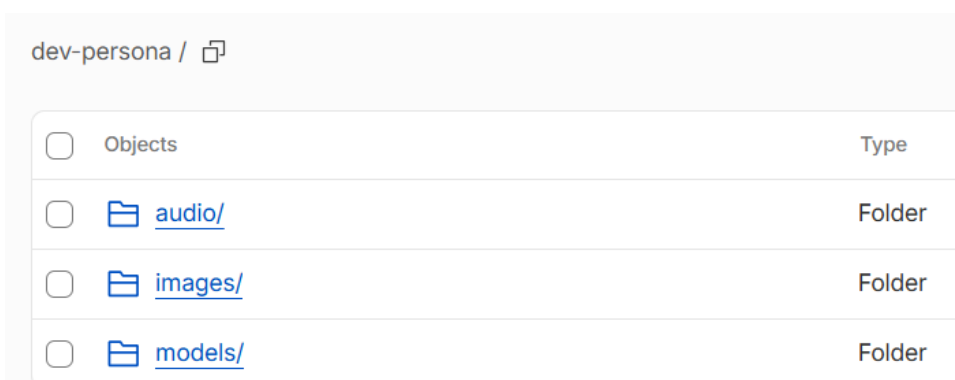
Hình 3.41: Tài liệu Swagger API Dịch vụ nhân vật

3.7.1 Các chức năng chính

- Truy xuất danh sách nhân vật:

- Cung cấp danh sách các nhân vật trong hệ thống, hỗ trợ phân trang và lọc theo `persona_id`.
- **Quản lý nhân vật (Dành cho ADMIN):**
 - Cho phép tạo mới, cập nhật thông tin hoặc xóa bỏ các nhân vật khỏi hệ thống.
 - Tải lên hình ảnh đại diện (**avatar**) và file âm thanh câu chào mẫu (**welcome audio**) cho từng nhân vật.
- **Quản lý công cụ TTS (engines):**
 - Cho phép quản trị viên ADMIN định nghĩa, cập nhật, xóa và truy xuất danh sách các nền tảng dịch vụ cung cấp TTS.
- **Quản lý giọng đọc (voices):**
 - Cho phép tạo, cập nhật, xóa và truy xuất danh sách các giọng đọc cụ thể, hỗ trợ bộ lọc theo công cụ/nền tảng.
- **Đồng bộ ElevenLabs:**
 - Tích hợp API đồng bộ danh sách giọng đọc mới nhất từ nhà cung cấp ElevenLabs trực tiếp về hệ thống cơ sở dữ liệu.
- **Chuyển đổi Text-to-Speech:**
 - Cung cấp chức năng tạo ra âm thanh từ văn bản đầu vào thông qua hai thư viện: **edge-tts** (tổng hợp online) và **piper-tts** (tổng hợp offline chất lượng cao).
 - Với **piper-tts**, hệ thống yêu cầu tệp mô hình giọng đọc được lưu trữ tại Cloudflare R2 và tự động tải xuống máy chủ cục bộ khi dịch vụ bắt đầu khởi động.
- **Cơ chế Cache tối ưu (TTS Caching):**
 - **Disk Cache:** Sử dụng thư viện **diskcache** (cơ chế LRU, kích thước giới hạn 1GB) để lưu giữ các file âm thanh đã tổng hợp lên ổ đĩa. Các yêu cầu TTS trùng lặp nội dung sẽ được phản hồi tức thì, tiết kiệm băng thông và CPU.
 - **Memory Cache:** Lưu giữ các đối tượng mô hình Piper đã load vào bộ nhớ RAM (`_loaded_voices`), tránh việc phải nạp lại tệp tin mô hình `.onnx` nặng sau mỗi request, tối ưu tốc độ phát âm thanh.
- **Lưu trữ tài nguyên:**
 - Toàn bộ dữ liệu mẫu âm thanh (**audio**), hình ảnh (**images**) và mô hình (**models**) phục vụ cho dịch vụ nhân vật được lưu trữ an toàn tại Cloudflare R2.

Quản lý thư mục tài nguyên của dịch vụ nhân vật trên Cloudflare R2:



Hình 3.42: Tài nguyên nhân vật lưu trữ trên Cloudflare R2

Quản lý migrations cơ sở dữ liệu của dịch vụ nhân vật:

```

migration
succeeded last week in 24s

▼ Preview Changes

28 * [Local](http://localhost:52003)
29 * [Dev](https://dev-code-persona-service.20206205.tech)
30 * [docs](http://localhost:52003/code-persona-service/docs)
31 * [redoc](http://localhost:52003/code-persona-service/redoc)
32 * [voyager](http://localhost:52003/code-persona-service/voyager)
33 * [scalar](http://localhost:52003/code-persona-service/scalar)
34 INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
35 INFO [alembic.runtime.migration] Will assume transactional DDL.
36 === Lịch sử Migration ===
37 8eaec96870ae -> b415223a43c9 (head), create engines, voices, persona table
38 d08f3fc2b8e3 -> 8eaec96870ae, create request_logs
39 <base> -> d08f3fc2b8e3, Initial database
40 === Các thay đổi chưa áp dụng ===
41 2026-06-03 19:01:16.246 | INFO | env:<module>:7 - Loading environment variables...
42 2026-06-03 19:01:16.246 | SUCCESS | env:<module>:47 - DESCRIPTION:
43 # Chào mừng đến với code-persona-service (production)
44
45 * [Đăng nhập Google](https://tcfxhnmvutbgmrdmwnjs.supabase.co/auth/v1/authorize?provider=google&redirect_to=
46 * [Database](https://console.neon.tech/app/org-still-feather-82034197/projects?q=code-persona-service)
47 * [Local](http://localhost:52003)
48 * [Dev](https://dev-code-persona-service.20206205.tech)
49 * [docs](http://localhost:52003/code-persona-service/docs)
50 * [redoc](http://localhost:52003/code-persona-service/redoc)
51 * [voyager](http://localhost:52003/code-persona-service/voyager)
52 * [scalar](http://localhost:52003/code-persona-service/scalar)
53 INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
54 INFO [alembic.runtime.migration] Will assume transactional DDL.
55 8eaec96870ae -> b415223a43c9 (head), create engines, voices, persona table
56 d08f3fc2b8e3 -> 8eaec96870ae, create request_logs
57 <base> -> d08f3fc2b8e3, Initial database

○ Apply Changes

```

Hình 3.43: Quản lý migration cơ sở dữ liệu của dịch vụ nhân vật

Cấu trúc các bảng trong cơ sở dữ liệu của Dịch vụ nhân vật:

Tables

main

neondb

Schema

public

Search...

alembic_version

engines

persona

request_logs

voices

DATA

STRUCTURE

<

>

Filters

Sort

Columns

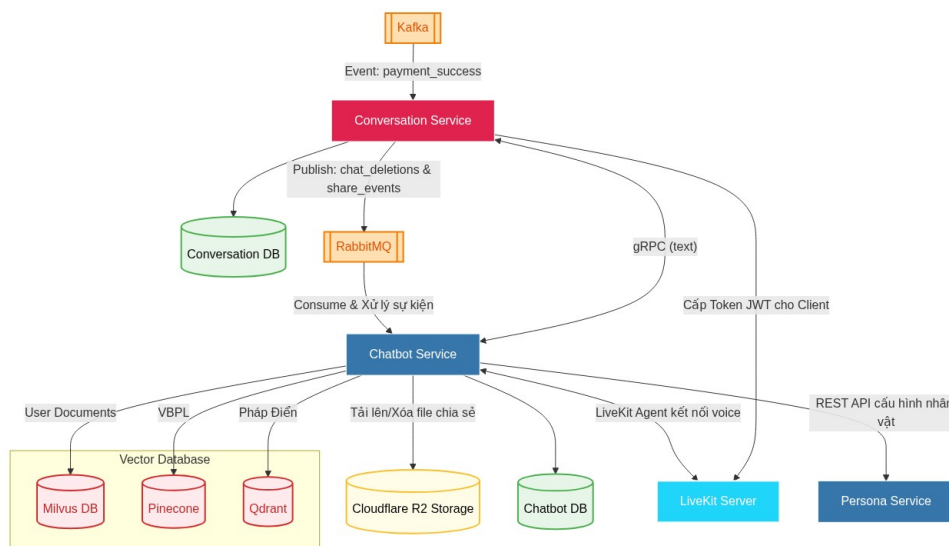
Add record

☐	id	uuid	code	varchar(50)	name	varchar(100)	is_active	boolean	voices
☐	a12e3456-789a-bcde-f01...		edge_tts		Edge TTS		TRUE		voices
☐	b12e3456-789a-bcde-f01...		elevenlabs		ElevenLabs		TRUE		voices
☐	e12e3456-789a-bcde-f01...		piper_tts		Piper TTS		TRUE		voices

Hình 3.44: Cấu trúc các bảng trong cơ sở dữ liệu của Dịch vụ nhân vật

3.8 Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service)

Sơ đồ tổng quan về Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service):



Hình 3.45: Sơ đồ tổng quan Dịch vụ trò chuyện và Dịch vụ chatbot

3.8.1 Dịch vụ trò chuyện (conversation service)

Vì chức năng trò chuyện văn bản suy luận (reasoning) và trò chuyện bằng giọng nói chỉ dành riêng cho tài khoản VIP, nên dịch vụ trò chuyện (conversation service) thực hiện lắng nghe sự kiện thanh toán thành công từ Kafka, lưu lại thông tin vào cơ sở dữ liệu và kiểm tra quyền VIP của người dùng hiện tại trước khi xử lý yêu cầu.

Chats			^
POST	/code-conversation-service/chats/start		🔒
POST	/code-conversation-service/chats/stream		🔒
DELETE	/code-conversation-service/chats/{chatId}		🔒
GET	/code-conversation-service/chats/{chat_id}/voice-token	Lấy token kết nối phiên hội thoại	🔒
Bookmarks			^
POST	/code-conversation-service/bookmarks		🔒
GET	/code-conversation-service/bookmarks		🔒
DELETE	/code-conversation-service/bookmarks/{folderId}		🔒
GET	/code-conversation-service/bookmarks/{folderId}		🔒
PUT	/code-conversation-service/bookmarks/{folderId}/item		🔒
DELETE	/code-conversation-service/bookmarks/{folderId}/item/{chatId}		🔒
Shared Chats			^
POST	/code-conversation-service/shared-chats		🔒
GET	/code-conversation-service/shared-chats		🔒
DELETE	/code-conversation-service/shared-chats/{shareId}		🔒
GET	/code-conversation-service/shared-chats/public/{shareId}/{token}		🔒

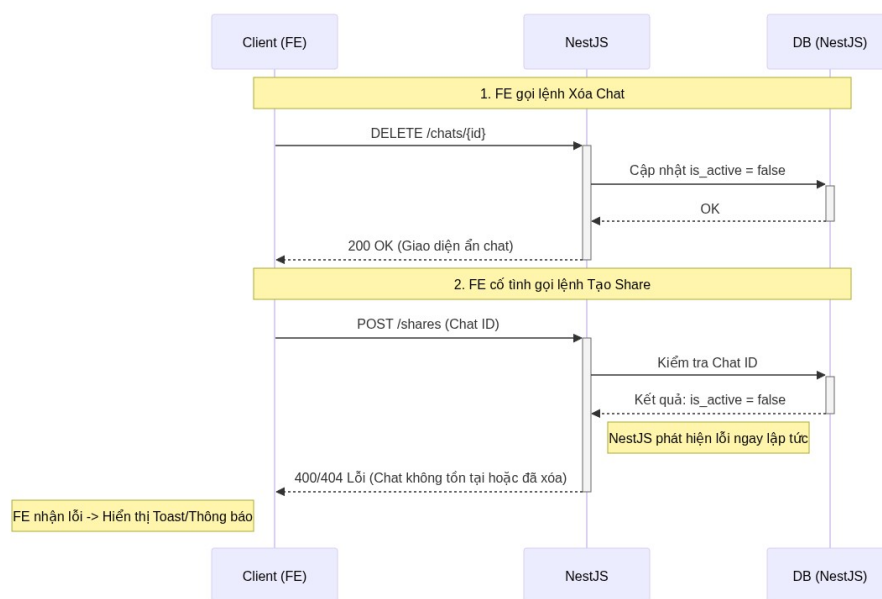
Hình 3.46: Tài liệu Swagger API Dịch vụ trò chuyện

Mục đích: Chịu trách nhiệm quản lý luồng giao tiếp giữa người dùng và hệ thống Agentic RAG. Dịch vụ này xử lý trực tiếp các cuộc trò chuyện văn bản theo cơ chế streaming, cấp phát token truy cập cho luồng giao tiếp giọng nói, và cung cấp các tiện ích lưu trữ, chia sẻ trò chuyện. Các tính năng chính:

- **Khởi tạo trò chuyện:** Bắt đầu một luồng trò chuyện mới trong hệ thống.
- **Cấp phát token thoại:** Cấp phát token bảo mật phục vụ kết nối đàm thoại bằng giọng nói với máy chủ LiveKit.
- **Định tuyến tin nhắn:** Tiếp nhận tin nhắn từ người dùng và kết nối với Dịch vụ chatbot (chatbot service) thông qua giao thức gRPC để xử lý thông tin.

- **Xóa cuộc trò chuyện:** Hỗ trợ xóa mềm (soft delete) một cuộc trò chuyện khỏi hệ thống.
- **Quản lý thư mục:** Cho phép người dùng tạo các thư mục lưu trữ để phân loại và sắp xếp các cuộc trò chuyện.
- **Đánh dấu và Ghi chú:** Đánh dấu một cuộc trò chuyện quan trọng và ghi chú các nội dung tóm tắt đi kèm.
- **Chia sẻ trò chuyện:** Tạo ra một bản snapshot chia sẻ đoạn trò chuyện với người dùng khác.
- **Đường dẫn chia sẻ:** Cung cấp URL công khai chứa tham số `shareId` và token bảo mật đi kèm. Chủ sở hữu đoạn chat có thể xem danh sách các liên kết đã tạo và thu hồi quyền truy cập bất cứ lúc nào.

Sơ đồ trình tự kiểm duyệt nghiệp vụ khi xóa cuộc trò chuyện và chặn đứng việc chia sẻ cuộc trò chuyện đã bị xóa:



Hình 3.47: Sơ đồ trình tự xóa cuộc trò chuyện và kiểm duyệt chia sẻ

Chi tiết luồng xử lý và kiểm duyệt (Validation Logic):

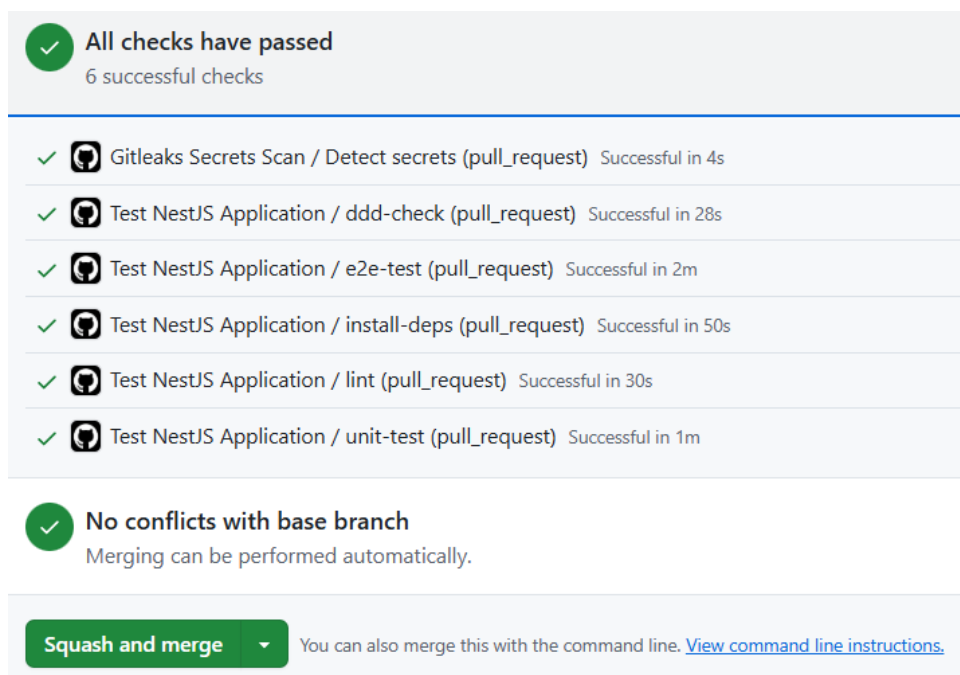
1. Xóa cuộc trò chuyện (Delete Chat):

- Client gửi yêu cầu `DELETE /chats/{id}` đến Dịch vụ trò chuyện (NestJS).
- Hệ thống tiến hành cập nhật trường trạng thái của cuộc trò chuyện trong Database thành `isActive = false` (thực hiện cơ chế xóa mềm - soft delete) và phản hồi `200 OK`. Lúc này, cuộc trò chuyện sẽ lập tức được ẩn đi trên giao diện người dùng.

2. Kiểm duyệt tạo liên kết chia sẻ (Validate Share Link):

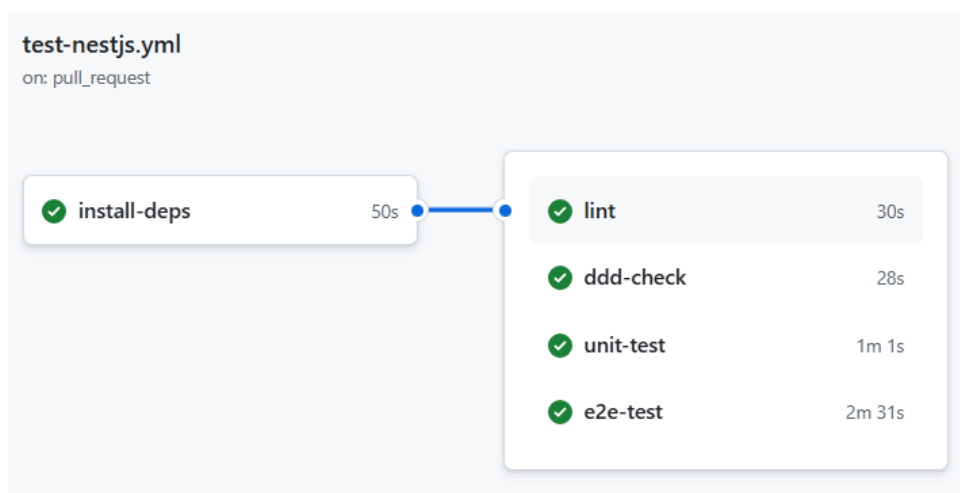
- Khi client gửi yêu cầu `POST /shares` (kèm theo `chat_id`) để tạo link chia sẻ công khai.
- Trước khi lưu bản ghi chia sẻ, `GenerateLinkShareCommandHandler` sẽ truy vấn thông tin cuộc trò chuyện từ Database dựa trên `chat_id`.
- Nếu cuộc trò chuyện không tồn tại hoặc có trạng thái `isActive = false` (đã bị xóa mềm), hệ thống sẽ chặn đứng hành động ngay lập tức và ném ra lỗi `ChatNotFoundException` (trả về mã lỗi `400/404` cho client kèm theo thông báo lỗi chi tiết trên giao diện UI).

Thông tin kiểm tra tự động Pull Request của dịch vụ trò chuyện trên GitHub:



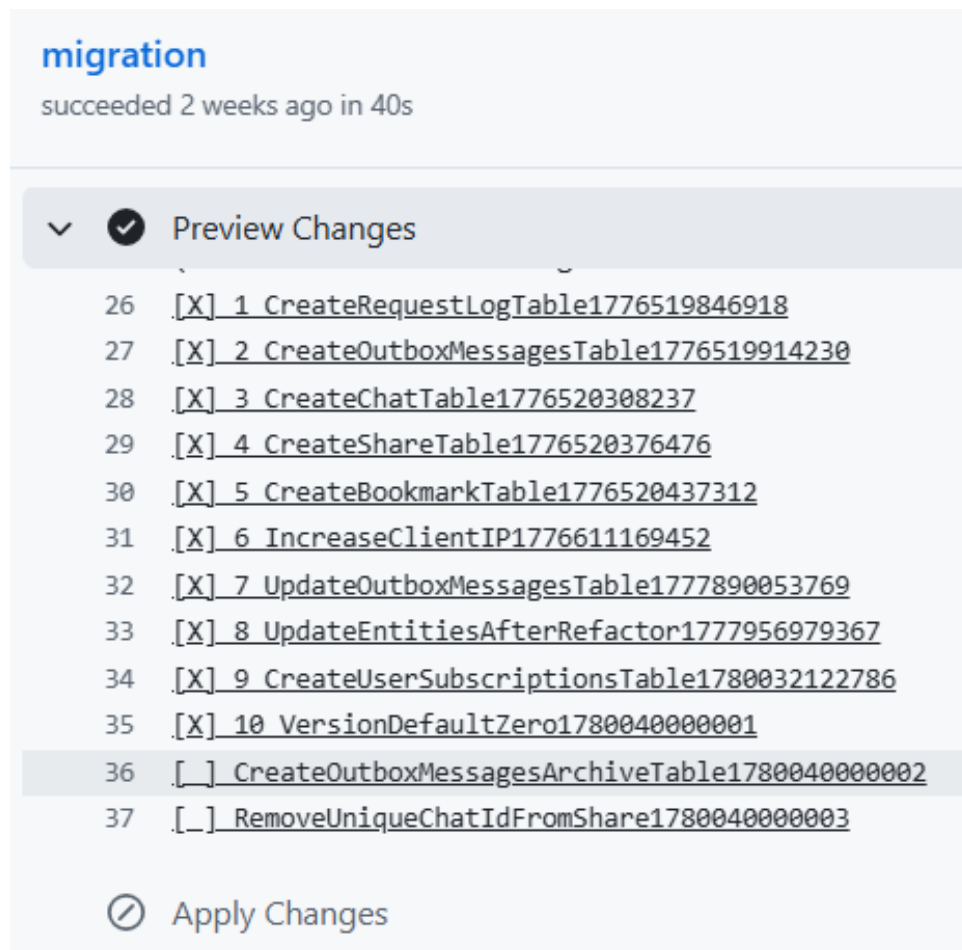
Hình 3.48: Kiểm tra Pull Request trên GitHub

Kiểm tra cú pháp nguồn (linting) tự động trong CI/CD:



Hình 3.49: Kiểm tra code linting dịch vụ trò chuyện

Quản lý migrations cơ sở dữ liệu của dịch vụ trò chuyện:



Hình 3.50: Quản lý database migrations dịch vụ trò chuyện

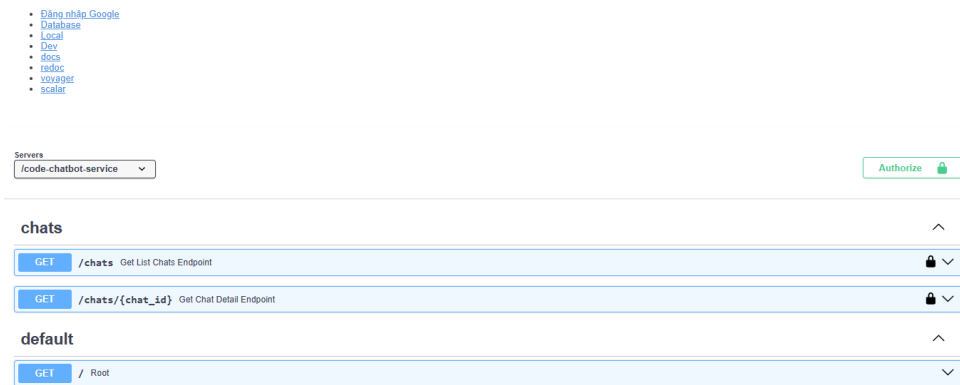
Cơ cấu các bảng trong cơ sở dữ liệu của Dịch vụ trò chuyện:

Tables	DATA	STRUCTURE	Filters	Sort	Columns	Add record
main						
neondb						
Schema						
public						
bookmark_folder						
bookmark_item						
chat						
migrations						
outbox_messages						
outbox_messages_archive						
request_logs						
shared_chat						
user_subscriptions						

Hình 3.51: Cấu hình các bảng trong cơ sở dữ liệu conversation

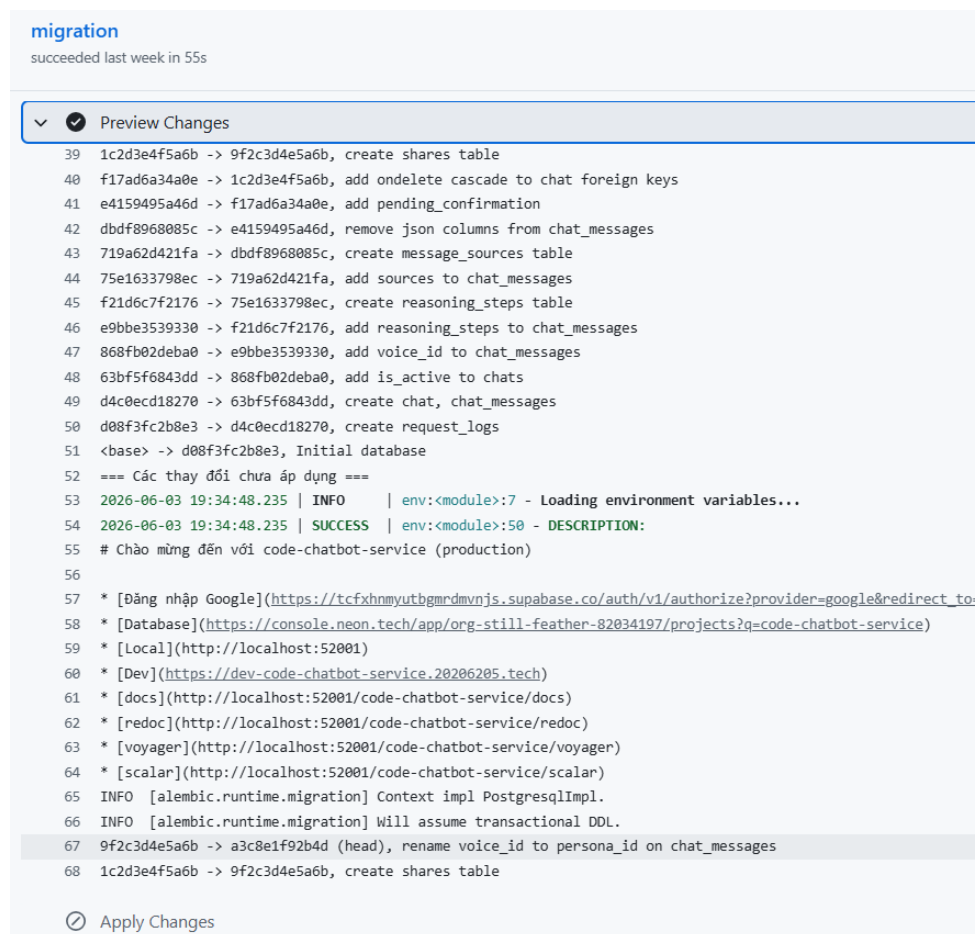
3.8.2 Dịch vụ chatbot (chatbot service)

Chào mừng đến với code-chatbot-service (production)



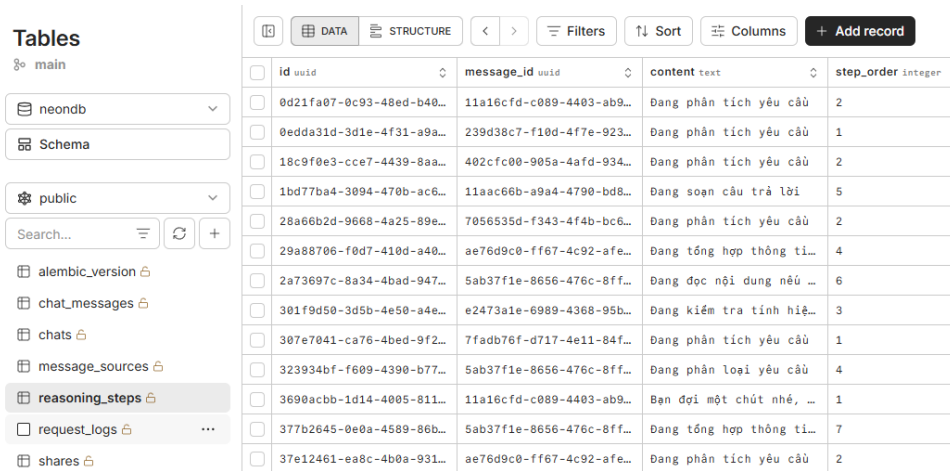
Hình 3.52: Tài liệu Swagger API Dịch vụ chatbot

Mục đích: Sử dụng các mô hình AI để xử lý và phản hồi thông minh các câu hỏi của người dùng. Sau đó, lưu lại lịch sử trò chuyện và cung cấp các API truy xuất lịch sử, thông tin suy luận chi tiết (reasoning details) và nguồn tài liệu tham khảo (references) của từng câu trả lời. Dịch vụ chatbot lắng nghe RabbitMQ để thực hiện việc tạo và thu hồi các bản snapshot chia sẻ trò chuyện, đồng thời kết nối với LiveKit để vận hành đàm thoại giọng nói với người dùng. Quản lý migrations cơ sở dữ liệu của dịch vụ chatbot:



Hình 3.53: Quản lý database migrations dịch vụ chatbot

Cơ cấu các bảng trong cơ sở dữ liệu của Dịch vụ chatbot:



id	message_id	content	step_order
0d21fa07-0c93-48ed-b40...	11a16cfd-c089-4403-ab9...	Đang phân tích yêu cầu	2
0edda31d-3d1e-4f31-a9a...	239d38c7-f10d-4f7e-923...	Đang phân tích yêu cầu	1
18c9f0e3-cce7-4439-8aa...	402cfc00-905a-4afd-934...	Đang phân tích yêu cầu	2
1bd77ba4-3094-470b-ac6...	11aac66b-a9a4-4790-bd8...	Đang soạn câu trả lời	5
28a66b2d-9668-4a25-89e...	7056535d-f343-4f4b-bc6...	Đang phân tích yêu cầu	2
29a88706-f0d7-410d-a40...	ae76d9c0-ff67-4c92-afe...	Đang tổng hợp thông tin...	4
2a73697c-8a34-4bad-947...	5ab37f1e-8656-476c-8ff...	Đang đọc nội dung nếu ...	6
301f9d50-3d5b-4e50-a4e...	e2473a1e-6989-4368-95b...	Đang kiểm tra tính hiệ...	3
307e7041-ca76-4bed-9f2...	7fad76f-d717-4e11-84f...	Đang phân tích yêu cầu	1
323934bf-f609-4390-b77...	5ab37f1e-8656-476c-8ff...	Đang phân loại yêu cầu	4
3690acbb-1d14-4005-811...	11a16cfd-c089-4403-ab9...	Bạn đợi một chút nhé, ...	1
377b2645-0e0a-4589-86b...	5ab37f1e-8656-476c-8ff...	Đang tổng hợp thông tin...	7
37e12461-ea8c-4b0a-931...	ae76d9c0-ff67-4c92-afe...	Đang phân tích yêu cầu	2

Hình 3.54: Cấu hình các bảng trong cơ sở dữ liệu chatbot

3.8.3 Hệ thống trao đổi thông điệp qua RabbitMQ (RabbitMQ Messaging)

Hệ thống sử dụng **RabbitMQ** làm Message Broker để thực hiện các giao tiếp bất đồng bộ, đáng tin cậy giữa **Dịch vụ trò chuyện (NestJS)** và **Dịch vụ chatbot (Python)**. Các tên hàng đợi được tiền tố hóa tự động theo môi trường (dev_, test_, prod_) dựa trên cấu hình hệ thống.

1. Danh sách các Hàng đợi (Queues) trong hệ thống

- [prefix]_chat_deletions:
 - **Mục đích:** Nhận yêu cầu xóa mềm (soft delete) lịch sử chat từ phía người dùng.
 - **Durable:** true
- [prefix]_share_generated:
 - **Mục đích:** Yêu cầu tạo trang chia sẻ trò chuyện công khai (đóng gói dạng JSONL và tải lên Cloudflare R2).
 - **Durable:** true
 - **DLQ Routing:** [prefix]_share_generated_dlq
- [prefix]_share_generated_retry:
 - **Mục đích:** Hàng đợi trung gian để xử lý lại (retry) khi gặp lỗi lệch thứ tự sự kiện (out of order). Cấu hình độ trễ cố định **x-message-ttl = 20s** trước khi gửi trả lại queue chính.
- [prefix]_share_generated_dlq:
 - **Mục đích:** Dead Letter Queue nhận thông điệp tạo share thất bại hoàn toàn sau **MAX_RETRY_COUNT = 5** lần thử lại.
- [prefix]_share_upload_completed:
 - **Mục đích:** Báo cáo kết quả đóng gói và upload file chia sẻ từ Python về lại NestJS.
 - **Durable:** true
 - **DLQ Routing:** [prefix]_share_upload_completed_dlq
- [prefix]_share_upload_completed_dlq:
 - **Mục đích:** Dead Letter Queue nhận thông điệp báo cáo kết quả upload bị xử lý thất bại ở NestJS.
- [prefix]_share_revoked:
 - **Mục đích:** Nhận yêu cầu thu hồi link chia sẻ, tiến hành xóa file tương ứng trên Cloudflare R2 và cập nhật DB.
 - **Durable:** true

2. Cấu trúc dữ liệu và Luồng xử lý sự kiện (Event Payloads & Flow) A. Sự kiện Xóa trò chuyện (Chat Deletion Event)

- Queue: [prefix]_chat_deletions
- Publisher: code-conversation-service (NestJS)
- Consumer: code-chatbot-service (Python)
- Payload Schema (JSON):

```
{
  "chat_id": "string", // Mã ID cuộc trò chuyện cần xóa
  "user_id": "string", // Mã ID người dùng sở hữu cuộc trò chuyện
  "version": 0 // Phiên bản sự kiện để kiểm tra thứ tự
}
```

B. Sự kiện Yêu cầu chia sẻ (Share Generated Event)

- Queue: [prefix]_share_generated
 - Publisher: code-conversation-service (NestJS)
 - Consumer: code-chatbot-service (Python)
 - Payload Schema (JSON):
- ```
{
 "share_id": "string", // ID duy nhất của liên kết chia sẻ
 "chat_id": "string", // ID cuộc trò chuyện được chia sẻ
 "user_id": "string", // ID người dùng tạo chia sẻ
 "version": 1, // Phiên bản sự kiện (khởi tạo thường là 1)
 "last_message_id": "string" // ID tin nhắn cuối cùng (hoặc null nếu chia sẻ hết)
}
```
- Cơ chế xử lý lệch thứ tự (Out-of-Order): Consumer Python so sánh phiên bản sự kiện nhận được với phiên bản hiện tại trong DB:
    - Nếu event\_version == current\_version + 1: Xử lý đóng gói lịch sử chat, upload R2 và gửi kết quả về NestJS.
    - Nếu event\_version <= current\_version: Bỏ qua (đã xử lý trước đó).
    - Nếu event\_version > current\_version + 1: Sự kiện đến sai thứ tự, ném ra ngoại lệ OutOfOrderException để đưa tin nhắn vào [prefix]\_share\_generated\_retry tạm chờ 20 giây trước khi thử lại. Sau 5 lần thất bại liên tục sẽ bị chuyển vào DLQ và gửi cảnh báo về hệ thống.

## C. Sự kiện Thu hồi liên kết (Share Revoked Event)

- Queue: [prefix]\_share\_revoked
- Publisher: code-conversation-service (NestJS)
- Consumer: code-chatbot-service (Python)
- Payload Schema (JSON):

```
{
 "share_id": "string", // ID liên kết chia sẻ cần thu hồi
 "file_url": "string", // URL tệp tin JSONL cần xóa trên Cloudflare R2
 "version": 2 // Phiên bản sự kiện (thường là phiên bản kế tiếp)
}
```

- Cơ chế xử lý: Xóa tệp tin trên Cloudflare R2, đánh dấu is\_revoked = True trong DB. Hỗ trợ cơ chế tự động gửi lại hàng đợi chính (requeue) nếu phát hiện sự kiện đến lệch thứ tự (revoke đến trước khi generate ip ghi DB).

#### D. Sự kiện Hoàn tất Upload (Share Upload Completed Event)

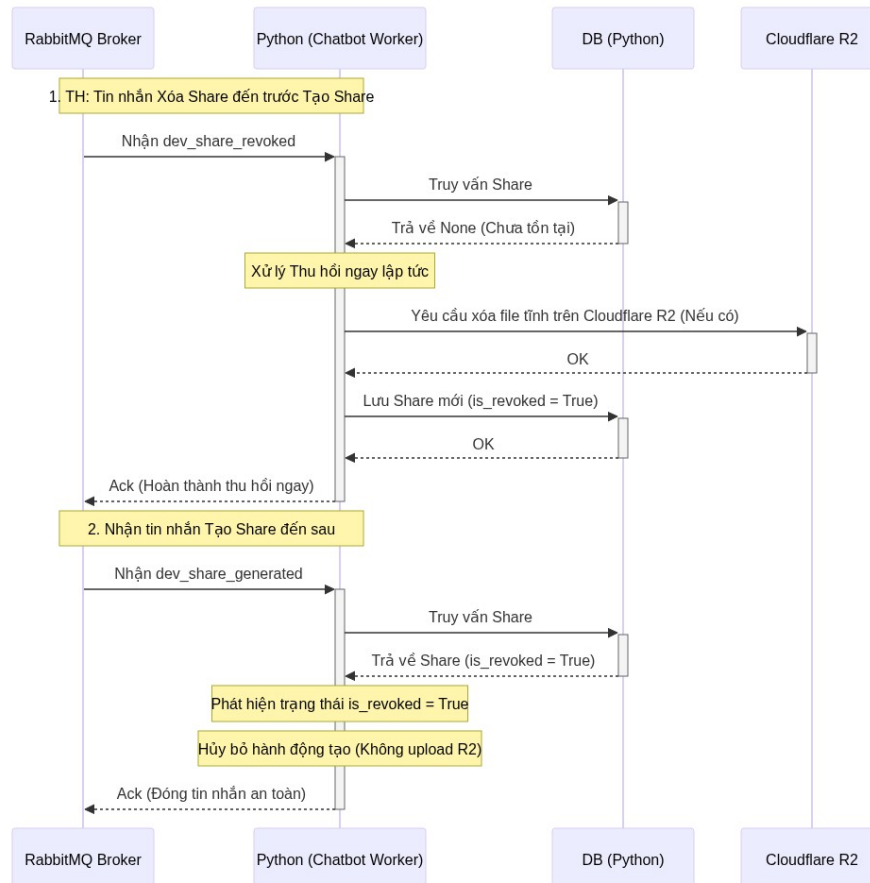
- **Queue:** [prefix]\_share\_upload\_completed
- **Publisher:** code-chatbot-service (Python)
- **Consumer:** code-conversation-service (NestJS)
- **Payload Schema (JSON):**

```
{
 "share_id": "string", // ID liên kết chia sẻ tương ứng
 "file_url": "string", // URL tệp tin JSONL chứa lịch sử chat đã upload R2 (hoặc rỗng nếu 1
 "is_success": true // Trạng thái thành công (true / false)
}
```

- **Cơ chế xử lý:** NestJS nhận thông điệp sẽ tiến hành cập nhật trạng thái của bản ghi chia sẻ thành công hoặc thất bại trong DB (sử dụng Manual Ack để đảm bảo không bị mất dữ liệu).

#### E. Xử lý lệch thứ tự sự kiện Tạo Share và Thu hồi/Xóa Share (Out-of-Order Handling)

- **Tình huống:** Người dùng vừa tạo Share và lập tức thực hiện xóa/thu hồi Share đó. Do tính chất bất đồng bộ của Message Broker, sự kiện Thu hồi Share (share\_revoked) có thể đến Python worker **trước** sự kiện Tạo Share (share\_generated).
- **Cơ chế kiểm tra trạng thái Lũy đẳng (State-based Idempotency):** Vì việc thu hồi liên kết thường liên quan đến vấn đề rõ ràng thông tin nhạy cảm của người dùng, nghiệp vụ yêu cầu lệnh thu hồi phải được thực hiện **ngay lập tức và ưu tiên cao nhất**. Do đó, hệ thống không áp dụng cơ chế tuần tự kiểm tra version phức tạp mà sử dụng cờ trạng thái **is\_revoked** để triệt tiêu sự kiện:



Hình 3.55: Sơ đồ xử lý lệch thứ tự tạo và thu hồi share

### 1. Khi sự kiện Thu hồi Share đến trước:

- Python kiểm tra trạng thái trong DB (chưa có bản ghi nào).
- Tiến hành gọi xóa tệp tin trên Cloudflare R2 (nếu đã được tạo trước đó).
- Lưu bản ghi Share vào DB với trạng thái `is_revoked = True`.
- Phản hồi Ack để hoàn thành tin nhắn.

### 2. Khi sự kiện Tạo Share đến sau:

- Python kiểm tra DB và phát hiện bản ghi đã tồn tại với trạng thái `is_revoked = True`.
- **Cơ chế Triệt tiêu sự kiện (Event Cancellation):** Nhận thấy liên kết đã bị thu hồi trước đó, Python worker lập tức bỏ qua (drop) hoàn toàn hành động tạo và upload tệp lên R2 để bảo vệ tuyệt đối dữ liệu nhạy cảm của người dùng, sau đó gửi Ack để đóng tin nhắn một cách an toàn.

## F. Thiết lập hàng đợi ưu tiên (Priority Queue) trên RabbitMQ

Để đảm bảo các thông điệp thu hồi khẩn cấp được đẩy lên xử lý trước trong trường hợp hàng đợi bị nghẽn (backlog), hệ thống hỗ trợ tích hợp **Priority Queue**:

1. **Khai báo hàng đợi hỗ trợ ưu tiên:** Khi khởi tạo queue trên RabbitMQ, gán đối số cấu hình mức ưu tiên cao nhất (khuyến nghị tối đa là 10 để tránh tốn tài nguyên RAM/CPU):

```
channel.queue_declare(
 queue='[prefix]_share_events_queue',
 arguments={'x-max-priority': 10}
)
```

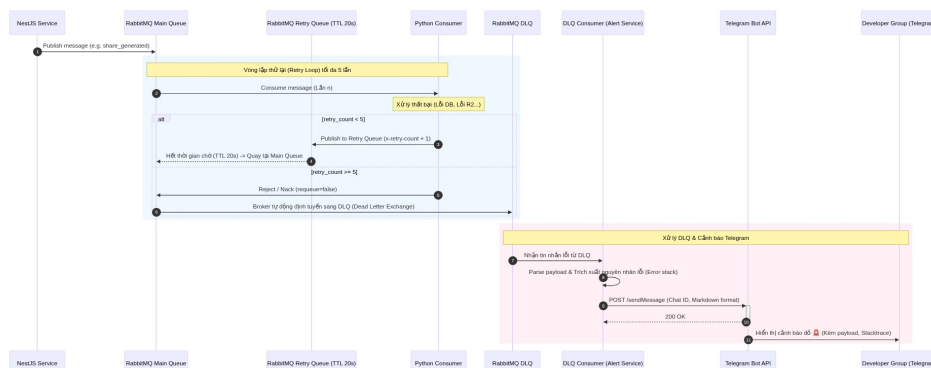
### 2. Gửi sự kiện kèm mức độ ưu tiên (Producer):

- Sự kiện Thu hồi Share (`share_revoked`): Gửi với độ ưu tiên khẩn cấp (`priority = 10`).
- Sự kiện Tạo Share (`share_generated`): Gửi với độ ưu tiên bình thường (`priority = 1`).

3. **Cấu hình QOS cho Consumer:** Thiết lập thuộc tính `prefetch_count = 1` trên consumer để bắt buộc RabbitMQ chỉ gửi đúng 1 thông điệp tại một thời điểm cho worker. Điều này giúp hàng đợi có cơ hội sắp xếp và đẩy các tin nhắn có độ ưu tiên cao (`priority = 10`) lên xử lý trước.

## G. Luồng xử lý hàng đợi thư chết (DLQ) kết hợp Cảnh báo Telegram

- **Mục đích:** Khi một tin nhắn gặp lỗi xử lý lặp đi lặp lại nhiều lần (lỗi logic, lỗi kết nối cơ sở dữ liệu kéo dài, lỗi phân tích cú pháp dữ liệu), hệ thống cần có lập tin nhắn đó để tránh làm nghẽn hàng đợi chính, đồng thời phát cảnh báo tức thời cho đội ngũ vận hành (Developers/DevOps).
- **Quy trình hoạt động:** Sự kết hợp giữa cơ chế Dead Letter Queue (DLQ) của RabbitMQ và Telegram Bot API được thực hiện như sau:



Hình 3.56: Sơ đồ luồng xử lý DLQ và thông báo Telegram

### 1. Ghi nhận và Thử lại (Retry Loop):

- Khi consumer (Python/NestJS) xử lý tin nhắn thất bại, tin nhắn sẽ được gửi sang hàng đợi Retry với độ trễ (TTL) cố định là 20 giây.
- Số lần thử lại (`x-retry-count`) được lưu và tăng dần trong header của tin nhắn.

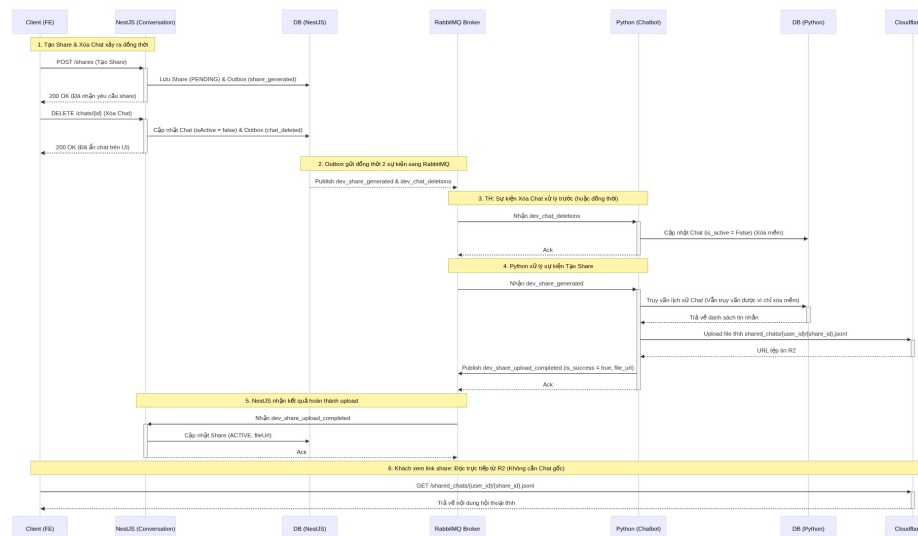
### 2. Chuyển sang DLQ:

- Sau khi thử lại thất bại quá số lần quy định (`MAX_RETRY_COUNT = 5`), consumer sẽ từ chối tin nhắn không điều kiện (`message.reject(requeue=False)`).
- RabbitMQ sẽ tự động chuyển hướng tin nhắn này sang hàng đợi thư chết tương ứng (`_dlq`) dựa trên cấu hình `x-dead-letter-routing-key` lúc khởi tạo queue.

### 3. Tiêu thụ DLQ và Cảnh báo:

- Một consumer chuyên biệt (`process_share_generated_dlq`) sẽ lắng nghe hàng đợi DLQ.
- Khi có tin nhắn đi vào DLQ, consumer này sẽ trích xuất thông tin lỗi, payload của sự kiện, và thời gian xảy ra sự cố.
- Thực hiện gửi yêu cầu HTTP POST tới Telegram Bot API thông qua endpoint: <https://api.telegram.org/bot<TOKEN>/sendMessage>
- Nhóm Telegram của đội ngũ phát triển sẽ lập tức nhận được tin nhắn định dạng Mark-down hiển thị biểu tượng cảnh báo màu đỏ kèm thông tin chi tiết về payload lỗi và stacktrace để nhanh chóng xử lý.

**3. Xử lý concurrency khi Tạo Share và Xóa Chat đồng thời (Concurrency & Snapshot Architecture)** Trong thực tế vận hành, người dùng có thể thực hiện thao tác tạo liên kết chia sẻ (Share) và ngay lập tức xóa cuộc trò chuyện (Delete Chat). Lúc này, cơ chế Outbox sẽ đẩy đồng thời hai sự kiện `share_generated` và `chat_deleted` vào RabbitMQ. Để giải quyết bài toán Race Condition (xung đột tài nguyên) khi thứ tự xử lý trên RabbitMQ không cố định, hệ thống áp dụng mẫu kiến trúc **Snapshot/Static Generation**:



Hình 3.57: Sơ đồ xử lý đồng thời tạo share và xóa chat

#### Cơ chế hoạt động chi tiết:

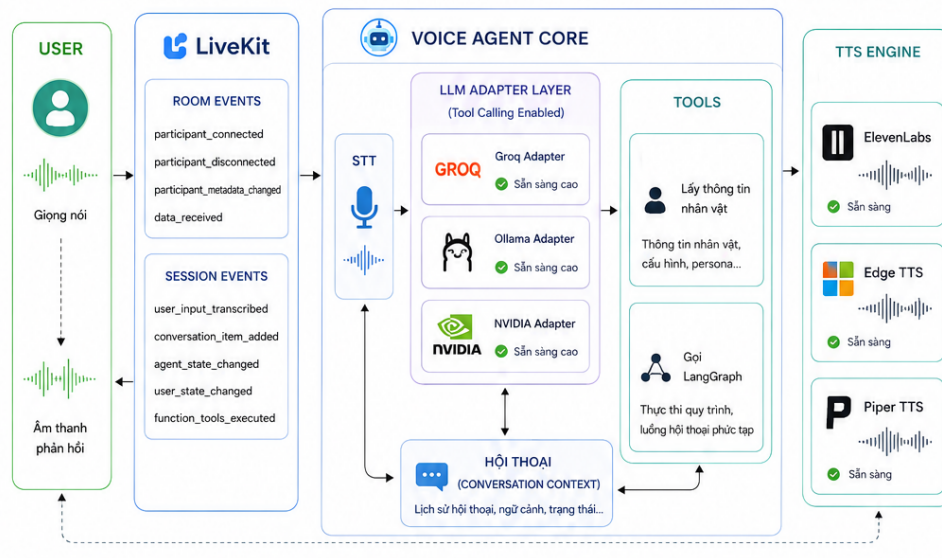
- **Không phụ thuộc vào trạng thái hoạt động của Chat:** Khi NestJS hoặc Python nhận sự kiện xóa chat, họ chỉ thực hiện **xóa mềm (soft delete)** bằng cách cập nhật trường `isActive = false/is_active = False` trong cơ sở dữ liệu. Bản ghi và dữ liệu tin nhắn vẫn tồn tại vật lý trong database.
- **Tạo Snapshot tĩnh (Static Generation):** Dù sự kiện `chat_deleted` có được xử lý trước sự kiện `share_generated` trên Python worker, Python vẫn có thể truy vấn đầy đủ nội dung lịch sử chat đã xóa mềm từ database để đóng gói thành tệp JSONL tĩnh và tải lên Cloudflare R2.

- **Tính độc lập của liên kết chia sẻ:** Sau khi upload thành công lên Cloudflare R2, liên kết chia sẻ trở trực tiếp đến tệp tĩnh này. Khi người nhận xem liên kết chia sẻ, Frontend sẽ fetch trực tiếp tệp tĩnh từ R2 (qua CDN) mà không cần truy vấn vào database gốc (Neon) hay đi qua NestJS. Do đó, nghiệp vụ chấp nhận trạng thái "Có liên kết Share hoạt động nhưng Chat gốc đã bị xóa".

### 3.8.4 Chi tiết chức năng đàm thoại giọng nói (Voice Agent)

Hệ thống thiết lập các bộ chuyển đổi (Adapters) cho phép kết nối linh hoạt với nhiều nhà cung cấp mô hình ngôn ngữ (Groq, Ollama và NVIDIA) có hỗ trợ gọi công cụ (Tool Calling) để đảm bảo tính sẵn sàng cao của hệ thống. AI Agent tích hợp sẵn công cụ lấy thông tin nhân vật (Get Persona) và công cụ gọi luồng xử lý LangGraph để phản hồi câu hỏi. Hệ thống hỗ trợ tự động chuyển đổi công cụ TTS giữa ElevenLabs, Edge TTS và Piper TTS để đáp ứng linh hoạt theo cấu hình nhân vật của người dùng.

Sơ đồ tổng quan luồng tích hợp LiveKit đàm thoại giọng nói:



Hình 3.58: Sơ đồ tích hợp LiveKit đàm thoại giọng nói

Các sự kiện của LiveKit được hệ thống Voice Agent tiếp nhận và xử lý:

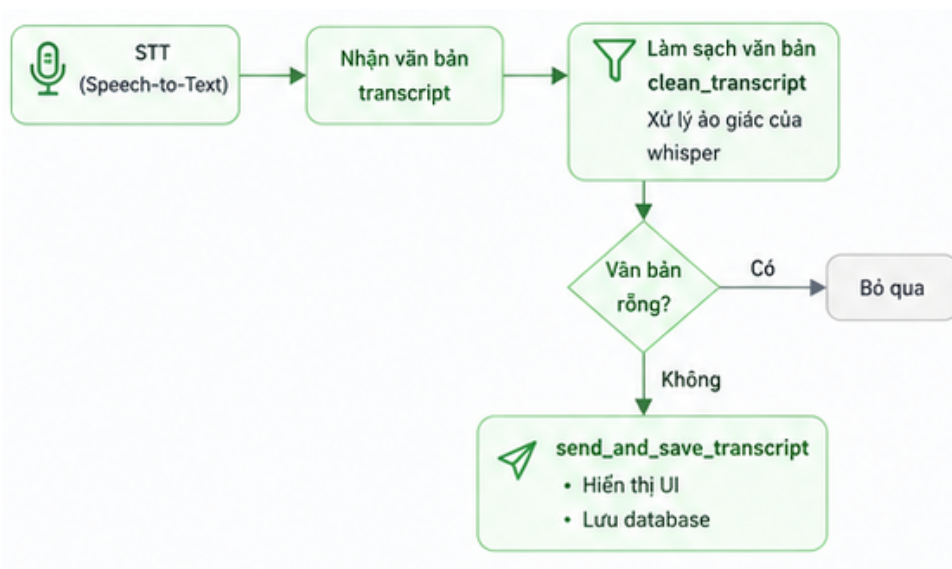
**Sự kiện phòng (Room Events)** Quản lý vòng đời kết nối và sự hiện diện của người dùng trong phòng đàm thoại:

- **participant\_connected:**
  - Kích hoạt khi người dùng tham gia vào phòng đàm thoại thành công.
  - Hệ thống tự động gọi hàm chủ động phát giọng nói chào hỏi người dùng (Lần đầu kết nối: `WELCOME_MESSAGE` = "Xin chào, tôi là trợ lý pháp luật. Bạn có cần tôi giúp đỡ gì không?", Các lần kết nối lại sau đó: `REJOIN_MESSAGE` = "Chào mừng bạn đã quay trở lại. Bạn có cần tôi giúp đỡ gì không?").
- **participant\_disconnected:**
  - Kích hoạt khi người dùng rời khỏi phòng hoặc bị ngắt kết nối đột ngột.
- **participant\_metadata\_changed:**
  - Lắng nghe sự thay đổi siêu dữ liệu (metadata) từ phía người dùng để cập nhật cấu hình giọng đọc, nhân vật mới một cách tức thời.
- **data\_received:**
  - Nhận dữ liệu truyền qua kênh data channel (thường là tin nhắn văn bản định dạng JSON). Cho phép nhà phát triển (developer) gửi tin nhắn văn bản thay cho âm thanh nói để debug luồng xử lý của AI một cách thuận tiện.

**Sự kiện phiên thoại (Session Events)** Quản lý các sự kiện trong luồng đàm thoại giữa người dùng và AI Agent:

- **user\_input\_transcribed:**
  - Nhận văn bản sau khi hệ thống Speech-to-Text (STT) chuyển đổi thành công giọng nói của người dùng.
  - Áp dụng bộ lọc `clean_transcript` để loại bỏ các đoạn văn bản ảo giác của Whisper, bỏ qua nếu chuỗi rỗng, và gọi `send_and_save_transcript` để hiển thị nội dung văn bản tương ứng lên giao diện UI và lưu vào cơ sở dữ liệu.
- **conversation\_item\_added:**
  - Kích hoạt mỗi khi có một tin nhắn mới (từ người dùng hoặc AI Assistant) được thêm vào ngữ cảnh trò chuyện.
  - Đối với phản hồi từ AI Assistant, hệ thống tiến hành trích xuất nội dung tin nhắn, kiểm tra và lọc bỏ các bước suy luận trung gian của LangGraph. Khi có câu trả lời cuối cùng, hệ thống gọi `send_and_save_transcript` để hiển thị trên UI và lưu cơ sở dữ liệu.
- **agent\_state\_changed và user\_state\_changed:**
  - Theo dõi sự thay đổi trạng thái hoạt động (như Speaking, Listening, Thinking) của cả AI Agent và người dùng để đồng bộ hiển thị trạng thái động trên UI.
- **function\_tools\_executed:**
  - Kích hoạt khi AI Agent quyết định gọi một công cụ bên ngoài (Function/Tool Calling) thành công.

Bộ lọc làm sạch văn bản `clean_transcript` xử lý loại bỏ ảo giác trong văn bản dịch từ giọng nói:



Hình 3.59: Bộ lọc làm sạch văn bản `clean_transcript`

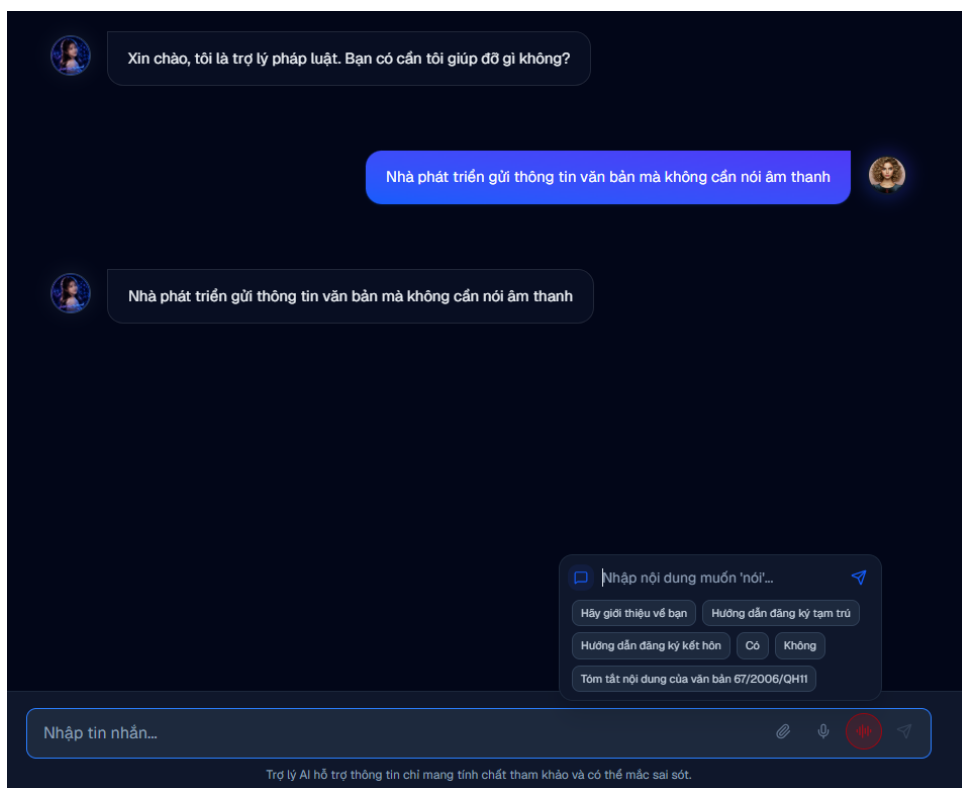
Quản lý các file âm thanh ghi âm trò chuyện lưu trữ trên Cloudflare R2:

[dev-share](#) / [shared\\_chats](#) / c36ce92d-704d-452a-883e-67a984645e4e /

| <input type="checkbox"/> Objects                                                    | Type                         | Storage Class |
|-------------------------------------------------------------------------------------|------------------------------|---------------|
| <input type="checkbox"/> <a href="#">01c2cadb-6450-4534-b173-f21190b8b7a0.jsonl</a> | application/x-ndjson; cha... | Standard      |
| <input type="checkbox"/> <a href="#">01ee2872-36bb-4491-8c4f-283f09e5eee5.jsonl</a> | application/x-ndjson; cha... | Standard      |
| <input type="checkbox"/> <a href="#">35945422-ccbf-4b91-8a0a-00fd1d5d5766.jsonl</a> | application/x-ndjson; cha... | Standard      |
| <input type="checkbox"/> <a href="#">451ad427-7eb1-4d99-95be-578b0e57fc12.jsonl</a> | application/x-ndjson; cha... | Standard      |
| <input type="checkbox"/> <a href="#">5811c0d9-d658-4e8b-9cdd-bf394ec3f10b.jsonl</a> | application/x-ndjson; cha... | Standard      |
| <input type="checkbox"/> <a href="#">5c04c7a3-0ba8-44c6-83d6-9f7dafb1e91e.jsonl</a> | application/x-ndjson; cha... | Standard      |

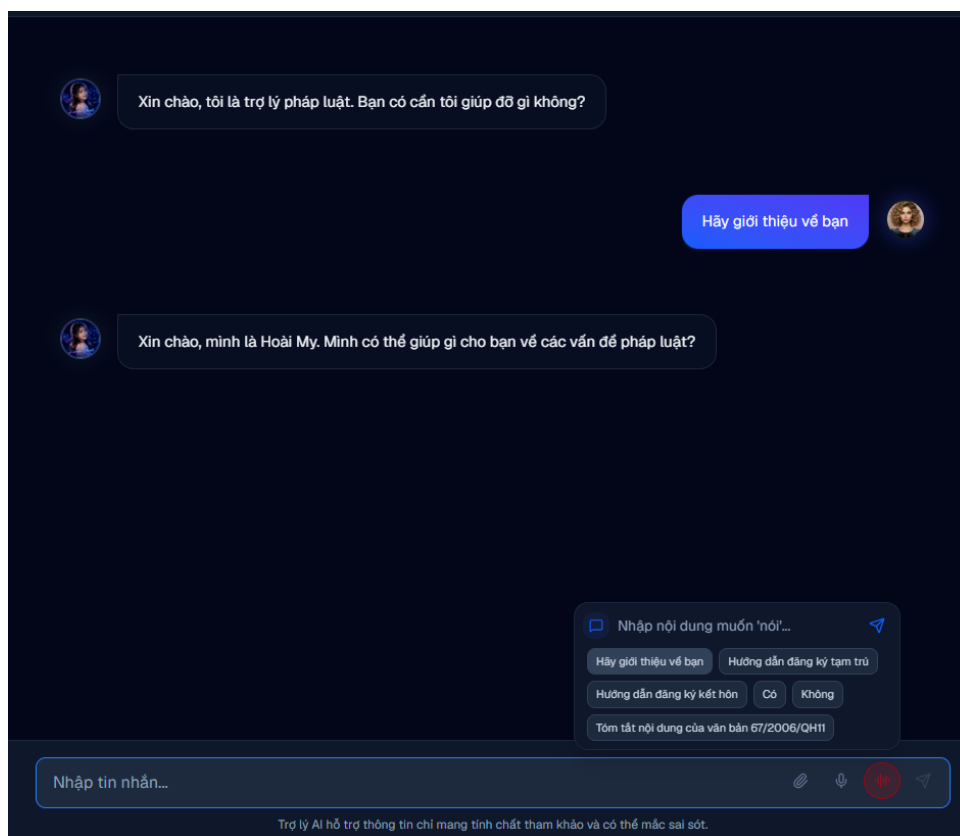
Hình 3.60: Lưu trữ các tệp tin âm thanh chat trên Cloudflare R2

Nhà phát triển gửi thông tin văn bản qua data channel mà không cần nói âm thanh:



Hình 3.61: Developer gửi tin nhắn văn bản thay giọng nói để debug

AI Agent sử dụng công cụ lấy thông tin nhân vật từ hệ thống:



Hình 3.62: Giao diện lấy thông tin nhân vật từ API Gateway

Thông tin log terminal khi AI thực thi công cụ lấy thông tin nhân vật thành công:

```
2026-06-15 01:24:21.512 | DEBUG | app.voice.events.session.on_function_tools_executed:
on_function_tools_executed:5 - AI vừa sử dụng công cụ: type='function_tools_executed' function_calls=[FunctionCall(id='item_e82d4166f61c/fnc_0', type='function_call', call_id='fc_3c032004-6093-48e4-913f-62646d31a182', arguments={}, name='get_persona_info', created_at=1781461461.1885478, extra={}, group_id=None)] function_call_outputs=[FunctionCallOutput(id='item_d388c3411560', type='function_call_output', name='get_persona_info', call_id='fc_3c032004-6093-48e4-913f-62646d31a182', output='Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?', is_error=False, created_at=1781461461.5105371)] created_at=1781461461.5115511
2026-06-15 01:24:23.007 - WARNING livekit.agents - no request_id provided for TTS livekit.plugins.openai.tts.TTS
2026-06-15 01:24:23.327 - WARNING livekit.agents - no request_id provided for TTS livekit.plugins.openai.tts.TTS
2026-06-15 01:24:28.977 - DEBUG livekit.agents - conversation_item_added {"role": "assistant", "text": "Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?"}
2026-06-15 01:24:28.977 | WARNING | app.voice.events.session.on_conversation_item_added:
on_conversation_item_added:45 - Agent (Final): Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?
```

Hình 3.63: Terminal log của việc gọi công cụ lấy thông tin nhân vật

### 3.8.5 Cơ chế Lập lịch Dọn dẹp dữ liệu (Data Cleanup Scheduling)

Vì tính năng xóa trò chuyện và thu hồi liên kết chia sẻ trong hệ thống sử dụng cơ chế **xóa mềm (soft delete)** để đảm bảo tính an toàn dữ liệu và tối ưu hiệu năng của các giao dịch tức thời, hệ thống triển khai các tác vụ lập lịch định kỳ (cron jobs / scheduler) trên cả **NestJS** và **Python** để dọn dẹp triệt để các dữ liệu đã bị xóa sau thời gian lưu trữ quy định.

**1. Phía Dịch vụ trò chuyện (NestJS - code-conversation-service)** NestJS sử dụng module `@nestjs/schedule` để quản lý các cron jobs dọn dẹp các bản ghi không còn hoạt động quá **30** ngày:

- ChatCleanupCron:

- **Mục đích:** Tìm và xóa vĩnh viễn các bản ghi Chat có `isActive = false` và thời gian cập nhật cách hiện tại từ 30 ngày trở lên.
- **Chu kỳ:** Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **ShareCleanupCron:**
  - **Mục đích:** Tìm và xóa vĩnh viễn các bản ghi liên kết Share có `isActive = false` (hoặc đã bị thu hồi) quá hạn 30 ngày.
  - **Chu kỳ:** Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).

**2. Phía Dịch vụ chatbot (Python - code-chatbot-service)** Python sử dụng thư viện APScheduler (AsyncIOScheduler) chạy nền tích hợp trong vòng đời (lifespan) của ứng dụng để tự động dọn dẹp cơ sở dữ liệu lưu trữ lịch sử:

- **run\_chat\_cleanup:**
  - **Mục đích:** Xóa vĩnh viễn các bản ghi Chat trong cơ sở dữ liệu chatbot có trạng thái `is_active = False` quá hạn 30 ngày.
  - **Chu kỳ:** Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **run\_share\_cleanup:**
  - **Mục đích:** Xóa vĩnh viễn các bản ghi Share trong cơ sở dữ liệu chatbot có trạng thái `is_revoked = True` quá hạn 30 ngày.
  - **Chu kỳ:** Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).

## Kết

## luận

Trong quá trình thực tập tại *Công ty TNHH FPT IS*, em thấy mình học được rất nhiều kiến thức về chuyên ngành Toán Tin. Các kiến thức được đề cập trong phạm vi nội dung của bài báo cáo này:

- Ứng dụng mật mã TOTP xác thực đa yếu tố cho hệ thống.
- Ứng dụng mạng máy tính DNS Sinkhole xây dựng máy chủ DNS chặn quảng cáo cấp mạng.
- Sử dụng mã nguồn mở Kong để xây API Gateway cho hệ thống microservices.

Trong quá trình thực tập, em không chỉ học tập tìm hiểu được kiến thức chuyên môn, kiến thức nghiệp vụ bài toán, mà còn được làm quen với môi trường và văn hóa công ty cùng với các đồng nghiệp. Từ đó, em có được kỹ năng làm việc nhóm hiệu quả, kỹ năng quản lý thời gian, ... để em hiểu rõ được lộ trình thăng tiến trong công việc. Điều này giúp em hoạch định được những mục tiêu của bản thân từ đó nắm bắt những cơ hội để tiến xa hơn nữa.

Bài báo cáo này là kết quả học tập và nghiên cứu tìm hiểu trong quá trình em thực tập tại *Công ty TNHH FPT IS* trong suốt một thời gian dài. Dù đã rất cố gắng, tuy nhiên trong quá trình hoàn thành bài báo cáo này do hiểu biết và thời gian của em còn nhiều hạn chế nên bài báo cáo không thể tránh khỏi những thiếu sót và cần bổ sung. Em kính mong sẽ nhận được sự góp ý chân thành từ các thầy cô và mọi người để có thể hoàn thiện bài báo cáo một cách tốt nhất, cũng như hoàn thiện kiến thức của mình hơn nữa.

*Em xin chân thành cảm ơn!*

# Tài liệu tham khảo

## Tài liệu

- [1] <https://vi.wikipedia.org/wiki/FPT>
- [2] Tài liệu đào tạo hội nhập của Công ty TNHH FPT IS
- [3] <https://fpt-is.com/san-pham-a-z>
- [4] <https://fastapi.tiangolo.com>
- [5] <https://docs.sqlalchemy.org/en/20>
- [6] <https://alembic.sqlalchemy.org/en/latest/index.html>
- [7] <https://www.rfc-editor.org/rfc/rfc7519>
- [8] <https://pyjwt.readthedocs.io/en/stable>
- [9] <https://pyauth.github.io/pyotp>
- [10] <https://www.rfc-editor.org/rfc/rfc3174>
- [11] <https://www.rfc-editor.org/rfc/rfc2104>
- [12] <https://www.rfc-editor.org/rfc/rfc4226>
- [13] <https://www.rfc-editor.org/rfc/rfc6238>
- [14] <https://bind9.readthedocs.io/en/v9.20.4>
- [15] <https://microservices.io>
- [16] <https://docs.konghq.com/gateway/3.8.x>



Chữ tạm màu đỏ  
 Dòng thứ hai vẫn có màu đỏ và không bị lỗi ngắt đoạn.  
<https://example.com>



Hình 3.64: Logo của Đại học Bách Khoa Hà Nội

- Nội dung mục 1
- Chấm màu xanh

*In nghiêng đoạn văn bản*

**In đậm đoạn văn bản**

Ứng dụng trong đồ án

Nội dung block

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.